Azure-Datenplattform Schritt für Schritt

AN-ITS-001 · Arbeitsanweisung für Mitarbeitende – vom leeren Azure-Konto bis zur automatischen Datenpipeline mit Bereinigung, Wetter und Gold-Tabelle (Anwendungsfall Bee Haven)

Version 3.0 · Stand 06.10.2026 · Grundlage: Pflichtenheft PH-ITS-002 · erstellt mit KI-Unterstützung (Claude)

> **Warum?** Diese Anleitung führt durch das **ganze Projekt** in der Reihenfolge, in der es gebaut wird: Azure einrichten (Phase 1), Daten mit Notebooks bereinigen und zu Gold zusammenführen (Phase 2), die Kette automatisieren und absichern (Phase 3). Jeder Schritt hat Begründung, Bild, Prüfung und typische Fehler – so können auch Einsteiger ihn nachbauen.

Inhaltsverzeichnis

So liest du diese Anleitung 2
Überblick: das ganze Projekt in drei Phasen 3
Phase 1 – Azure einrichten: Überblick 4
Schritt 0 – Voraussetzungen und Kostenkontrolle 5
Schritt 1 – Create Resource Group (Ressourcengruppe erstellen) + Tags 5
Schritt 2 – Storage Account (Speicherkonto) Quelle: mabeehavenraw 8
Schritt 3 – Storage Account Data Lake: mabeehavenlakehouse 9
Schritt 4 – Container lakehouse und Ordner anlegen 10
Schritt 5 – Create Data Factory (Data Factory erstellen) 11
Schritt 6 – Access control (IAM) (Zugriffssteuerung): Rechte für die Data Factory 13
Schritt 7 – Launch Studio (Studio starten) 16
Schritt 8 – Manage (Verwalten) → Linked services (Verknüpfte Dienste) 17
Schritt 9 – Author (Erstellen) → Datasets (Datensätze) 19
Schritt 10 – Pipeline pl_ingest_raw_to_bronze: Rohdaten nach Bronze 20
Schritt 11 – Dynamische Pipeline: Get Metadata + ForEach 22
Schritt 12 – Bronze → Silver (CSV, Ordner je Sensor) 24
Schritt 13 – Archiv mit Zeitstempel (Historie statt Überschreiben) 28
Schritt 14 – Silver aufteilen: processing (Arbeitskorb) und cleaned (bereinigt) 30
Phase 2 – Notebooks: Überblick 34
Schritt 15 – VS Code: Projektordner öffnen und vertrauen 35
Schritt 16 – Python-Umgebung „beehaven“ mit Conda anlegen 36
Schritt 17 – Notebook öffnen und Kernel wählen 37
Schritt 18 – N1 Bereinigung (technisch) 37
Schritt 19 – N2 Datenqualität (fachlich) 41
Schritt 20 – N3 Wetter und Sonnenstand 43
Schritt 21 – N4 Gold: eine Tabelle je Stunde 45
Schritt 22 – N5 Analyse (bei Bedarf) 47
Schritt 23 – Steuer-Notebook und Vorbereitung für Azure 48
Phase 3 – Automatisierung und Betrieb: Überblick 50
Schritt 24 – Master-Pipeline pl_daily_processing 50
Schritt 25 – Standort-Tabelle in den Data Lake laden 52
Schritt 26 – Budget-Warnung einrichten 53
Schritt 27 – Testlauf und Gegenprobe 55
Schritt 28 – Synapse-Arbeitsbereich und Spark-Pool anlegen 56
Schritt 29 – Notebooks in Synapse importieren und testen 61
Schritt 30 – Data Factory an Synapse anbinden (verknüpfter Dienst, Rollen, Notebook-Aktivität) 63
Schritt 31 – Erfolgspfad: Arbeitskorb nach erfolgreichem Lauf leeren 67
Schritt 32 – Gold-Pipeline pl_silver_to_gold (je Stadt aus der Standort-Tabelle) 69
Schritt 33 – Gesamtlauf pl_daily_processing (Abnahme Bronze → Silver → Gold) 71

Schritt 34 – Zweite Stadt (Würzburg) und Standortregeln aus der Konfiguration 73
Schritt 35 – Zeitplan-Trigger, Nebenläufigkeit und Fehler-E-Mail (Betrieb absichern) 77
Ausblick – nächste Schritte 82
Häufige Fragen (FAQ) 82
Anhang A – Typische Fehler Azure 85
Anhang B – Typische Fehler Notebooks 86
Anhang C – Projektbezogene Befehle 87
Anhang D – Quellen 89
Anhang E – Alternative Wege (was wir gewählt haben, was ginge auch, warum so) 90
Anhang F – Begriffe (Glossar) 91

Tipp: Eintrag oder Seitenzahl anklicken (Word: Strg/cmd + Klick) springt zum Schritt. Fragt Word beim Öffnen „Felder aktualisieren?“ → **Ja** (Seitenzahlen werden neu berechnet).

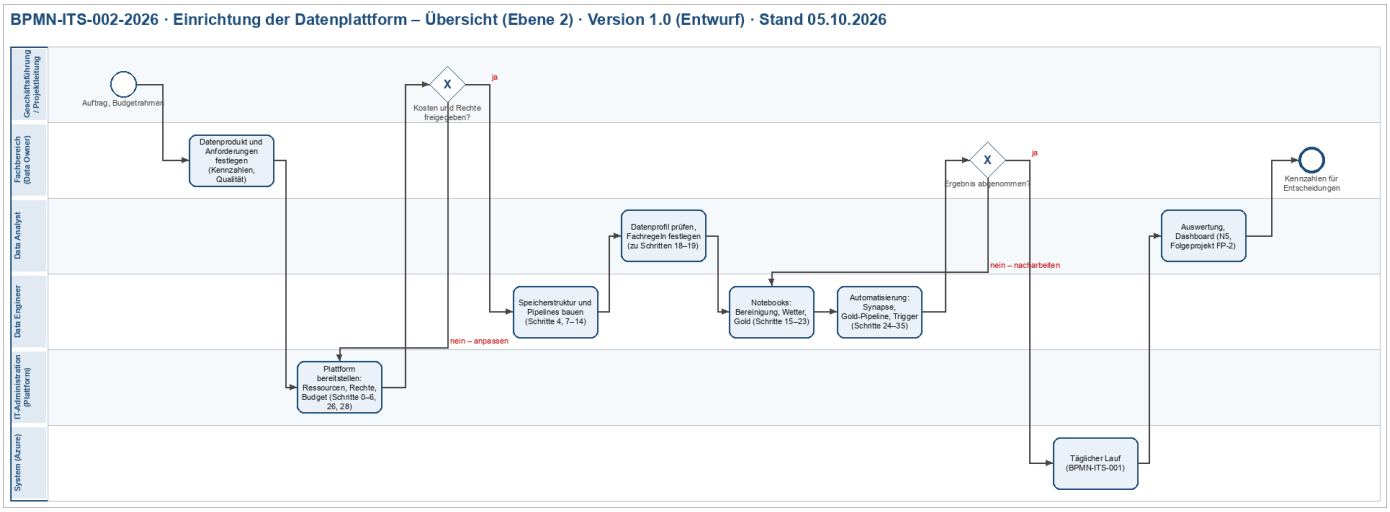


Abb. 1: Ablauf auf einen Blick (Prozessmodell Ebene 2): drei Phasen, zwei Freigaben, danach der tägliche Lauf.

So liest du diese Anleitung

- **Ein nummerierter Punkt = eine Handlung.** Das Bild dazu steht direkt darunter.
- **Englischer Begriff zuerst, deutscher Begriff in Klammern**, z. B. **Go to resource (Zu Ressource wechseln)**. Das Portal war auf Deutsch eingestellt; die Übersetzungen sind teils uneinheitlich (z. B. „Azurblau“ statt „Azure“). Im Zweifel gilt der englische Begriff.
- **Fett** = Schaltfläche, Reiter oder Menüpunkt zum Anklicken. `Grau hinterlegt` = Name oder Ausdruck zum Eintippen (genau so schreiben).
- Rote Rahmen und Pfeile in den Bildern zeigen, wohin du klicken sollst.
- Datenschutz: Name, E-Mail-Adresse, Mandant und Abonnement-ID sind in allen Bildern unkenntlich gemacht („USERNAME“, „xxxx“).
- Du findest einen Dienst nicht? Oben im Portal in die Suchleiste tippen.

Die farbigen Kästen bedeuten:

> **Warum?** Begründung – damit du verstehst, wozu der Schritt dient.

> ⚠️ **Achtung:** typischer Fehler oder Stolperfalle – hier bitte langsam lesen.

> 💡 **Gut zu wissen:** Hintergrundwissen und Tipps aus der Praxis.

> ✅ **Prüfen:** So kontrollierst du, ob der Schritt geklappt hat.

Überblick: das ganze Projekt in drei Phasen

Phase	Inhalt	Schritte	Ergebnis
1 – Azure einrichten	Ressourcengruppe, Speicherkonten, Data Lake, Data Factory, Rechte, Pipelines, Archiv	0–14	Rohdaten laufen automatisch nach Bronze und Silver
2 – Notebooks	Arbeitsumgebung, Bereinigung, Qualität, Wetter, Gold, Analyse, Steuer-Notebook	15–23	bereinigte Daten, Gold-Tabelle je Stunde, Grafiken
3 – Automatisierung und Betrieb	Master-Pipeline, Standort-Tabelle, Budget-Warnung, Testlauf mit Gegenprobe, Synapse, Data Factory ↔ Synapse, Erfolgspfad, Gold-Pipeline, Gesamtlauf, zweite Stadt, Trigger und Warnungen	24–35	eine Kette mit einem Start Bronze → Silver → Gold, Kosten im Blick

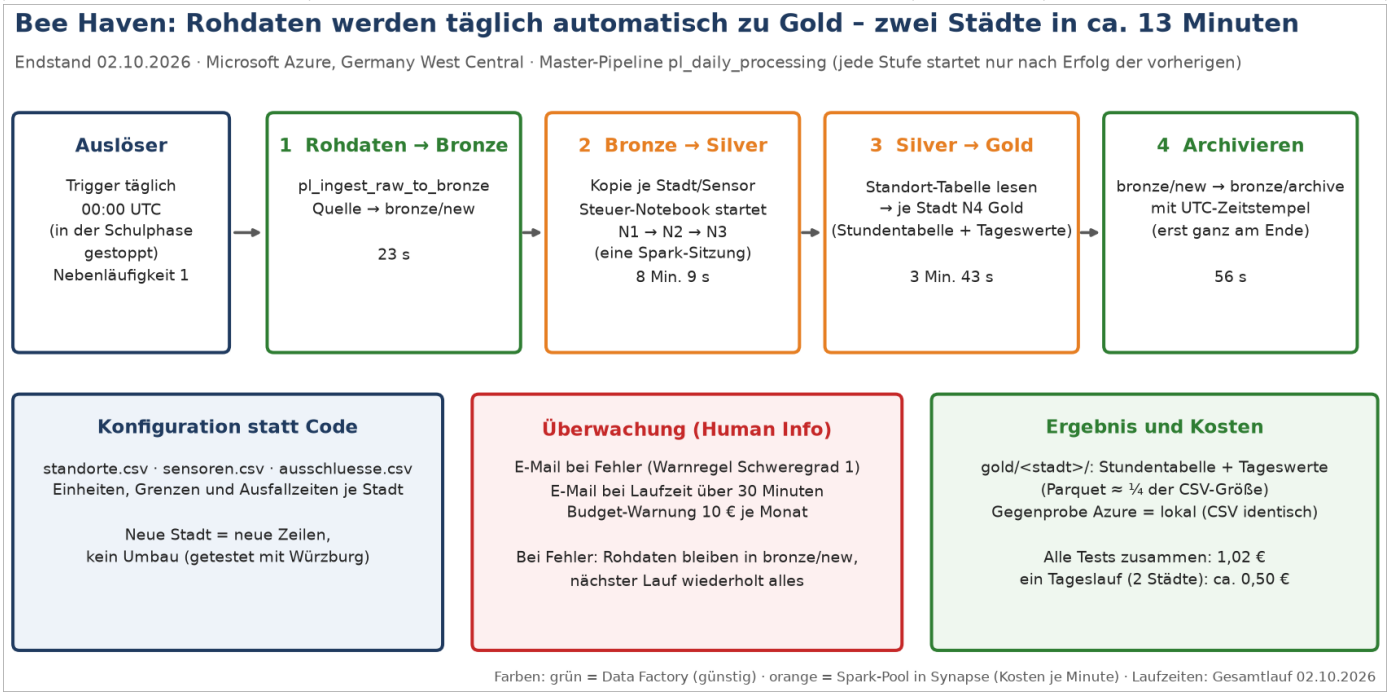


Abb. 2: Fertige Pipeline (Endstand 02.10.2026): täglich automatisch von Rohdaten bis Gold, zwei Städte in ca. 13 Minuten.

> 💡 **Gut zu wissen:** **Einfach erklärt:** Phase 1 baut die Küche (Räume, Leitungen, Zugänge), Phase 2 schreibt die Rezepte (Notebooks), Phase 3 stellt den Küchenplan auf, damit jeden Tag automatisch gekocht wird – mit einem Blick aufs Haushaltsbuch.

Phase 1 – Azure einrichten: Überblick

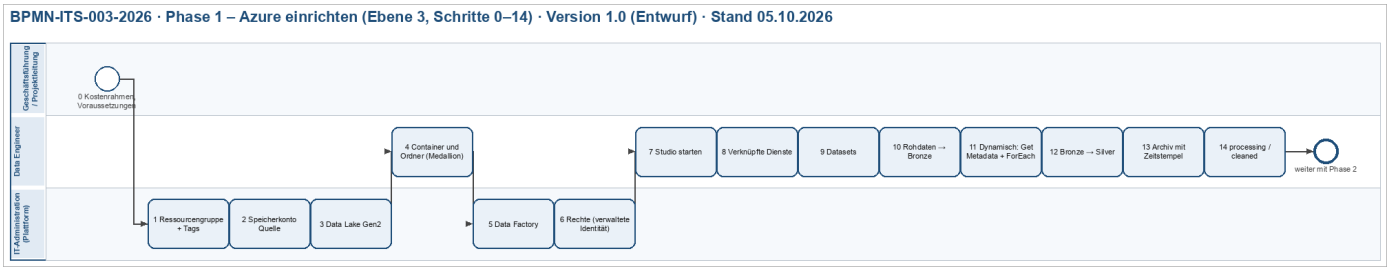


Abb. 3: Phase 1 im Überblick (Ebene 3): Schritte 0–14 nach Rollen.

Rohdaten (4 CSV-Dateien mit Sensordaten eines Bienenstocks in Schwartau) werden von einem Quell-Speicher automatisch per Azure Data Factory in einen Data Lake übertragen und dort in Stufen veredelt (ELT: Extract, Load, Transform – erst laden, dann umwandeln).

Ressource EN (DE)	Name	Zweck
Resource Group (Ressourcengruppe)	`rg-beehaven-dev-gwc`	Projektordner für alle Ressourcen, Region Germany West Central
Storage Account (Speicherkonto) – Quelle	`mabeehavenraw`	einfacher Blob-Speicher, Container source-data mit 4 CSV („Posteingang“)
Storage Account – Data Lake	`mabeehavenlakehouse`	ADLS Gen2, Container lakehouse: bronze/new, bronze/archive, silver, gold
Data Factory	`adf-beehaven-dev-gwc`	Pipelines: holen, verteilen, archivieren

Pipeline	Von → Nach	Schritt
`pl_ingest_raw_to_bronze`	source-data → bronze/new (Rohdaten 1:1)	10
`pl_bronze_to_silver`	bronze/new → silver/<sensor>/ (CSV, Ordner je Sensor)	12
`pl_bronze_new_to_archive`	bronze/new → bronze/archive/<name>_raw/ <name>_raw__<Zeitstempel>_UTC.csv	11 + 13

> 💡 **Gut zu wissen:** Reihenfolge im Betrieb: 1. Ingest → 2. Bronze nach Silver → 3. Archivieren (verschieben). Erst archivieren, wenn Silver die Dateien hat – sonst ist bronze/new schon leer. Die Master-Pipeline (Phase 3, Schritt 24) übernimmt diese Reihenfolge automatisch.

Begriffe: Speicherkonto – Container – Ordner

Ebene	Beispiel	Vergleich
Storage Account (Speicherkonto)	`mabeehavenlakehouse` = ma (Kürzel) + beehaven (Projekt) + lakehouse (Zweck)	das Gebäude
Container (Container)	`lakehouse` bzw. `source-data`	ein Raum im Gebäude
Directory (Verzeichnis/Ordner)	`bronze/new`, `bronze/archive`, `silver`, `gold`	Regale im Raum

Medallion-Architektur: Bronze – Silver – Gold

Stufe	Inhalt	Vergleich Küche
Bronze	Rohdaten 1:1, unverändert (new = noch nicht verarbeitet, archive = verarbeitet, mit Zeitstempel)	eingekaufte Zutaten
Silver	bereinigt: Duplikate, Messfehler, Zeitzonen, Datentypen	gewaschen und geschnitten
Gold	fertige Kennzahlen, z. B. Gewicht und Flugaktivität pro Tag – für Power BI und Machine Learning (nur Lesezugriff)	das servierte Gericht

> 💡 **Gut zu wissen:** **Warum ordnen wir die Ordner nach dem Medallion-Prinzip?** (1) **Nachverfolgbarkeit:** Fällt in Gold ein falscher Wert auf, lässt er sich über Silver bis zu den unveränderten Rohdaten in Bronze zurückverfolgen – die Fehlerursache wird sichtbar. (2) **Kontrollpunkte:** Jede Stufe hat eine klare Aufgabe (roh → geprüft → angereichert) und

kann einzeln geprüft werden. (3) **Skalierbarkeit:** Neue Quellen – z. B. Wetterdaten oder weitere Bienenstock-Standorte – kommen in Bronze dazu, ohne das Gesamtsystem umzubauen. (4) **Vertrauen:** Analysen, Berichte (Power BI) und Machine Learning bauen nur auf geprüften Gold-Daten auf. Quelle: Kursunterlagen WBS „Das Medallion Lakehouse“, Microsoft Learn „What is the medallion lakehouse architecture?“.

Quelle (Beispiele)	→ Bronze (Raw)	→ Silver (Validated)	→ Gold (Enriched)	→ Nutzung
Datenbank, IoT-Sensoren, CSV, Parquet	unverändert	geprüft, bereinigt	Kennzahlen	Reports (Power BI), ML

Schritt 0 – Voraussetzungen und Kostenkontrolle

💡 **Gut zu wissen:** **Hintergrund:** Cloud-Dienste kosten nach Verbrauch (Pay-as-you-go). Wer vorher prüft, was er braucht, und die Kosten im Blick hat, erlebt keine Überraschung auf der Rechnung. Prüfungsthema DP-900: CapEx (Investition vorab) vs. OpEx (laufende Kosten).

Warum? Die Cloud kostet Geld, solange Ressourcen existieren. Ein Blick auf Guthaben und Ablaufdatum verhindert böse Überraschungen.

1. Im **Azure Portal (Azure-Portal)** anmelden: `portal.azure.com`.
2. Auf der Startseite **Home (Startseite)** das verbleibende Guthaben und das Ablaufdatum ansehen (im Bild rot markiert: Restguthaben 171,73; daneben das Ablaufdatum des Guthabens).

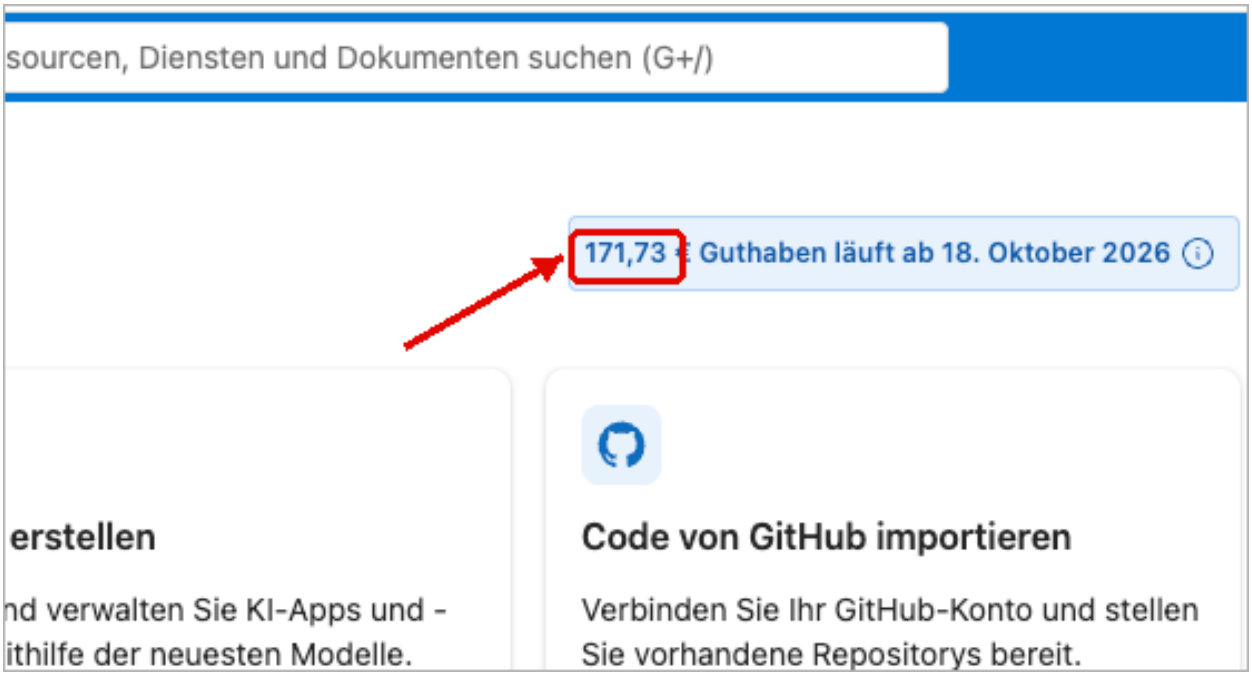


Abb. 4: Startseite: verbleibendes Testguthaben und Ablaufdatum im Blick behalten.

3. Einige Tage vor Ablauf eine Erinnerung in den Kalender eintragen.
4. Empfohlen: **Cost Management (Kostenverwaltung)** → **Budgets** → **+ Add (Hinzufügen)** → Budget mit Warnungen bei 50 %, 80 % und 100 % anlegen.

💡 **Gut zu wissen:** Ohne Upgrade wird das kostenlose Konto (Free account) nach Ablauf nur deaktiviert – es entstehen keine Kosten. Unser Aufbau (wenige MB Speicher, einige Pipeline-Läufe) kostet ohnehin nur Cent-Beträge.

✅ **Prüfen:** Du kennst dein Restguthaben und das Ablaufdatum.

Schritt 1 – Create Resource Group (Ressourcengruppe erstellen) + Tags

💡 **Gut zu wissen:** **Hintergrund:** Eine **Ressourcengruppe** ist ein Ordner für alle Azure-Teile eines Projekts. Rechte, Kosten und Löschen lassen sich so auf einen Schlag steuern. **Tags** (Schlagworte) zeigen später in der Kostenanalyse, wofür das Geld ausgegeben wurde.

> **Warum?** Die Ressourcengruppe ist der Projektordner: Alles zu Bee Haven liegt darin – Überblick, Kosten gesammelt, am Ende gemeinsam entfernbar.

1. Oben in die Suchleiste **Resource groups (Ressourcengruppen)** tippen und öffnen.
2. Auf **+ Create (Erstellen)** klicken.
3. Im Feld **Subscription (Abonnement)** dein Abonnement wählen, im Feld **Resource group (Ressourcengruppenname)** `rg-beehaven-dev-gwc` eintippen und im Feld **Region (Europe) Germany West Central** wählen.

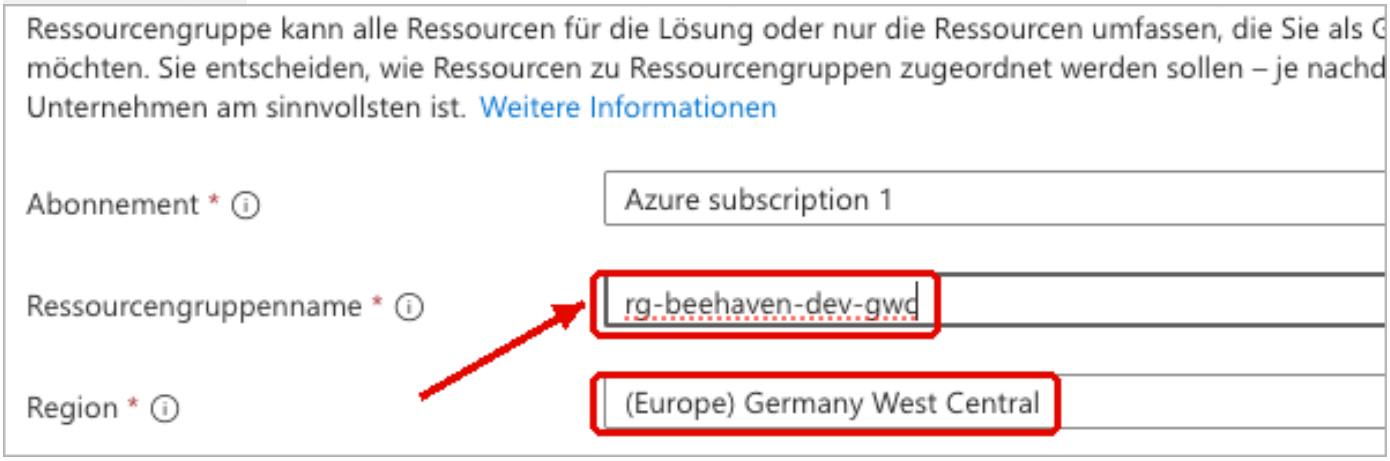


Abb. 5: Resource group (Ressourcengruppe) anlegen: Name nach Konvention und Region Germany West Central.

> ⚠ **Achtung:** Die Region steht oft auf „(US) East US“ – immer aktiv umstellen.

4. Den Reiter **Tags** im Assistenten überspringen → **Review + create (Überprüfen + erstellen)**: In der Zusammenfassung stehen Abonnement, `rg-beehaven-dev-gwc` und Germany West Central → **Create (Erstellen)**. Die Gruppe ist sofort angelegt.



Abb. 6: Überprüfen und erstellen: Zusammenfassung prüfen → Erstellen. (Ausschnitte 1–2)

5. Die neue Gruppe `rg-beehaven-dev-gwc` öffnen → links im Menü **Tags (Markierungen)**: je Zeile links unter **Name** den Namen und rechts unter **Wert** den Wert eintragen – `projekt` = `beehaven`, `umgebung` = `dev`, `verantwortlich` = `projektleitung`, `zweck` = `datenanalyse` (alles kleingeschrieben). Warum erst jetzt: An der fertigen Gruppe sieht man die Tags danach direkt in der Übersicht und kann sie jederzeit ändern.



Abb. 7: Menü Tags der Ressourcengruppe: vier Namen projekt, umgebung, verantwortlich, zweck (Werte rechts daneben).

6. **Apply (Übernehmen)** → unten prüfen, ob die vier Tags an `rg-beehaven-dev-gwc` stehen (z. B. `zweck : datenanalyse`).



Abb. 8: Tags an der Ressourcengruppe sind gespeichert.

Namensteil	Bedeutung
rg	Resource Group (Ressourcengruppe)
beehaven	Projekt
dev	Development (Lern-/Testumgebung); im Betrieb zusätzlich -prod
gwc	Germany West Central (Frankfurt)

💡 **Gut zu wissen:** Namenskonvention nach Microsoft Cloud Adoption Framework: <Typ-Kürzel>-<Projekt>-<Umgebung>-<Region>. Ressourcengruppen lassen sich nicht umbenennen – falsche Namen früh korrigieren (Ressourcen per **Move (Verschieben)** in eine neue Gruppe umziehen, Daten bleiben erhalten).

> 💡 **Gut zu wissen:** Tags werden nicht vererbt: Jede Ressource braucht eigene Tags. Mehrere Ressourcen auf einmal: Kästchen anhaken → **Assign tags (Tags zuweisen)**.

> ⚠️ **Achtung:** Keine Personennamen in Tags – Azure speichert Tags weltweit (auch außerhalb der EU). Rollen statt Personen verwenden und klein schreiben. Beim Löschen alter Gruppen den Namen zweimal lesen: Das Löschen einer Gruppe löscht alles darin.

> ✅ **Prüfen:** `rg-beehaven-dev-gwc` steht in der Liste mit Standort Germany West Central und 4 Tags.

Schritt 2 – Storage Account (Speicherkonto) Quelle: mabeehavenraw

> 💡 **Gut zu wissen:** **Hintergrund:** Ein **Speicherkonto** ist der Ablageort in der Cloud. Hier simuliert es die Quelle, aus der neue Rohdaten kommen (z. B. eine Lieferung von Sensoren). Quelle und Verarbeitung sind getrennt, damit Rohdaten nie versehentlich verändert werden.

> **Warum?** Simuliert die Realität: Daten kommen aus einem anderen System (z. B. ERP-Export, Sensoren). Das Quell-Speicherkonto ist dieser „Posteingang" – absichtlich ein einfacher Blob-Speicher ohne Ordnerstruktur.

1. Die Ressourcengruppe `rg-beehaven-dev-gwc` öffnen → **+ Create (Erstellen)**.
2. **Storage account (Speicherkonto)** suchen → **Create (Erstellen)**.
3. Reiter **Basics (Grundlagen)**: Name `mabeehavenraw`, Region **Germany West Central**, **Performance** = **Standard**, **Redundancy (Redundanz)** = **LRS**.
4. Reiter **Advanced (Erweitert)**: **Enable hierarchical namespace (Hierarchischen Namespace aktivieren)** NICHT anhaken → **Review + create** → **Create**. In der Übersicht prüfen: Redundanz **Lokal redundanter Speicher (LRS)**, Hierarchischer Namespace **Deaktiviert**. Warum deaktiviert: Die Quelle ist bewusst ein einfacher Blob-Speicher ohne Ordnerstruktur – wie ein Posteingang, in den ein fremdes System Dateien legt. Echte Ordner braucht erst das Lakehouse (Schritt 3). Warum LRS: drei Kopien in einem Rechenzentrum, die günstigste Redundanz – reicht für Lern- und Testdaten.

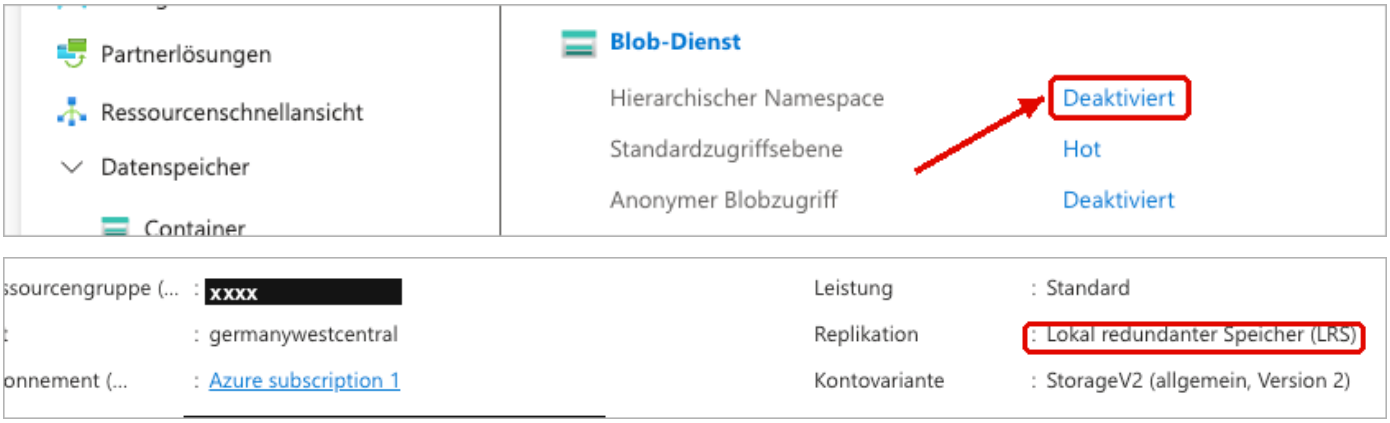


Abb. 9: Quelle mabeehavenraw: Hierarchischer Namespace deaktiviert (einfacher Blob-Speicher), LRS. (Ausschnitte 1–2)

5. Im Speicherkonto: **Containers (Container)** → **+ Container** → im Feld **Name** `source-data` eintragen. Das Feld **Anonyme Zugriffsebene** steht fest auf **Privat (kein anonymer Zugriff)** und ist grau – nichts ändern; so kann niemand ohne Anmeldung die Rohdaten lesen → **Create**.

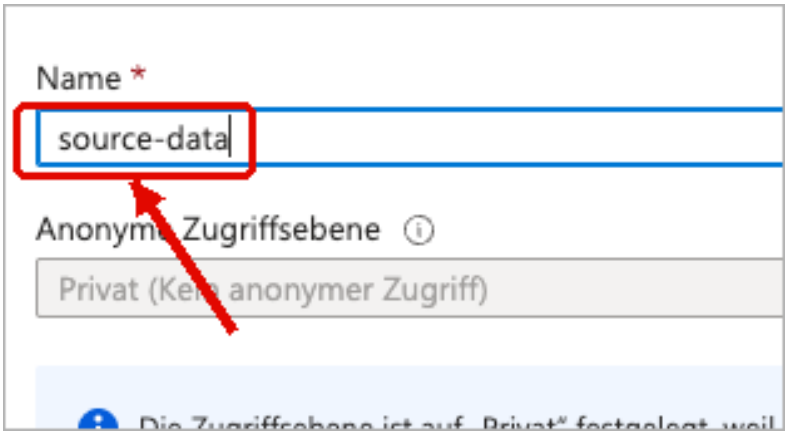


Abb. 10: Container source-data anlegen (Zugriffsebene Privat).

> ⚠️ **Achtung:** Container nach Zweck benennen (`source-data`), nicht nach Projekt. Container lassen sich nicht umbenennen.

6. `source-data` öffnen → **Upload (Hochladen)** → die 4 Dateien `flow_schwartau.csv`, `humidity_schwartau.csv`, `temperature_schwartau.csv`, `weight_schwartau.csv` wählen → **Upload**.



Abb. 11: Die 4 CSV-Dateien sind hochgeladen.

> 💡 **Gut zu wissen:** Speicherkontonamen: nur Kleinbuchstaben und Ziffern, 3–24 Zeichen, weltweit eindeutig. LRS (lokal redundant) = 3 Kopien in einem Rechenzentrum – die günstigste Option, reicht zum Lernen. Im Betrieb für wichtige Daten ZRS/GRS prüfen.

> ✅ **Prüfen:** Im Container `source-data` liegen 4 CSV-Dateien (flow ca. 54 MiB, temperature ca. 6,2 MiB, humidity/weight ca. 45–50 KiB).

Schritt 3 – Storage Account Data Lake: mabeehavenlakehouse

> 💡 **Gut zu wissen:** **Hintergrund:** Der **Data Lake Gen2** ist ein Speicherkonto mit **hierarchischem Namespace** (echten Ordnern). Spark, Synapse und Data Factory arbeiten damit schneller, und Rechte lassen sich je Ordner vergeben. Ein normaler Blob-Speicher kennt nur flache Dateinamen.

> **Warum?** Hier werden die Daten gesammelt, veredelt und analysiert. Der hierarchische Namespace macht daraus einen Data Lake (ADLS Gen2) mit echten Ordnern und Rechten je Ordner – Voraussetzung für die Medallion-Architektur.

1. Wieder in der Ressourcengruppe: **+ Create (Erstellen)** → **Storage account (Speicherkonto)**.
2. Name `mabeehavenlakehouse`, Region **Germany West Central**, **Redundancy** = **LRS**.
3. Reiter **Advanced (Erweitert)**: **Enable hierarchical namespace** diesmal ANHAKEN.

> ⚠️ **Achtung:** Vor **Create** prüfen: Nachträglich geht nur ein einmaliges Upgrade (nicht umkehrbar); ausschalten lässt sich der Namespace nie mehr.

4. **Review + create** → **Create** → in der **Overview (Übersicht)** kontrollieren: **Hierarchischer Namespace** = **Aktiviert**. Warum: Erst damit wird das Konto zum Data Lake (Gen2) mit echten Ordnern – nötig für `bronze`, `silver`, `gold` und für Rechte je Ordner.

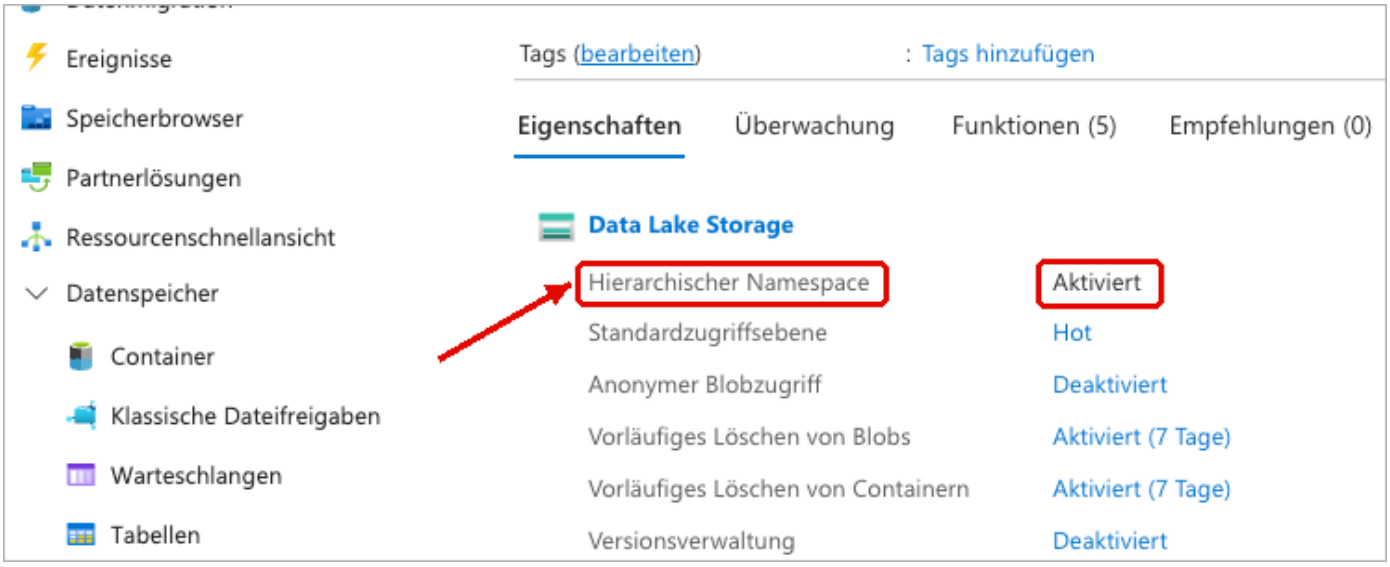


Abb. 12: Data Lake mabeehavenlakehouse: Hierarchischer Namespace aktiviert = ADLS Gen2.

> ✅ **Prüfen:** Übersicht zeigt „Hierarchical namespace (Hierarchischer Namespace): Enabled (Aktiviert)“.

Schritt 4 – Container lakehouse und Ordner anlegen

> 💡 **Gut zu wissen:** **Hintergrund:** Die Ordner folgen dem **Medallion-Prinzip**: Bronze = Rohdaten unverändert, Silver = bereinigt und geprüft, Gold = für Auswertung und Dashboard aufbereitet. So bleibt jeder Zwischenstand nachvollziehbar.

> **Warum?** Dateien liegen immer in einem Container. Die Ordner bilden die Qualitätsstufen ab: Rohdaten bleiben unverändert, bereinigte Daten und Kennzahlen sind klar getrennt.

1. `mabeehavenlakehouse` → **Containers** → **+ Container** → im Feld **Name** `lakehouse` eintragen; **Anonyme Zugriffsebene** bleibt **Privat** → **Create**.

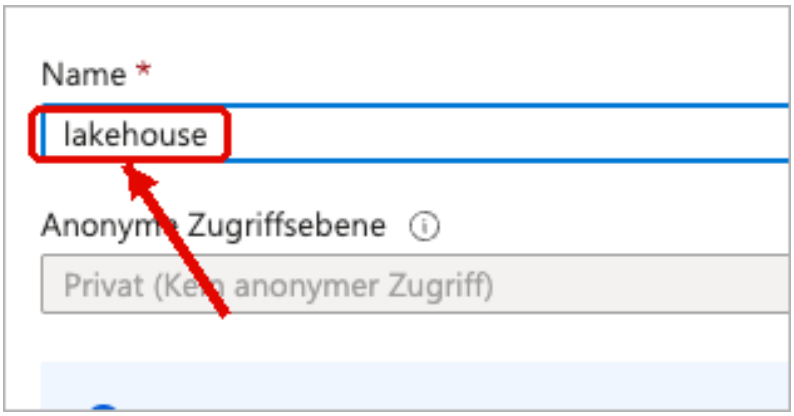


Abb. 13: Container lakehouse anlegen – Name nach Zweck, ohne Kürzel.

2. `lakehouse` öffnen → **+ Add Directory (Verzeichnis hinzufügen)** → Name `bronze` → **Save (Speichern)**.

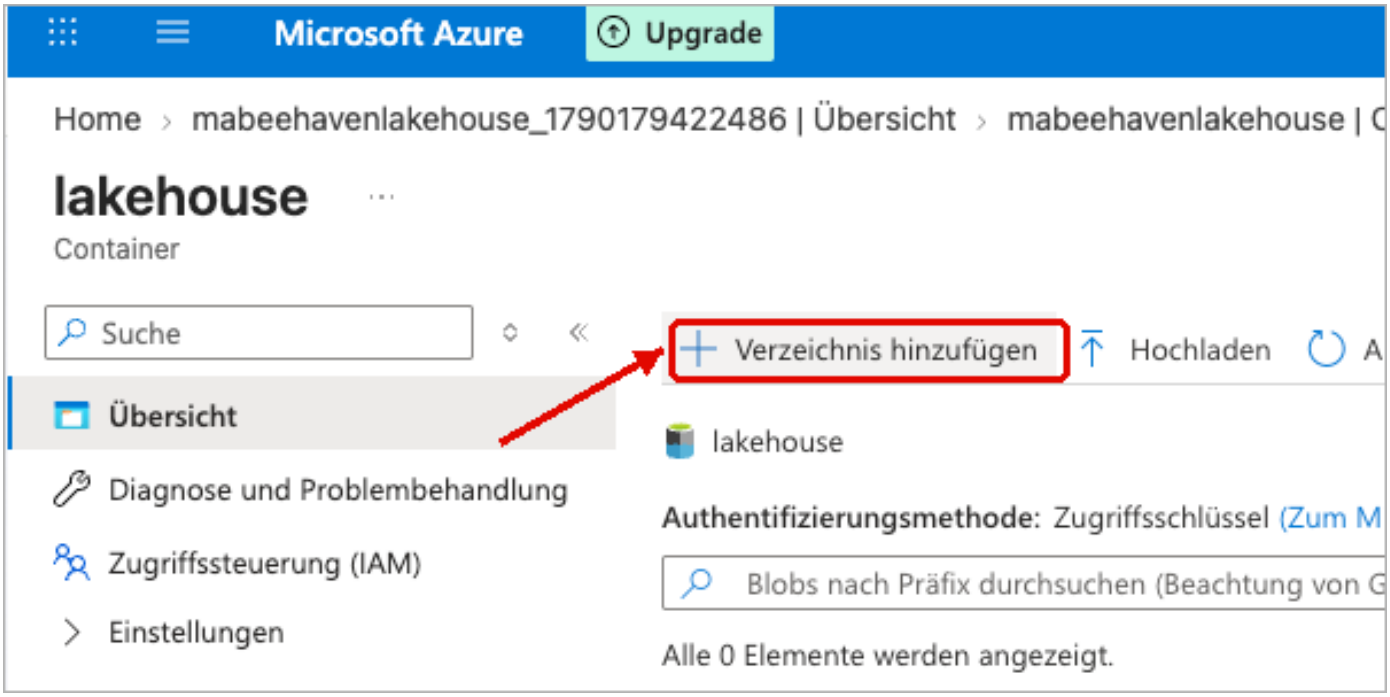


Abb. 14: Im Container: Add Directory (Verzeichnis hinzufügen).

> ⚠️ **Achtung:** Nur einmal auf **Save** klicken. Ein Doppelklick erzeugt die harmlose Meldung „PathAlreadyExists“ (siehe Anhang A).

3. In `bronze` wechseln → dort die Unterordner `new` und `archive` anlegen.

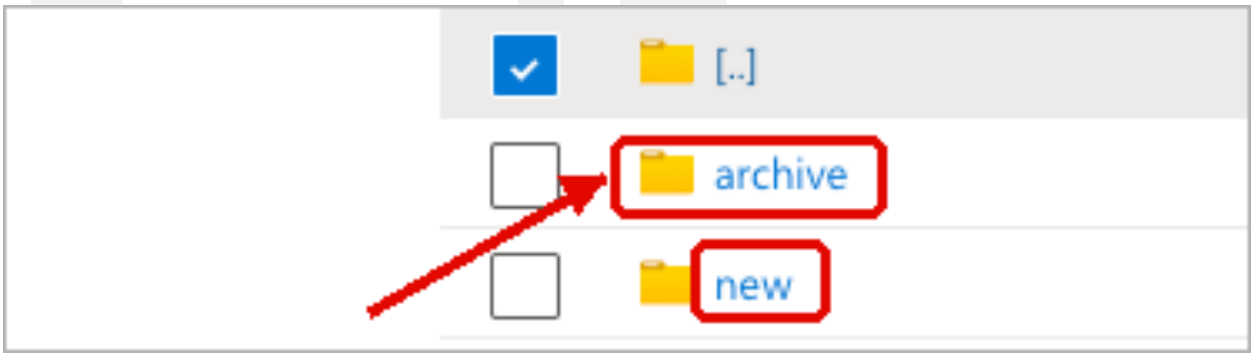


Abb. 15: In bronze die Unterordner new und archive.

4. Zurück zur obersten Ebene von `lakehouse` → Ordner `silver` und `gold` anlegen.



Abb. 16: Fertige Medallion-Struktur: bronze, silver, gold.

> 💡 **Gut zu wissen:** Den Container `$logs` legt Azure automatisch an (Protokolle) – nicht löschen, nicht verwenden.

> ✅ **Prüfen:** `lakehouse` enthält `bronze` (mit `new`, `archive`), `silver` und `gold`.

Schritt 5 – Create Data Factory (Data Factory erstellen)

> 💡 **Gut zu wissen:** **Hintergrund:** Die **Data Factory** ist der Dirigent: Sie bewegt Daten und startet Verarbeitungsschritte in der richtigen Reihenfolge – selbst rechnet sie kaum. Wir arbeiten nach **ELT**: erst laden (Bronze), dann umwandeln (Silver/Gold).

> **Warum?** Die Data Factory ist der „Transporter“: Sie holt Dateien automatisch aus der Quelle und legt sie im Data Lake ab – wiederholbar, zeitgesteuert, protokolliert.

1. Die Ressourcengruppe öffnen (nicht die Liste aller Gruppen!) → **+ Create (Erstellen)**.
2. Im **Marketplace** nach `Data Factory` suchen (optional Häkchen **Nur Azure-Dienste**, dann erscheinen nur Microsoft-Dienste) → die Kachel **Data Factory** von **Microsoft** (Untertitel „Hybriddaten-Integrationsdienst …“) wählen → **Create**.

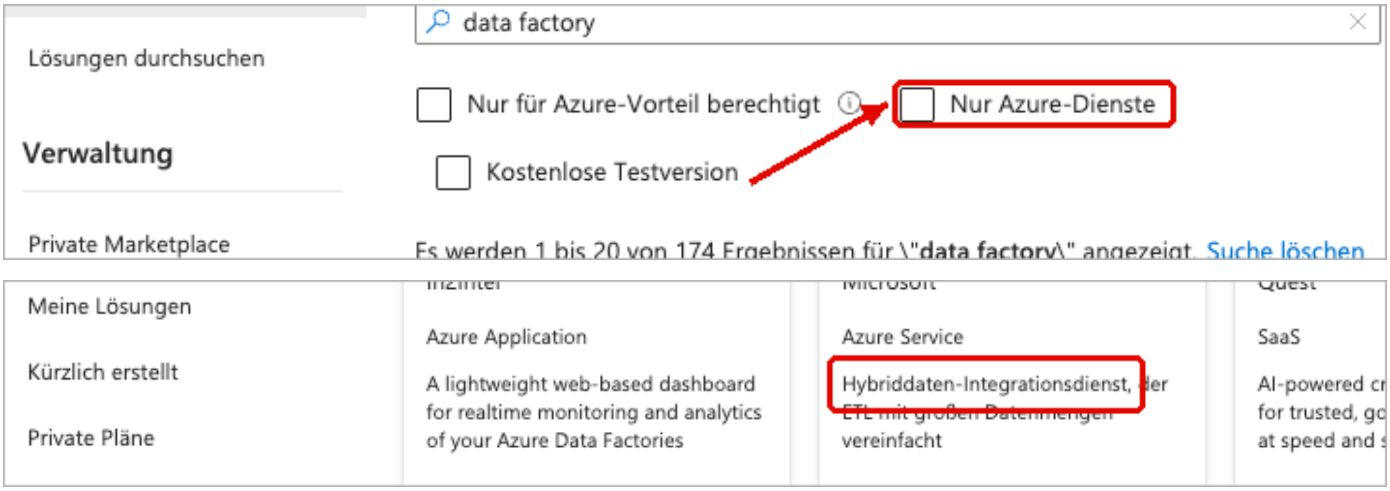


Abb. 17: Marketplace: die Kachel „Data Factory“ von Microsoft wählen – „Nur Azure-Dienste“ hilft gegen Verwechslung. (Ausschnitte 1–2)

> ⚠ **Achtung:** Es gibt Fremdanbieter-Kacheln mit ähnlichen Namen und teils teuren Abos. Richtig: Herausgeber Microsoft, Typ Azure Service. Tipp: **Azure services only (Nur Azure-Dienste)** anhaken.

3. Reiter **Basics (Grundlagen)**: Name `adf-beehaven-dev-gwc`, Region **Germany West Central**, **Version** = **V2**. Den blauen Hinweis auf **Fabric Data Factory** ignorieren – das Projekt nutzt Azure Data Factory.

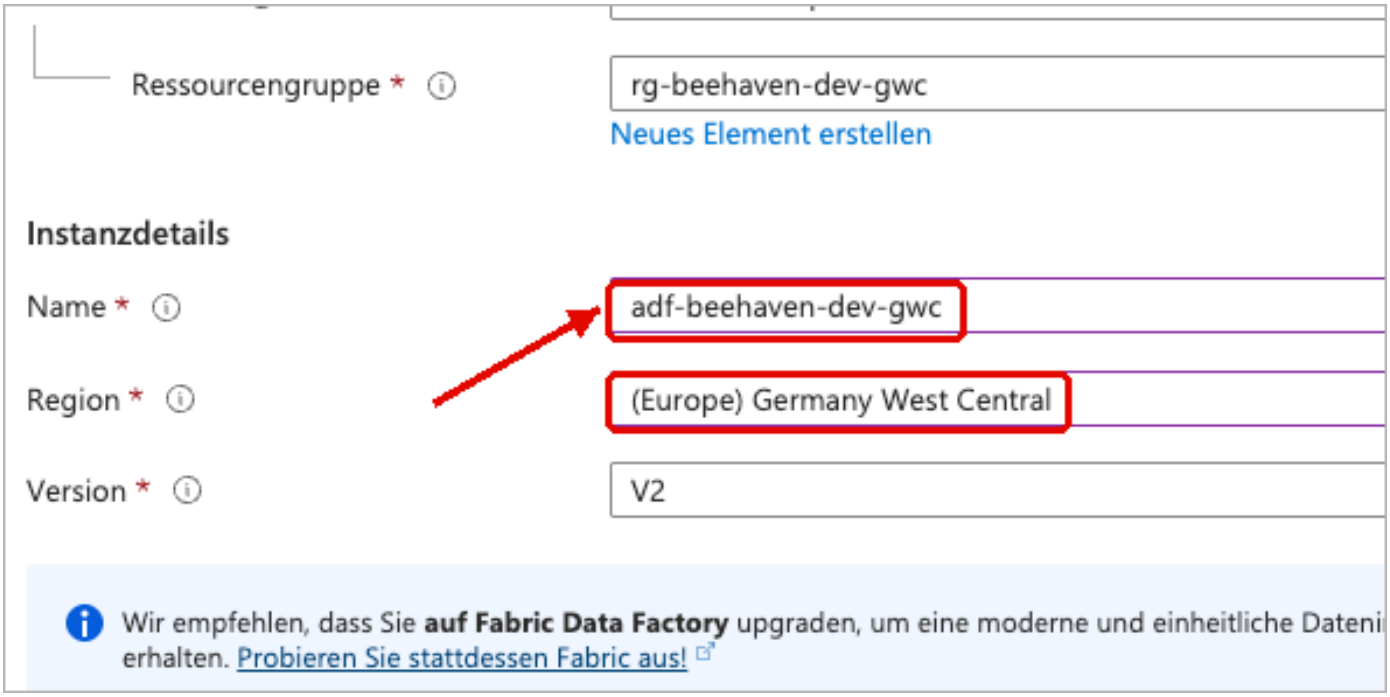


Abb. 18: Name adf-beehaven-dev-gwc und Region Germany West Central (Voreinstellung East US ändern!).

> ⚠ **Achtung:** Die Region steht voreingestellt auf „(US) East US“ – umstellen.

4. Reiter **Git configuration (Git-Konfiguration)**: **Configure Git later (Git später konfigurieren)** anhaken. Warum: Die Versionsverwaltung (Git) folgt im Betrieb (Folgeprojekt FP-1); für den Aufbau reicht Veröffentlichen.



Abb. 19: Git später konfigurieren.

5. **Review + create** → **Create** → warten, bis **Ihre Bereitstellung wurde abgeschlossen** erscheint → **Go to resource (Zu Ressource wechseln)**.

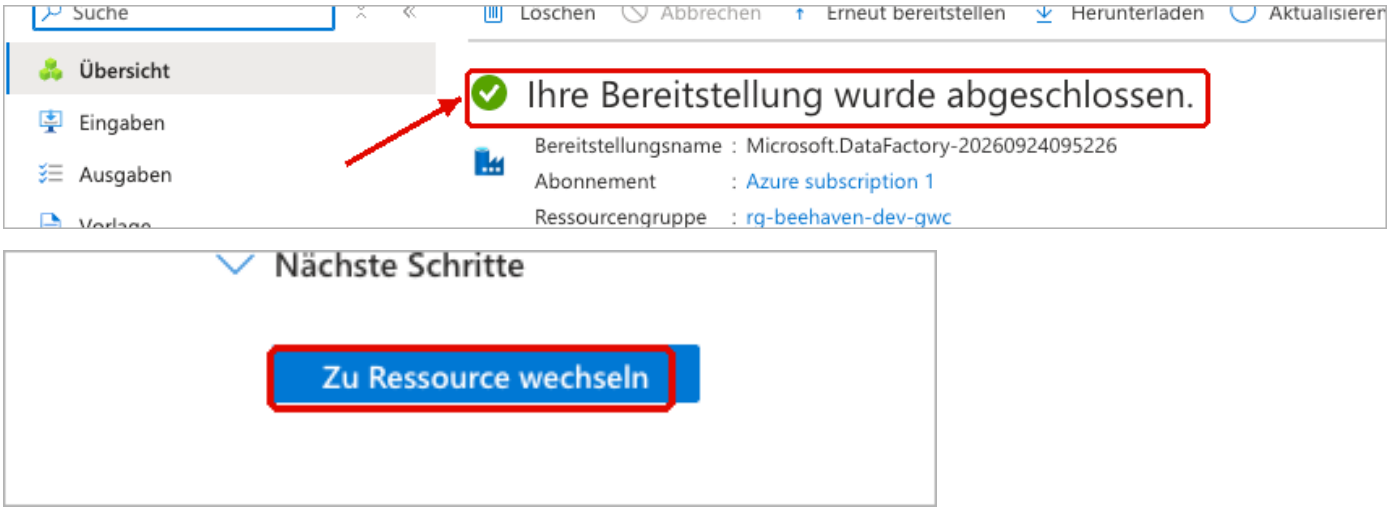


Abb. 20: Bereitstellung abgeschlossen → Zu Ressource wechseln. (Ausschnitte 1–2)

> 💡 **Gut zu wissen:** Den Hinweis „Upgrade to Fabric (Upgrade auf Fabric)“ nicht anklicken: Fabric braucht eine eigene, kostenpflichtige Kapazität. Git (Versionierung der Pipelines) ist im Betrieb sinnvoll und kann jederzeit nachgerüstet werden.

> ✅ **Prüfen:** Die Ressourcengruppe zeigt 3 Ressourcen: `adf-beehaven-dev-gwc`, `mabeehavenlakehouse`, `mabeehavenraw`.

Schritt 6 – Access control (IAM) (Zugriffssteuerung): Rechte für die Data Factory

> 💡 **Gut zu wissen:** **Hintergrund: RBAC** (rollenbasierte Zugriffssteuerung) vergibt Rechte über Rollen statt über Passwörter. Die Data Factory bekommt eine **verwaltete Identität** und genau die Rollen, die sie braucht (Least Privilege – geringste Rechte).

> **Warum?** Die Data Factory hat automatisch eine eigene Identität (Managed Identity / verwaltete Identität) – wie ein Mitarbeiterausweis. Statt eines Schlüssels mit Vollzugriff bekommt sie genau die nötigen Rechte (Least Privilege): Quelle nur lesen, Data Lake lesen und schreiben.

1. `mabeehavenraw` öffnen → **Access control (IAM) (Zugriffssteuerung (IAM))** → **+ Add (Hinzufügen)** → **Add role assignment (Rollenzuweisung hinzufügen)**.

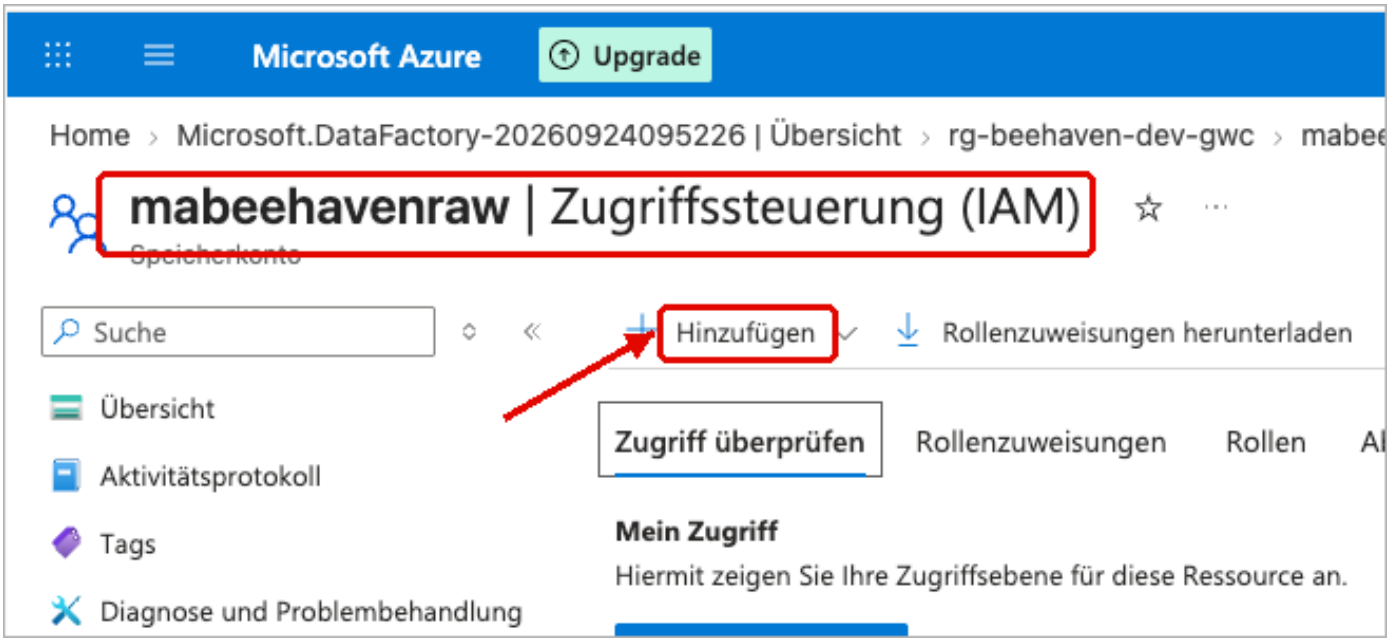


Abb. 21: Speicherkonto → Access control (IAM) (Zugriffssteuerung) → Hinzufügen.

2. In der Suche `blob` eingeben → **Storage Blob Data Reader (Storage-Blobdatenleser)** markieren → **Next (Weiter)**.



Abb. 22: Suche „blob“: Storage-Blobdatenleser (nur lesen) – nicht „Leser“, nicht „Besitzer“.

> ⚠ **Achtung:** „Reader (Leser)“ ist NICHT „Storage Blob Data Reader“: „Leser“ sieht nur Einstellungen, keine Dateien – die Pipeline scheitert dann mit „Zugriff verweigert“. Die deutsche Suche „speicherblob“ findet nur „Besitzer…“ → besser `blob` suchen.

3. **Assign access to (Zugriff zuweisen zu)**: ZUERST **Managed identity (Verwaltete Identität)** wählen → **+ Select members (Mitglieder auswählen)** → **Data Factory (V2)** → `adf-beehaven-dev-gwc` anklicken → **Select (Auswählen)**.

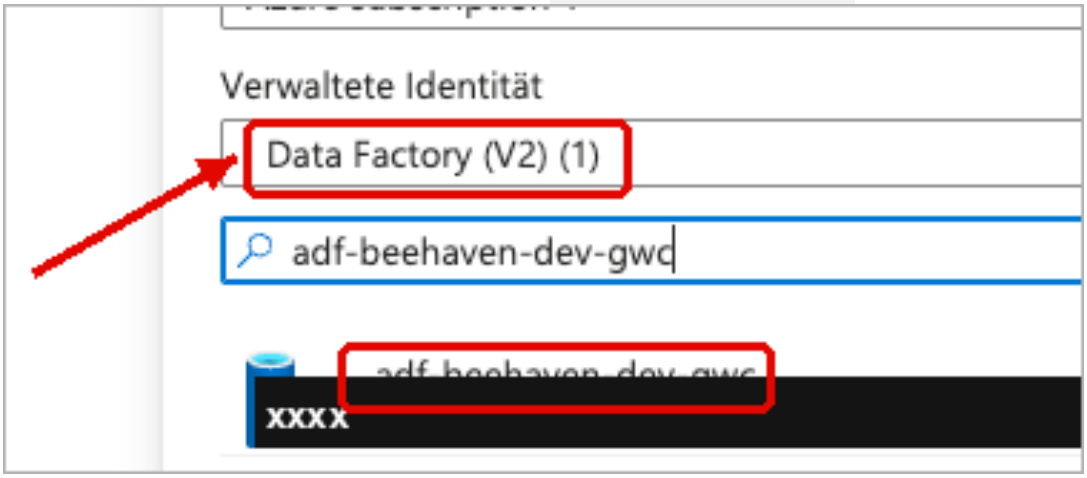


Abb. 23: Zuerst „Verwaltete Identität“ wählen, dann Data Factory (V2) → adf-beehaven-dev-gwc.

> ⚠ **Achtung:** Ohne Umschalten auf „Verwaltete Identität“ erscheint nur deine eigene Person. Die Zeile anklicken, bis sie unter „Ausgewählte Mitglieder“ steht.

4. Bei **Description (Beschreibung)** eintragen: `ADF liest Rohdaten aus source-data`.



Abb. 24: Beschreibung eintragen: wozu das Recht vergeben wird.

5. Zweimal **Review + assign (Überprüfen und zuweisen)** → im Reiter **Role assignments (Rollenzuweisungen)** kontrollieren: unter **Storage-Blobdatenleser (1)** steht die Data Factory.

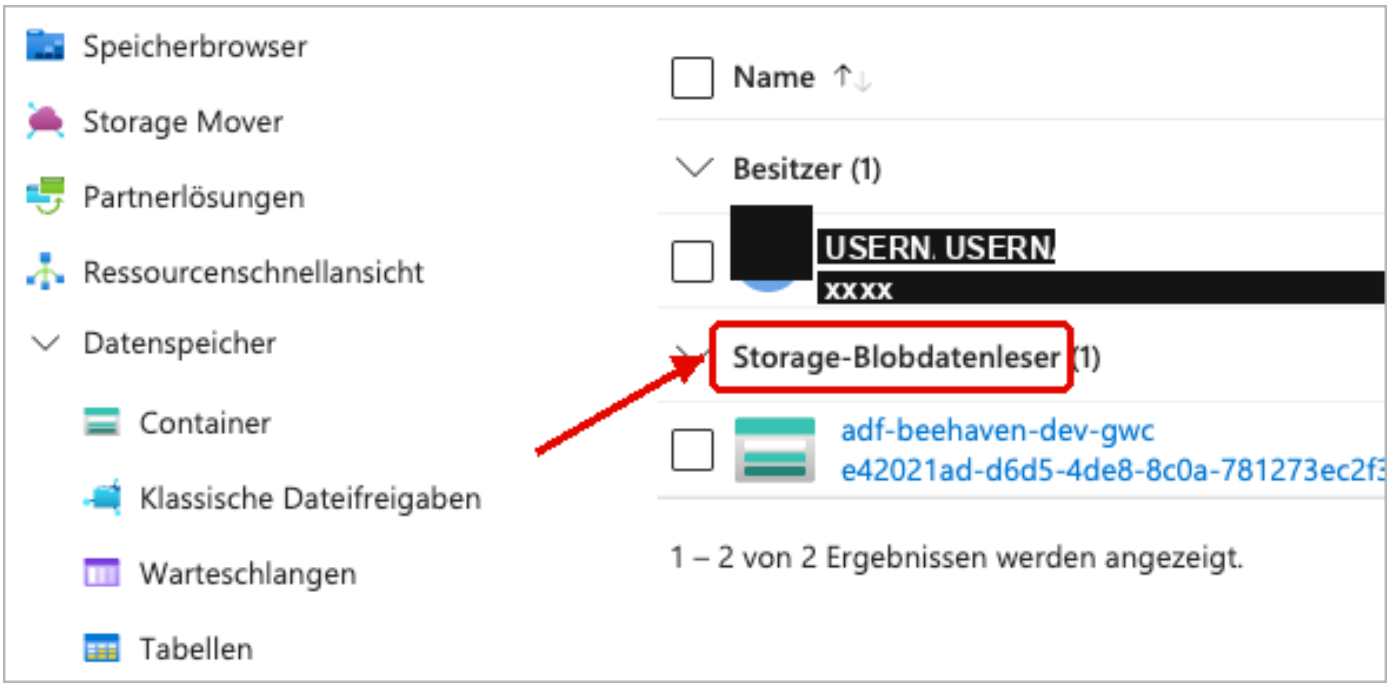


Abb. 25: Kontrolle: adf-beehaven-dev-gwc hat die Rolle nur für „Diese Ressource“.

6. Dasselbe für `mabeehavenlakehouse` – aber mit der Rolle **Storage Blob Data Contributor (Mitwirkender an Storage-Blobdaten)** und der Beschreibung `ADF schreibt in lakehouse`.

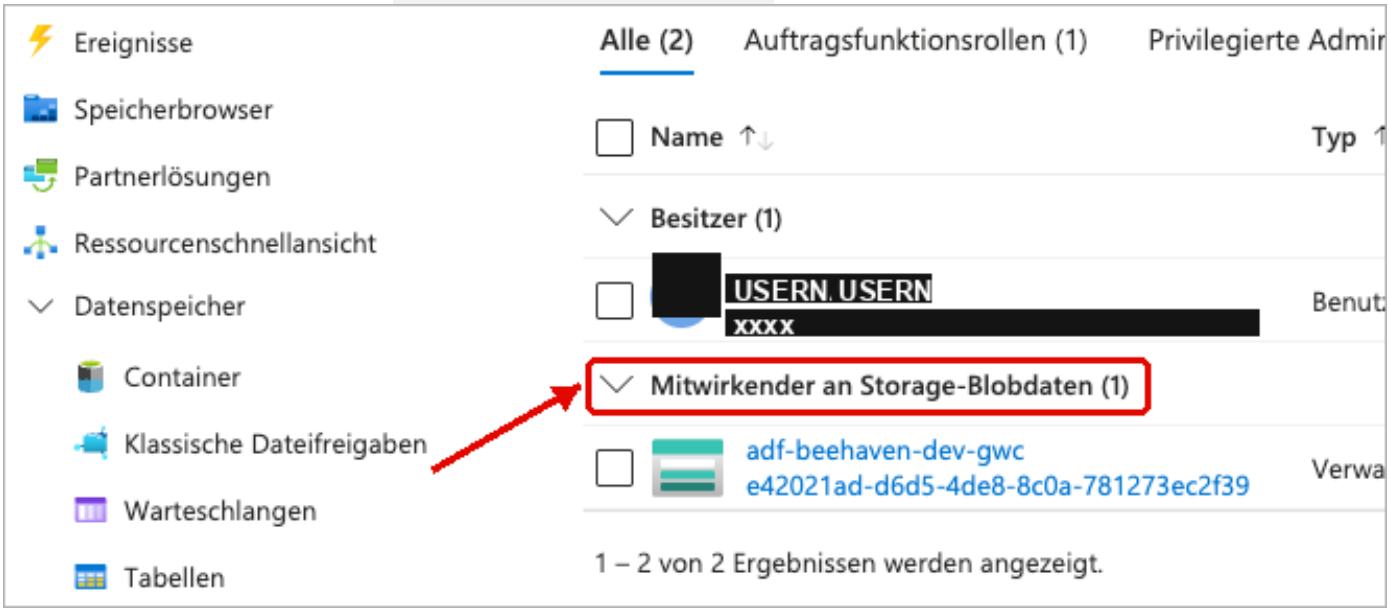


Abb. 26: Data Lake: Mitwirkender an Storage-Blobdaten (lesen und schreiben).

Rolle	Darf	Einsatz
Storage-Blobdatenleser	nur lesen	Quelle `mabeehavenraw`
Mitwirkender an Storage-Blobdaten	lesen, schreiben, löschen	Data Lake `mabeehavenlakehouse`
Besitzer von Speicherblobdaten	alles inkl. Rechtevergabe	NICHT verwenden (zu viel Recht)

> 💡 **Gut zu wissen:** Rechte werden vererbt: Abonnement → Ressourcengruppe → Ressource. Die Data Factory bekommt ihr Recht nur auf „Diese Ressource“ (Scope) – so eng wie möglich. Die Beschreibung dokumentiert, WOZU das Recht vergeben wurde – die erste Frage bei jeder Prüfung (Revision, Datenschutz).

> ✅ **Prüfen:** Reiter **Role assignments**: `adf-beehaven-dev-gwc`, Typ Verwaltete Identität, richtige Rolle, Bereich „Diese Ressource“.

Schritt 7 – Launch Studio (Studio starten)

> 💡 **Gut zu wissen:** **Hintergrund:** Das **ADF Studio** ist die Arbeitsoberfläche der Data Factory im Browser. Änderungen sind dort zuerst Entwürfe – erst **Veröffentlichen** macht sie gültig.

> **Warum?** Im Portal legt man Ressourcen an. Verbindungen, Datasets und Pipelines baut man in einer eigenen Oberfläche: Azure Data Factory Studio.

1. `adf-beehaven-dev-gwc` → **Overview (Übersicht)** → **Launch Studio (Studio starten)**. Ein neuer Browser-Tab öffnet sich.



Abb. 27: Übersicht der Data Factory → Launch Studio (Studio starten).

> ⚠️ **Achtung:** Erscheint „Sie haben keine Berechtigung …“: noch einmal klicken; sonst `adf.azure.com` direkt öffnen oder ein Inkognito-Fenster nutzen.

2. Den Hinweis „Coderepository einrichten“ oben mit **X** schließen (Git später). Oben in der Leiste steht `adf-beehaven-dev-gwc` – du bist in der richtigen Data Factory.

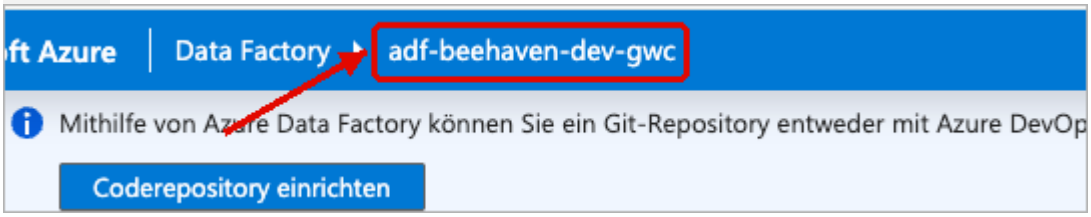


Abb. 28: ADF Studio: links Home, Author (Erstellen), Monitor, Manage (Verwalten).

Symbol links	Name EN (DE)	Wofür
🏠	Home (Startseite)	Übersicht
✏️	Author (Erstellen)	Pipelines und Datasets bauen
🎯	Monitor (Überwachen)	Läufe prüfen
🧰	Manage (Verwalten)	Linked services (verknüpfte Dienste)

> 💡 **Gut zu wissen:** Faustregel: Ressourcen, Rechte (IAM), Tags und Dateien → Azure-Portal. Verbindungen, Datasets, Pipelines, Debuggen, Veröffentlichen → ADF Studio.

Schritt 8 – Manage (Verwalten) → Linked services (Verknüpfte Dienste)

> 💡 **Gut zu wissen:** **Hintergrund:** Ein **Linked Service** (verknüpfter Dienst) ist die gespeicherte Verbindung zu einem Speicher oder Dienst – vergleichbar mit einer Telefonnummer im Adressbuch. Er wird einmal angelegt und von vielen Datasets genutzt.

Warum? Ein verknüpfter Dienst ist wie ein gespeicherter Kontakt im Telefon: Er weiß, WO der Speicher ist und WIE man sich anmeldet. Alle Pipelines nutzen diese eine Verbindung.

1. Links **Manage (Verwalten)** → **Linked services (Verknüpfte Dienste)** → **+ New (Neu)**.
2. Oben im Suchfeld `Azure-Blobspeicher` eingeben → Kachel **Azure Blob Storage (Azure-Blobspeicher)** wählen → **Continue (Weiter)**.

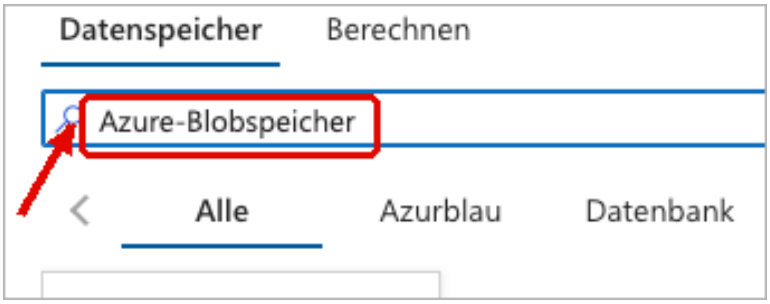


Abb. 29: Linked service (verknüpfter Dienst) für die Quelle: Kachel „Azure-Blobspeicher“.

> ⚠️ **Achtung:** Im deutschen Studio ist der Reiter „Azure“ teils als „Azurblau“ übersetzt.

3. Name `ls_blob_mabeehavenraw`, **Authentication type (Authentifizierungstyp)** = **System-assigned managed identity (Systemseitig zugewiesene verwaltete Identität)**, Speicherkonto `mabeehavenraw` → **Test connection (Verbindung testen)** → **Create (Erstellen)**.

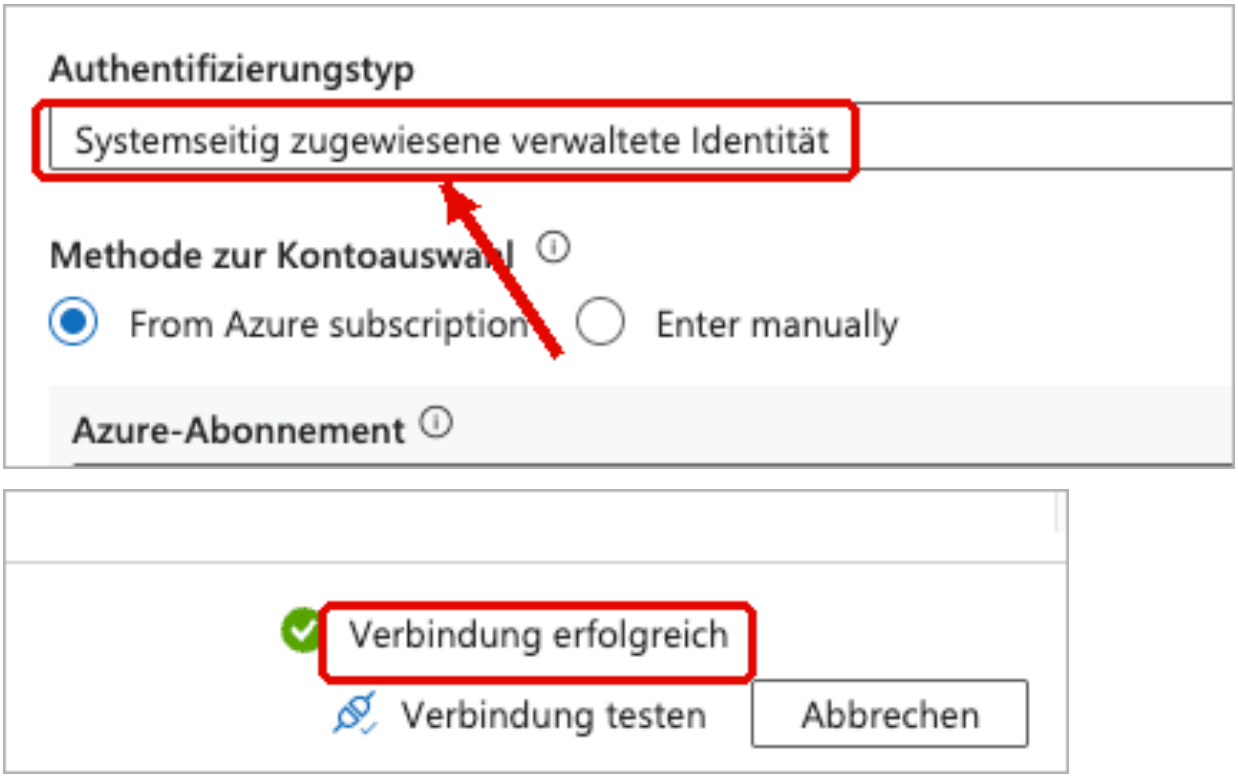


Abb. 30: Authentifizierung: systemseitig zugewiesene verwaltete Identität → Verbindung testen: erfolgreich. (Ausschnitte 1–2)

> ⚠️ **Achtung:** Nicht **Account key (Kontoschlüssel)** wählen – das wäre ein Generalschlüssel mit Vollzugriff.

4. Noch einmal **+ New**: Kachel **Azure Data Lake Storage Gen2**, Name `ls_adls_mabeehavenlakehouse`, **Beschreibung** `Ziel: Data Lake (bronze/silver/gold)`, gleiche Authentifizierung (**Systemseitig zugewiesene verwaltete Identität**), Speicherkonto `mabeehavenlakehouse` → testen → **Create**.

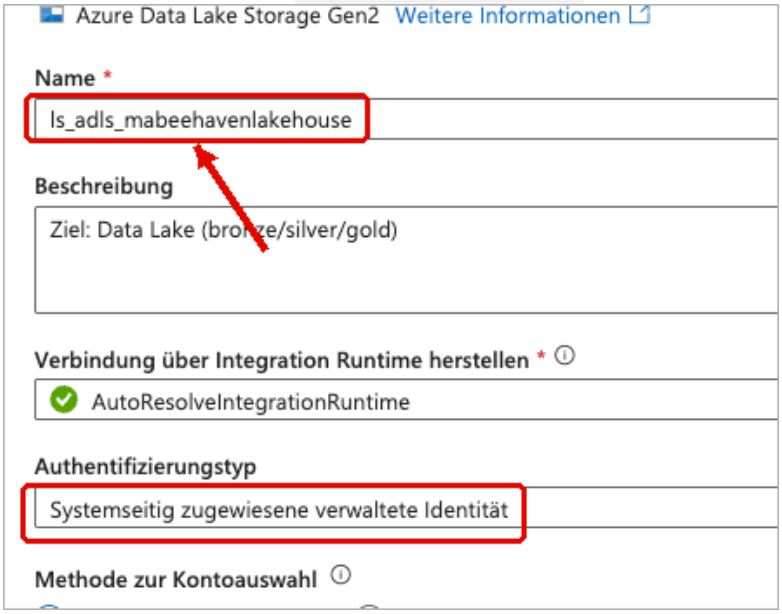


Abb. 31: Linked service zum Data Lake: Azure Data Lake Storage Gen2.

> 💡 **Gut zu wissen:** Den Data Lake immer mit „Azure Data Lake Storage Gen2“ verbinden (nicht Blob) – nur so nutzt ADF die echten Ordnerfunktionen.

5. Oben **Publish all (Alle veröffentlichen)** → Liste prüfen → **Publish (Veröffentlichen)** → danach ↻ **Refresh (Aktualisieren)**. In der Liste stehen `ls_blob_mabeehavenraw` und `ls_adls_mabeehavenlakehouse`.

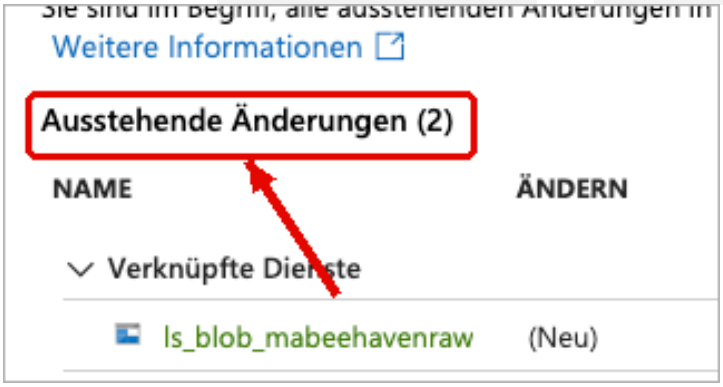


Abb. 32: Alle veröffentlichen: Liste der ausstehenden Änderungen prüfen, dann Veröffentlichen.

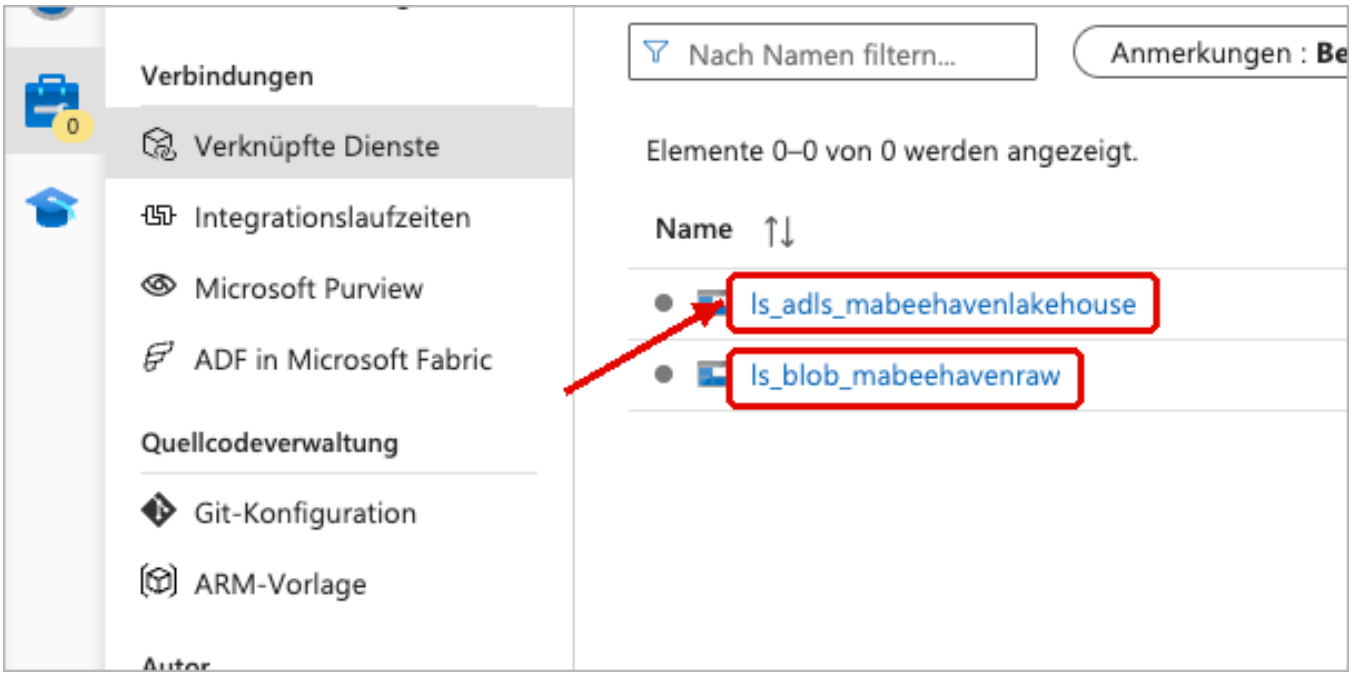


Abb. 33: Beide verknüpften Dienste angelegt.

> ⚠️ **Achtung:** Änderungen im Studio sind erst nach **Publish all** dauerhaft gespeichert. Ein Punkt ● neben einem Namen heißt „noch nicht veröffentlicht“.

> ✅ **Prüfen:** Die Liste zeigt `ls_adls_mabeehavenlakehouse` und `ls_blob_mabeehavenraw`.

Schritt 9 – Author (Erstellen) → Datasets (Datensätze)

> 💡 **Gut zu wissen:** **Hintergrund:** Ein **Dataset** (Datensatz) zeigt auf einen Ordner oder eine Datei und beschreibt das Format. **Parameter** machen es wiederverwendbar: ein Dataset für viele Dateien.

> **Warum?** Der verknüpfte Dienst sagt, WO die Daten liegen. Das Dataset sagt, WELCHE Dateien und in welchem FORMAT. Vergleich: Kontakt = Firma, Dataset = die Abteilung, die man dort anruft.

1. Links **Author (Erstellen)** ✏️ → bei **Datasets** auf **…** → **New dataset (Neues Dataset)**.
2. **Azure Blob Storage (Azure-Blobspeicher)** → Format **Binary (Binär)** → Name `ds_raw_sourcedata_bin` → Linked service `ls_blob_mabeehavenraw`.
3. Über das Ordnersymbol 📁 nur den Container `source-data` wählen – **Directory (Verzeichnis)** und **File name (Dateiname)** bleiben leer → **OK**.



Abb. 34: Dataset ds_raw_sourcedata_bin: nur Container source-data, Verzeichnis und Dateiname leer.

> ⚠️ **Achtung:** Nur den ORDNER wählen, keine Datei anklicken – ein fester Dateiname würde nur diese eine Datei kopieren.

4. Zweites Dataset: **Azure Data Lake Storage Gen2** → **Binary** → Name `ds_lake_bronze_new_bin` → `ls_adls_mabeehavenlakehouse` → Pfad `lakehouse` / `bronze/new` / leer.



Abb. 35: Dataset ds_lake_bronze_new_bin: lakehouse / bronze/new.

> 💡 **Gut zu wissen:** Beim Data Lake heißt das erste Feld **File system (Dateisystem)** statt Container.

5. **Publish all (Alle veröffentlichen)** → ↻. Unter **Datasets** stehen jetzt `ds_raw_sourcedata_bin` und `ds_lake_bronze_new_bin`.

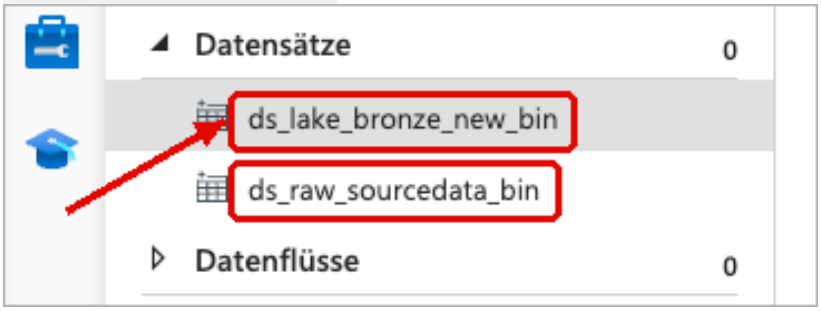


Abb. 36: Beide Datasets veröffentlicht (kein Punkt ● mehr).

> 💡 **Gut zu wissen:** Binär statt CSV: Für Bronze werden Dateien 1:1 kopiert, ohne sie zu lesen. Die Endung `.csv` bleibt erhalten. Erst ab Silver wird als CSV (DelimitedText, Text mit Trennzeichen) gelesen.

> ✅ **Prüfen:** Unter **Datasets** stehen `ds_raw_sourcedata_bin` und `ds_lake_bronze_new_bin` ohne Punkt ●.

Schritt 10 – Pipeline pl_ingest_raw_to_bronze: Rohdaten nach Bronze

> 💡 **Gut zu wissen:** **Hintergrund:** Eine **Pipeline** ist eine Kette von Aktivitäten. Die **Copy-Aktivität** kopiert Daten von A nach B – hier binär, also Byte für Byte, ohne sie zu verändern (Bronze = unverändert).

> **Warum?** Der eigentliche Transport: alle CSV-Dateien unverändert aus der Quelle nach `bronze/new` – wiederholbar statt Handarbeit (ELT: erst laden, später umwandeln).

1. Bei **Pipelines** auf **…** → **New pipeline (Neue Pipeline)** → rechts unter **Properties (Eigenschaften)** Name `pl_ingest_raw_to_bronze`. Bis der Name geändert ist, heißt die Pipeline z. B. „Pipeline2“ – so steht es noch in einigen Bildern.
2. Links **Move and transform (Verschieben und transformieren)** aufklappen → **Copy data (Daten kopieren)** auf die Fläche ziehen → Name `copy_raw_to_bronze_new`.
3. Reiter **Source (Quelle)**: Dataset `ds_raw_sourcedata_bin`, **File path type (Dateipfadtyp)** = **Wildcard file path (Platzhalterdateipfad)**, Dateiname `*.csv`, **Recursively (Rekursiv)** an.

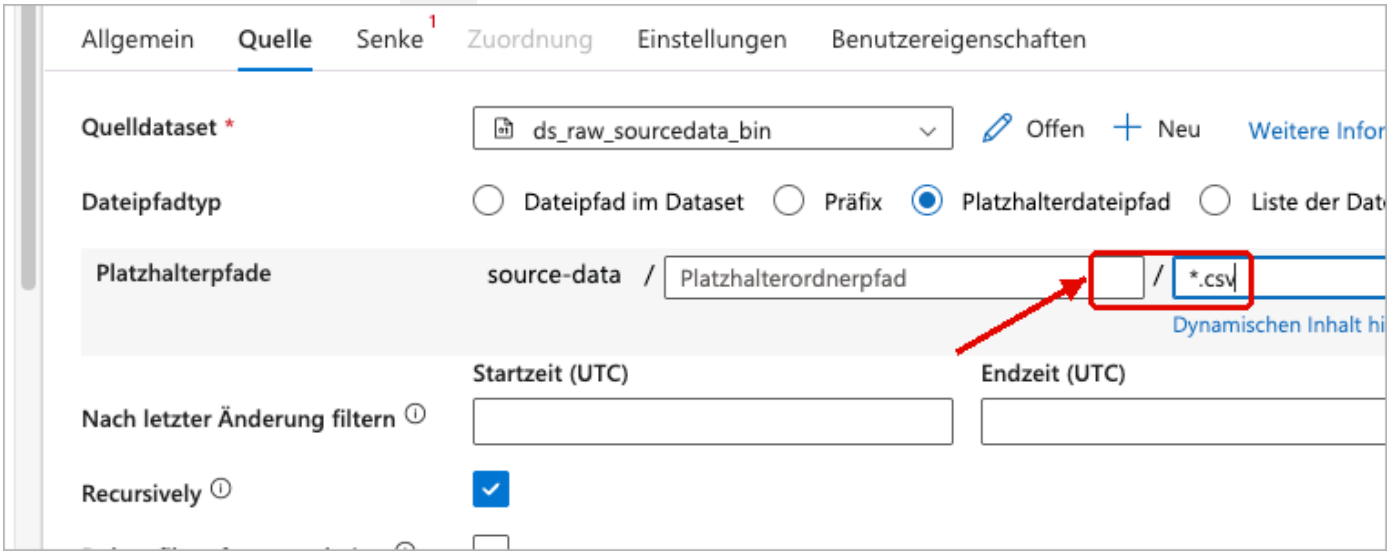


*Abb. 37: Quelle: Platzhalterdateipfad *.csv, Recursively an, Delete files after completion AUS.*

> ⚠️ **Achtung:** **Delete files after completion (Dateien nach Abschluss löschen)** hier AUS lassen – sonst werden die Originale in der Quelle gelöscht.

4. Reiter **Sink (Senke)**: Dataset `ds_lake_bronze_new_bin`.

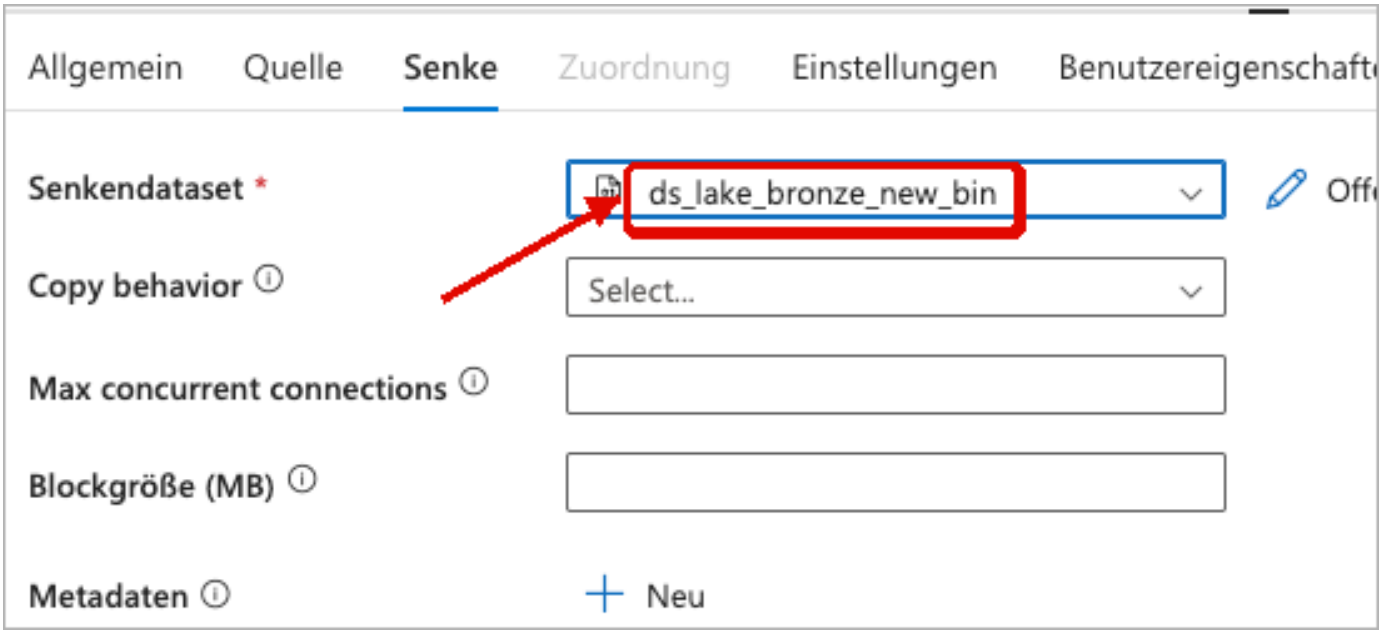


Abb. 38: Senke: ds_lake_bronze_new_bin.

5. **Validate (Überprüfen)** → **Debug (Debuggen)** → unten im Reiter **Output (Ausgabe)** muss **Succeeded (Erfolgreich)** stehen.

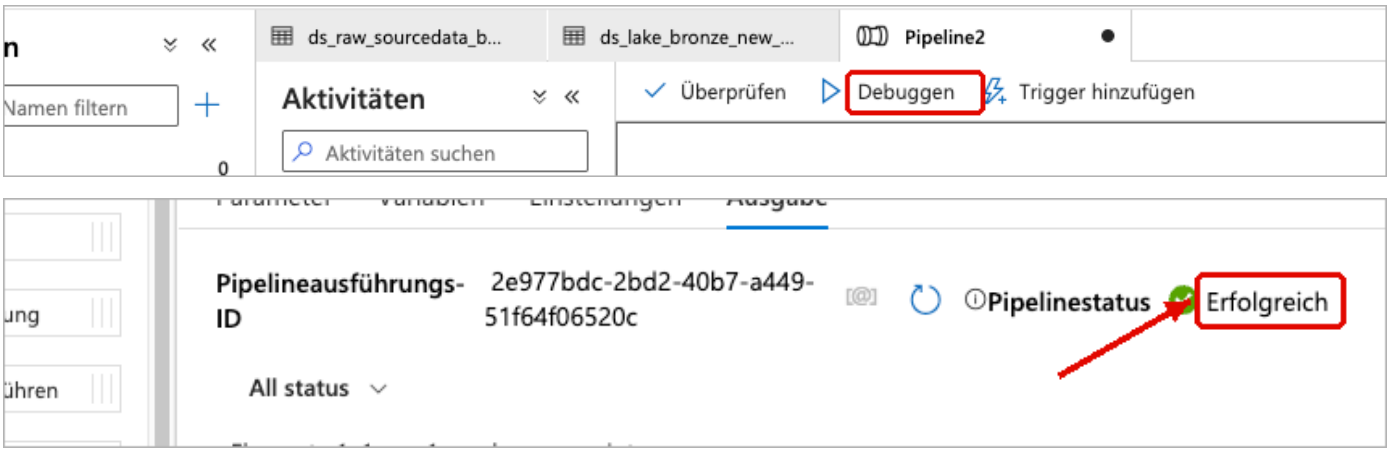


Abb. 39: Debuggen → Pipelinestatus Erfolgreich. (Ausschnitte 1–2)

6. Gegenprobe im Portal: Liegen die 4 Dateien in `bronze/new` (z. B. `flow_schwartau.csv`, `weight_schwartau.csv`)? Dann **Publish all** → ↻.

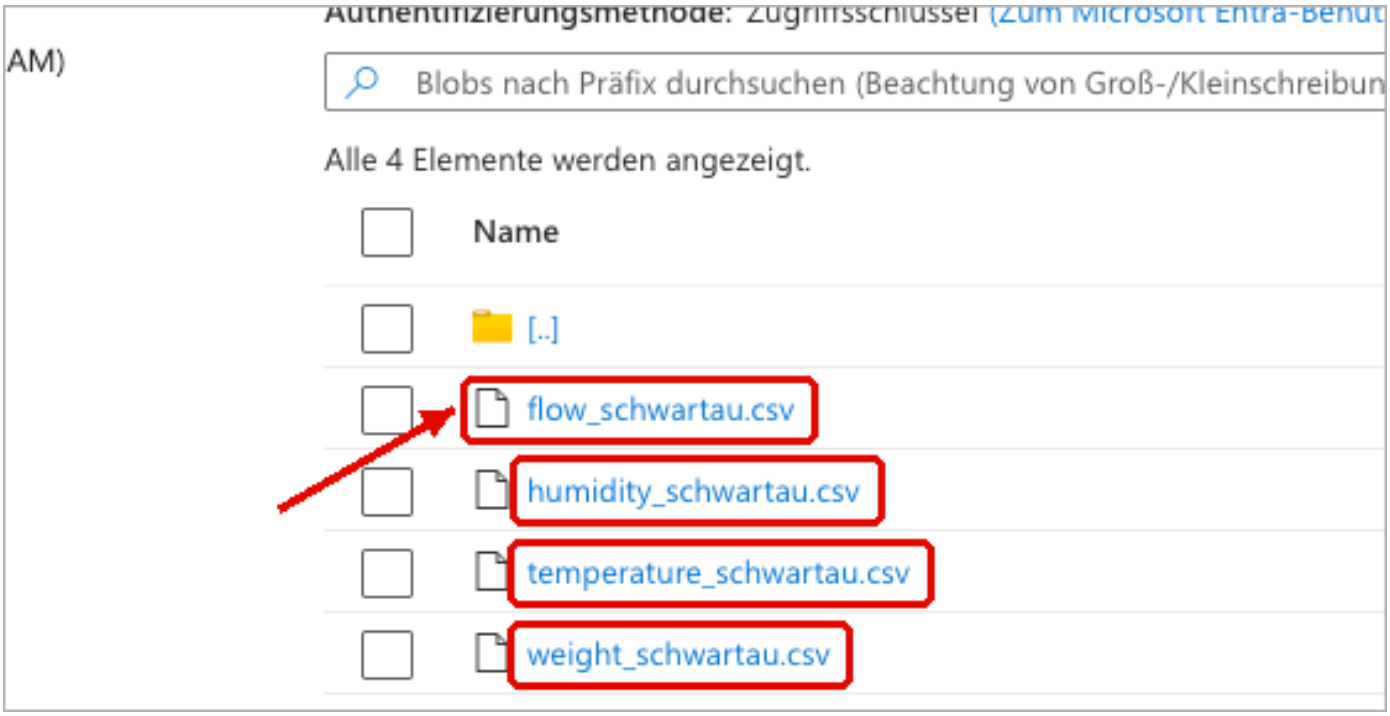


Abb. 40: Gegenprobe im Portal: 4 CSV in bronze/new, Größen wie in der Quelle.

> ⚠ **Achtung:** „Erfolgreich“ allein beweist nicht, dass die Dateien am richtigen Ort liegen – immer im Ziel nachsehen.

> 💡 **Gut zu wissen:** „Sink (Senke)“ = das Ziel, in das die Daten fließen (Quelle → Senke = Von → Nach). `*.csv` lässt versehentlich hochgeladene Systemdateien (z. B. `.DS_Store` vom Mac) draußen. Debuggen = echter Probelauf ohne Veröffentlichen.

> 💡 **Gut zu wissen:** Code kommentieren: ADF-Ausdrücke haben keine Kommentarfunktion. Deshalb bei jeder Aktivität im Reiter **General (Allgemein)** das Feld **Description (Beschreibung)** ausfüllen, z. B. „Kopiert alle CSV aus source-data unverändert nach bronze/new“.

> ✅ **Prüfen:** `bronze/new` enthält 4 CSV mit denselben Größen wie `source-data`.

Schritt 11 – Dynamische Pipeline: Get Metadata + ForEach

> 💡 **Gut zu wissen:** **Hintergrund: Get Metadata** liest den Inhalt eines Ordners (Liste der Dateien), **ForEach** wiederholt Aktivitäten für jedes Element. Zusammen verarbeitet die Pipeline jede neue Datei, ohne dass ein Dateiname fest eingetragen ist (dynamischer Inhalt).

> **Warum?** Statt „alles fest auf einmal“ schaut die Pipeline selbst nach, welche Dateien da sind, und verarbeitet jede einzeln. Kommt eine neue Datei (anderer Bienenstock, andere Stadt) dazu, wird sie automatisch mitverarbeitet – ohne Änderung an der Pipeline.

1. Neue Pipeline anlegen und rechts unter **Properties (Eigenschaften)** `pl_bronze_new_to_archive` nennen (im Bild steht noch der vorläufige Name „Pipeline 1“).
2. **General (Allgemein)** → **Get Metadata (Metadaten abrufen)** auf die Fläche → Name `get_files_bronze_new`. Reiter **Settings (Einstellungen)**: Dataset `ds_lake_bronze_new_bin`, **Field list (Feldliste)** → **+ New (Neu)** → im Feld **Argument Child items (Untergeordnete Elemente)** wählen – liefert die Liste der Dateien im Ordner, die die Schleife später abarbeitet. **Nach letzter Änderung filtern** bleibt leer (alle Dateien).

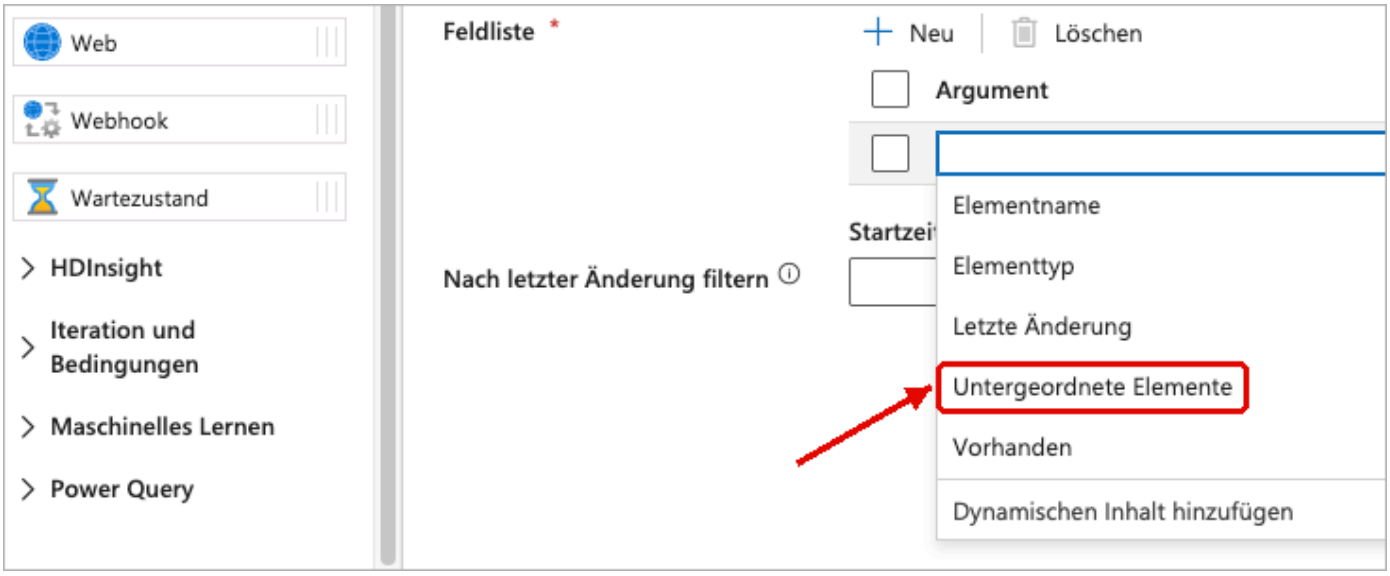


Abb. 41: Get Metadata: Dataset ds_lake_bronze_new_bin, Feldliste „Untergeordnete Elemente“ (childItems).

3. **Debug** → in der Ausgabezeile auf **[→ (Output / Ausgabe)** klicken: Im Fenster **Ausgabe** steht die Liste `"childItems"` mit `name` und `type` je Datei (hier 4 CSV-Dateien) – so prüft man vor dem Weiterbauen, dass die Aktivität die Dateien findet.

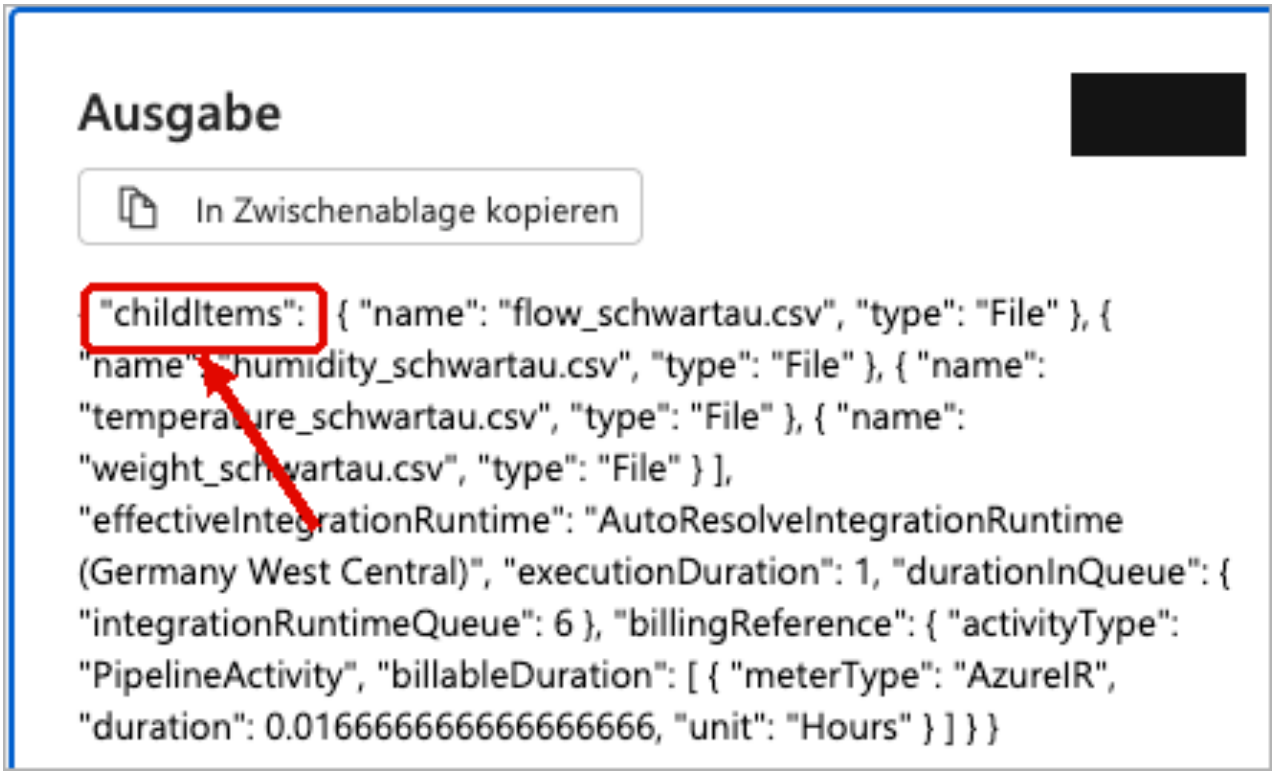


Abb. 42: Ausgabe [→: Liste childItems mit name und type je Datei.

4. **Iteration & conditionals (Iteration und Bedingungen)** → **ForEach (Für jedes)** daneben ziehen. Vom grünen Kästchen ☑ rechts an **Metadaten abrufen** (`get_files_bronze_new`) einen Pfeil zu **ForEach** ziehen – grün heißt: Die Schleife startet nur, wenn die Dateiliste erfolgreich gelesen wurde.



Abb. 43: ForEach per grünem Pfeil (bei Erfolg) mit Get Metadata verbinden.

5. ForEach → Name `foreach_file` → Reiter **Settings** → **Items (Elemente)** → **Add dynamic content (Dynamischen Inhalt hinzufügen)**: `@activity('get_files_bronze_new').output.childItems` – die Schleife läuft einmal je Datei aus der Liste.



Abb. 44: Elemente: @activity('get_files_bronze_new').output.childItems.

> 💡 **Gut zu wissen: Sequential (Sequenziell)** bleibt leer = die Dateien werden parallel verarbeitet (schneller); **Batchanzahl (Batch count)** leer = Standard (bis zu 20 gleichzeitig).

6. In der ForEach-Box auf den Stift ✏️ klicken (du bist jetzt IN der Schleife) → **Copy data (Daten kopieren)** hinzufügen → Name `move_file_to_archive`. Quelle und Senke richtest du in Schritt 13 ein.
7. Oben im Pfad zurück zur Pipeline. Jetzt **nicht** debuggen und **nicht** veröffentlichen: `move_file_to_archive` hat noch keine Quelle und kein Ziel – **Validate** und **Publish all** würden das als Fehler melden. Den Browser-Tab mit ADF Studio offen lassen: Ohne Git-Anbindung sind nicht veröffentlichte Änderungen beim Schließen weg. Der erste vollständige Testlauf folgt in Schritt 13.

⚠️ **Achtung:** Erst Schritt 12 (Kopie nach Silver), dann Schritt 13 (Archiv): Das Archiv verschiebt die Dateien aus `bronze/new` – danach fände Silver nichts mehr.

Begriff EN (DE)	Einfach erklärt	Bei Bee Haven
Get Metadata (Metadaten abrufen)	liest aus, welche Dateien in einem Ordner liegen	Liste der 4 CSV
ForEach (Für jedes)	Schleife: gleicher Schritt für jedes Element	für jede Datei: verschieben
Dynamic content (dynamischer Inhalt)	Ausdruck mit @, wird zur Laufzeit berechnet	`@item().name`
`@activity('…').output.childItems`	Ausgabe einer vorherigen Aktivität	Liste aus Get Metadata
`@item()`	aktuelles Element der Schleife	flow_…, humidity_…
Parameter (Dataset)	Platzhalter im Dataset, von außen gefüllt: `@dataset().name`	Silver, Archiv

Kästchen rechts an einer Aktivität	Bedeutung
grün ☑	nächster Schritt bei Erfolg (Success)
rot ☒	bei Fehler (Failure) – z. B. Warnung senden
blau ➔	bei Abschluss (Completion), egal ob Erfolg oder Fehler
grau ↱	wenn übersprungen (Skipped)

⚠️ **Achtung:** `@item().name` – nicht `@item().file_name`: Get Metadata liefert das Feld „name“; `file_name` ist ein Dataset-Parameter. Einen versehentlich geöffneten Ausdrucksgenerator mit **Cancel (Abbrechen)** schließen, nicht mit OK.

1. Typischer Fehler: Eine Binärkopie von einem Ordner (Platzhalter) in eine einzelne Datei ist nicht erlaubt. Die Meldung lautet sinngemäß „Binärkopieren … nicht unterstützt“. Lösung in Schritt 13: Datei → Datei mit einem Parameter-Dataset.

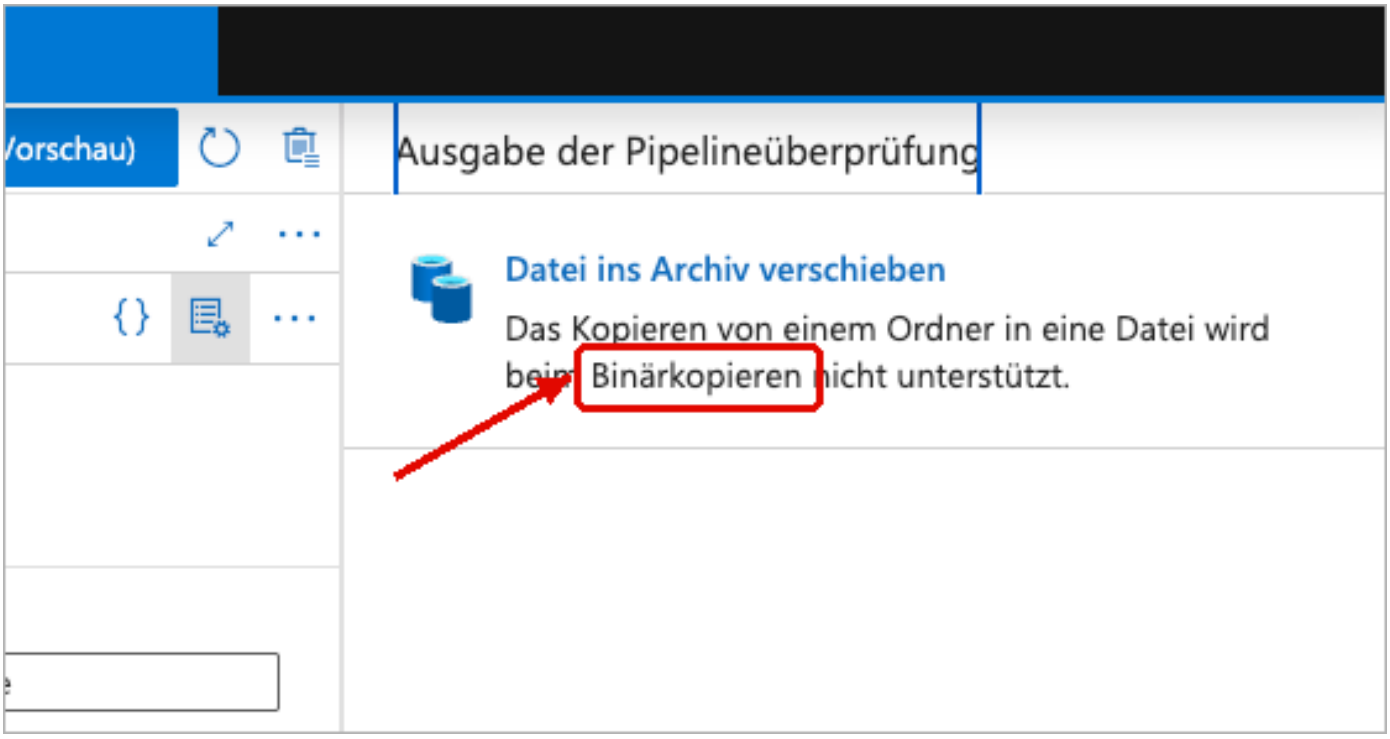


Abb. 45: Typischer Fehler: Binärkopie von Ordner (Platzhalter) in Datei wird nicht unterstützt.

✅ **Prüfen:** Debug von `get_files_bronze_new` zeigt `childItems` mit den 4 Dateien; `foreach_file` hängt mit grünem Pfeil daran und enthält `move_file_to_archive` (noch ohne Quelle/Senke).

Schritt 12 – Bronze → Silver (CSV, Ordner je Sensor)

💡 **Gut zu wissen:** **Hintergrund:** Beim Kopieren nach Silver wird aus dem Dateinamen der Zielordner abgeleitet (Stadt, Sensor). So liegt jede Messreihe in einem eigenen Ordner – Voraussetzung für mehrere Standorte.

> **Warum?** In Silver werden die Daten bereinigt. Dafür muss ADF sie als Tabelle lesen (CSV), nicht nur als Binärdatei. Jeder Sensortyp bekommt automatisch einen eigenen Ordner – abgeleitet aus dem Dateinamen –, damit auch weitere Städte ohne Änderung einsortiert werden.

1. Neues Dataset: **Azure Data Lake Storage Gen2** → **DelimitedText (Text mit Trennzeichen)** → Name `ds_lake_bronze_new_csv` → Pfad `lakehouse` / `bronze/new` / leer, **First row as header (Erste Zeile als Überschrift)** an, **Import schema (Schema importieren)** = **None (Keine)**.
2. Im Dataset Reiter **Parameters (Parameter)** → `file_name` anlegen. Reiter **Connection (Verbindung)**: **Verknüpfter Dienst** `ls_adls_mabeehavenlakehouse`, **Dateipfad** `lakehouse` / `bronze/new` / im Feld Dateiname `@dataset().file_name` – der Dateiname kommt je Durchlauf von außen, ein Dataset reicht für alle Dateien. **Komprimierungstyp** (Keine Komprimierung), **Spaltentrennzeichen** (Comma), **Zeilentrennzeichen** (Default) und **Codierung** (UTF-8) bleiben – sie passen zu den Rohdaten.

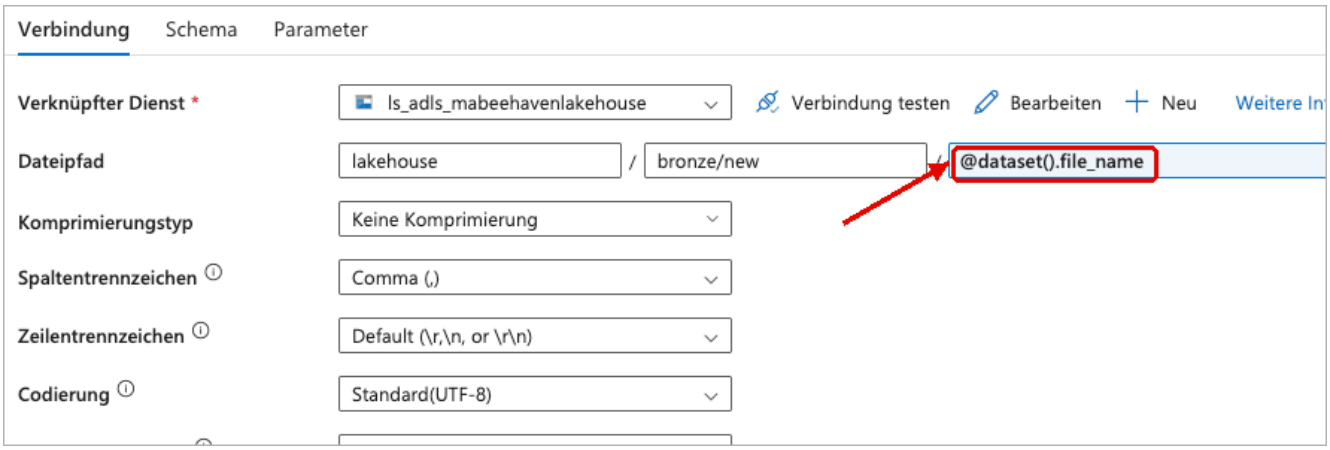


Abb. 46: ds_lake_bronze_new_csv: DelimitedText, Dateiname @dataset().file_name, erste Zeile als Überschrift.

3. Zweites Dataset `ds_lake_silver_csv`: DelimitedText, nur Dateisystem `lakehouse`, Überschrift an, Schema **None**. Parameter `folder_name` und `file_name`. Reiter **Verbindung**: **Verknüpfter Dienst** `ls_adls_mabeehavenlakehouse`, **Dateipfad** `lakehouse` / Verzeichnis `@concat('silver/', dataset().folder_name)` / Dateiname `@dataset().file_name` – Ordner und Datei werden je Durchlauf eingesetzt, ein Dataset reicht für alle Sensoren. **Komprimierungstyp** (Keine), **Spaltentrennzeichen** (Comma), **Zeilentrennzeichen** (Default) und **Codierung** (UTF-8) bleiben.

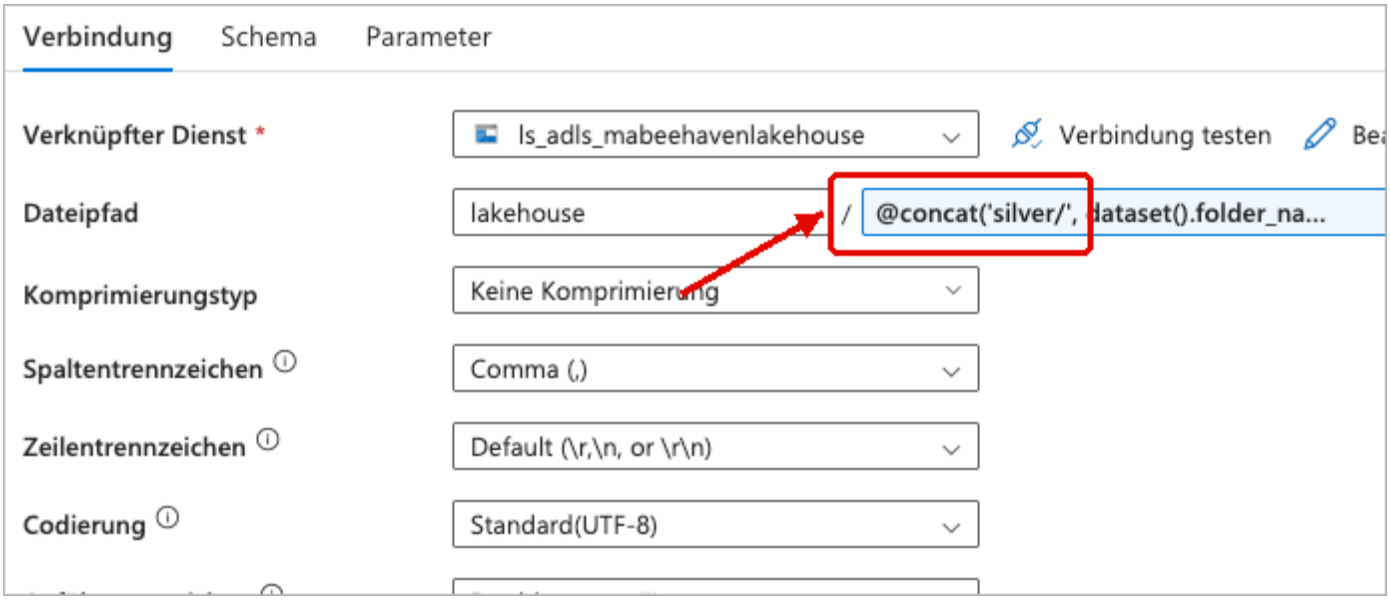


Abb. 47: ds_lake_silver_csv: Verzeichnis @concat('silver/', dataset().folder_name).

4. Neue Pipeline `pl_bronze_to_silver`: **Get Metadata** (wie Schritt 11) → **ForEach** → darin **Copy data** mit Namen `copy_to_silver`.
5. Reiter **Source (Quelle)**: `ds_lake_bronze_new_csv`, `file_name` = `@item().name`, Dateipfadtyp **File path in dataset (Dateipfad im Dataset)** – der Pfad kommt aus dem Dataset, `@item().name` ist der Dateiname des aktuellen Schleifendurchlaufs.

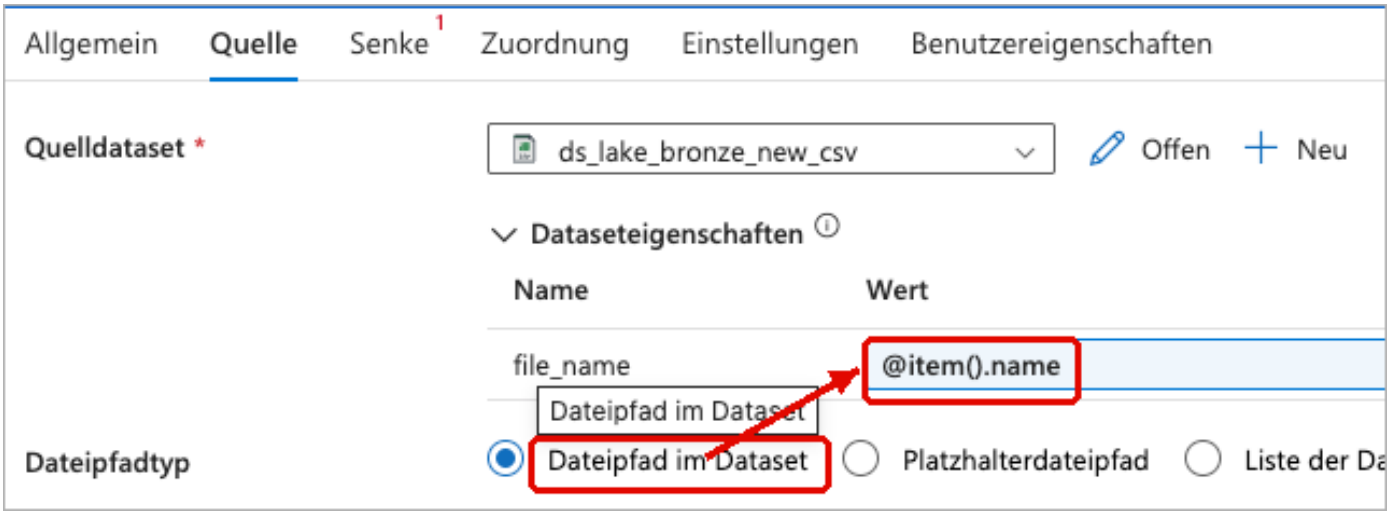


Abb. 48: copy_to_silver – Quelle: file_name = @item().name, Dateipfad im Dataset.

6. Reiter **Sink (Senke)**: `ds_lake_silver_csv`, unter **Dateseteigenschaften** `folder_name` = `@split(item().name, '_')[0]` (Teil vor dem ersten `_`, also der Sensorname → eigener Ordner je Sensor) und `file_name` = `@item().name` (Dateiname bleibt gleich).

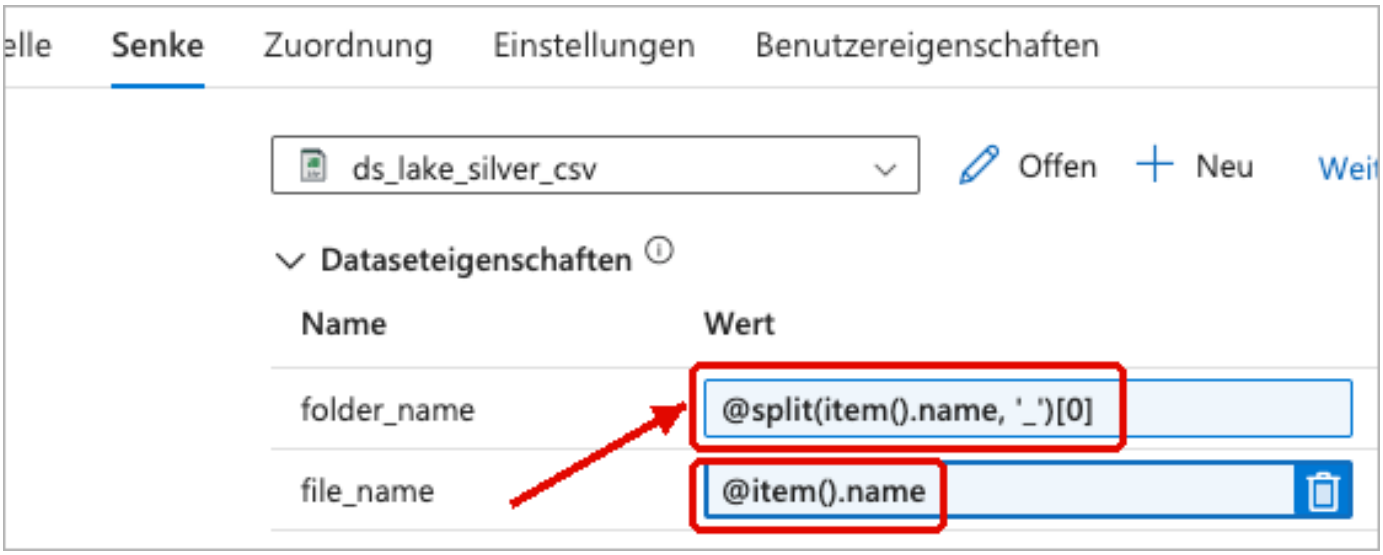


Abb. 49: Senke: folder_name = @split(item().name, '_')[0], file_name = @item().name.

> ⚠ **Achtung:** ADF sortiert Dataset-Parameter alphabetisch (`file_name` vor `folder_name`) – die Zeilen am Namen erkennen, nicht an der Reihenfolge.

7. **Validate** → **Debug** → unten in der Ausgabe: viermal `copy_to_silver` (eine Kopie je Datei) und `foreach_file` mit **Erfolgreich**.

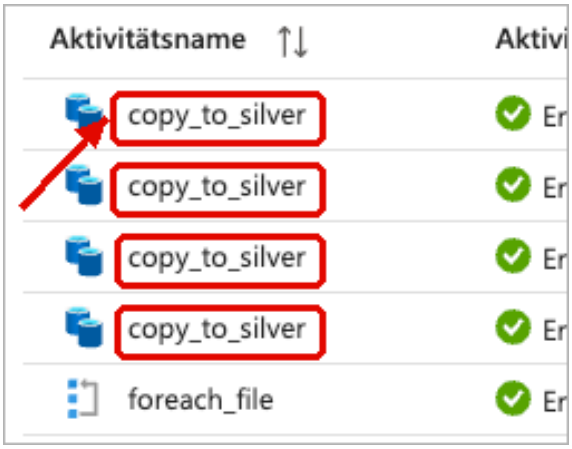


Abb. 50: Debug: 4× copy_to_silver erfolgreich.

8. Gegenprobe im Portal: Container `lakehouse` → `silver` → je ein Ordner **flow**, **humidity**, **temperature**, **weight** – die Schleife hat für jeden Sensor einen eigenen Ordner angelegt.

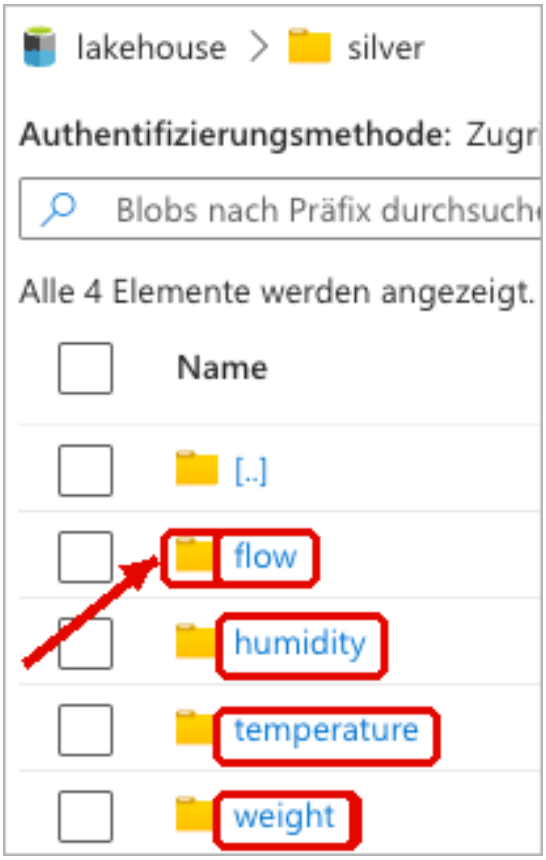


Abb. 51: Gegenprobe: silver/flow, humidity, temperature, weight.

9. Einen Ordner öffnen (z. B. `flow`): Darin liegt die CSV-Datei **flow_schwartau.csv**. Noch nicht veröffentlichen – **Publish all** prüft alle Pipelines und würde die unfertige Archiv-Pipeline aus Schritt 11 melden; veröffentlicht wird am Ende von Schritt 13.

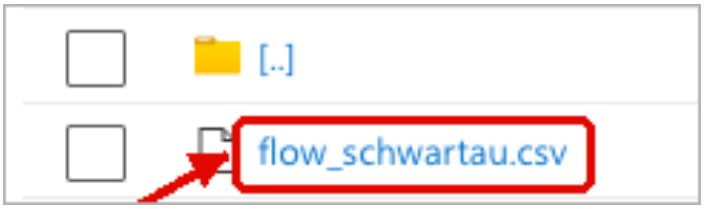


Abb. 52: silver/flow/flow_schwartau.csv.

Datei	split(..., '_')	[0] = Ordner
`flow_schwartau.csv`	flow | schwartau.csv	`silver/flow/`
`humidity_schwartau.csv`	humidity | schwartau.csv	`silver/humidity/`
`temperature_schwartau.csv`	temperature | schwartau.csv	`silver/temperature/`
`weight_schwartau.csv`	weight | schwartau.csv	`silver/weight/`

> 💡 **Gut zu wissen:** `split` zerschneidet den Dateinamen am Unterstrich in eine Liste; `[0]` nimmt das erste Element – in der Programmierung zählt man ab 0 (wie in Python `liste[0]`). `concat` verbindet Texte: 'silver/' + 'flow' = 'silver/flow'. „Schema: None“, weil ein Dataset für alle Sensoren mit unterschiedlichen Spalten passen soll.

> ⚠️ **Achtung:** Binär → CSV passt nicht: Soll nach CSV geschrieben werden, muss auch die Quelle ein CSV-Dataset sein.

> 💡 **Gut zu wissen:** Seit Version 1.2 schreibt diese Pipeline nicht mehr nach `silver/<sensor>/`, sondern nach `silver/processing/<stadt>/<sensor>/` – siehe Schritt 14.

> ✅ **Prüfen:** `silver` enthält `flow`, `humidity`, `temperature`, `weight` mit je einer CSV.

Schritt 13 – Archiv mit Zeitstempel (Historie statt Überschreiben)

> 💡 **Gut zu wissen:** **Hintergrund:** Ein **Archiv mit Zeitstempel** bewahrt jede Lieferung auf, statt sie zu überschreiben. Fehler lassen sich so jederzeit mit den Originaldaten nachvollziehen (Nachweis, Audit).

> **Warum?** Ohne Zeitstempel überschreibt jede neue Lieferung die vorige im Archiv – die Historie geht verloren. Mit Zeitstempel bleibt jede Lieferung nachvollziehbar erhalten (wichtig für Prüfungen und Aufbewahrungspflichten).

1. Neues Dataset `ds_lake_bronze_new_file_bin`: ADLS Gen2 → **Binary** → `lakehouse` / `bronze/new`, Parameter `file_name`, Dateiname `@dataset().file_name`. Damit wird Datei → Datei kopiert.
2. Dataset `ds_lake_bronze_archive_bin`: Parameter `folder_name` und `file_name`, Reiter **Verbindung**: **Verknüpfter Dienst** `ls_adls_mabeehavenlakehouse`, **Dateipfad** `lakehouse` / `@concat('bronze/archive/', dataset().folder_name)` / `@dataset().file_name` – Archivordner und Dateiname mit Zeitstempel werden je Datei eingesetzt. **Komprimierungstyp** bleibt **Keine Komprimierung** (Archiv bleibt direkt lesbar).



Abb. 53: ds_lake_bronze_archive_bin: Verzeichnis und Dateiname über Parameter.

3. Pipeline `pl_bronze_new_to_archive` → in die Schleife → `move_file_to_archive` → Reiter **Source (Quelle)**: `ds_lake_bronze_new_file_bin`, `file_name` = `@item().name`, **File path in dataset (Dateipfad im Dataset)**, **Delete files after completion (Dateien nach Abschluss löschen)** anhaken – Kopieren + Löschen = Verschieben, `bronze/new` ist danach leer. **Nach letzter Änderung filtern** und **Recursively (Rekursiv)** bleiben leer (alle Dateien, nur dieser Ordner).

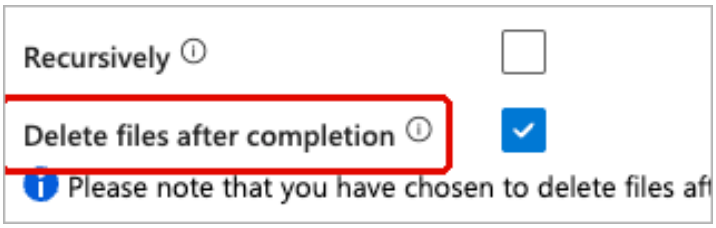


Abb. 54: move_file_to_archive – Quelle: ds_lake_bronze_new_file_bin, file_name = @item().name, Delete files after completion AN (= verschieben).

4. Reiter **Sink (Senke)**: `folder_name` = `@concat(replace(item().name, '.csv', ''), '_raw')` und `file_name` = `@concat(replace(item().name, '.csv', ''), '_raw__', formatDateTime(utcNow(), 'yyyy-MM-dd_HHmmss'), '_UTC.csv')` – je Rohdatei ein eigener Archivordner (`flow_schwartau_raw`), und der UTC-Zeitstempel im Dateinamen verhindert, dass eine spätere Lieferung eine frühere überschreibt.

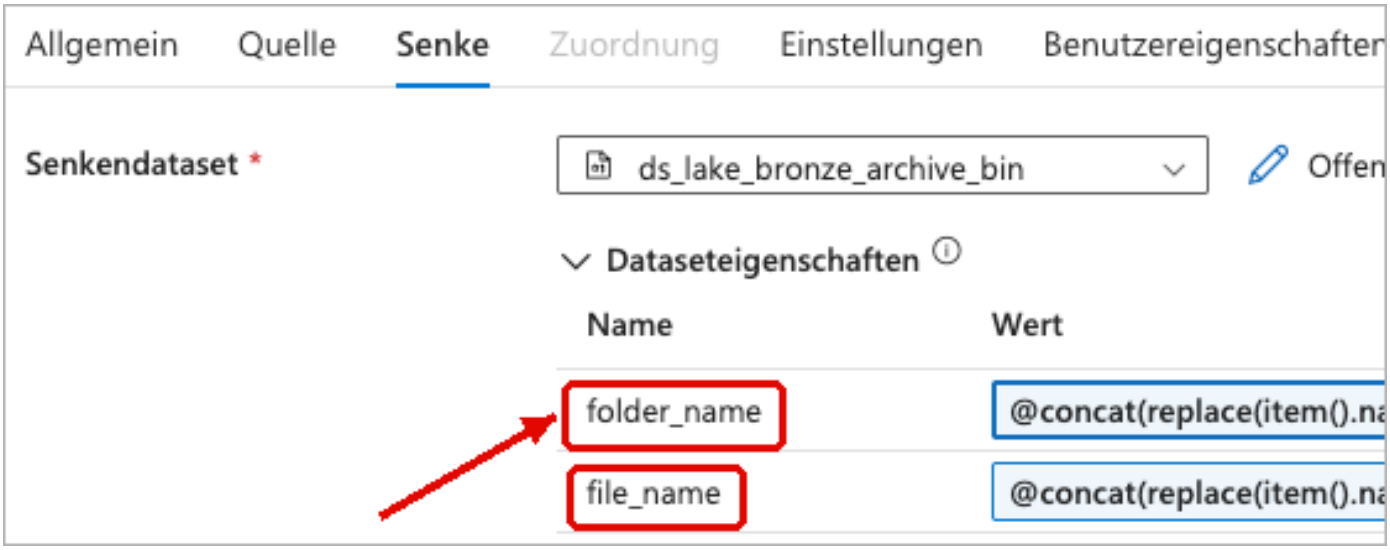


Abb. 55: Senke: folder_name und file_name mit Zeitstempel-Ausdruck.

5. **Validate** → **Debug** → Gegenprobe im Portal: `lakehouse` → `bronze` → `archive`: je Rohdatei ein Ordner **flow_schwartau_raw** … **weight_schwartau_raw** (die einzelnen CSV-Dateien darunter stammen aus einem früheren Testlauf ohne Ordner).

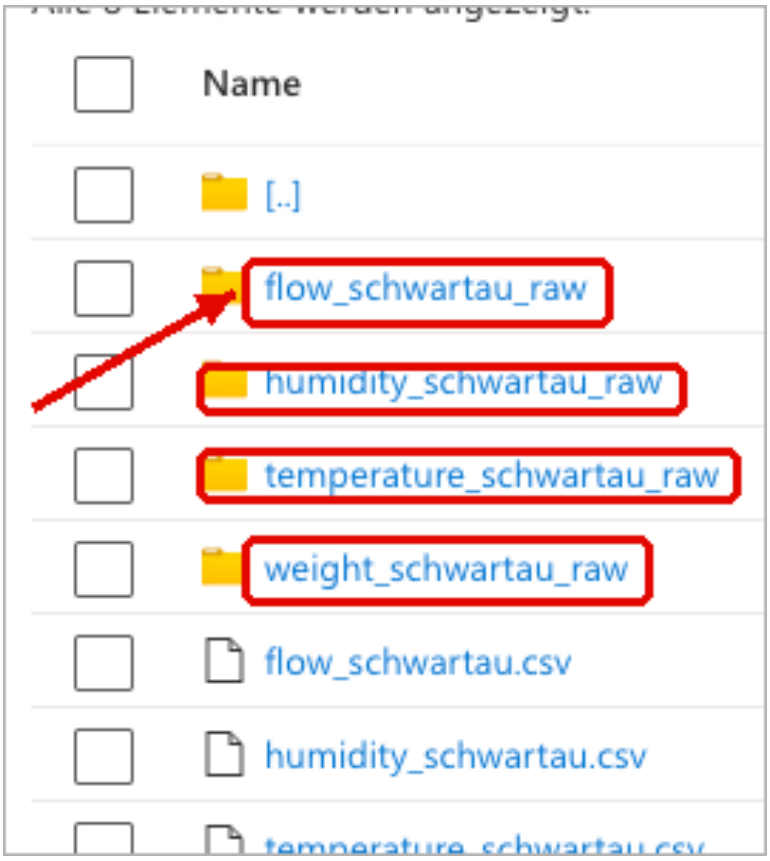


Abb. 56: Ergebnis: je Quelle ein Ordner …_raw in bronze/archive.

6. Ordner **flow_schwartau_raw** öffnen: Darin liegt **flow_schwartau_raw__2026-09-24_154714_UTC.csv** – Name, Doppel-Unterstrich und UTC-Zeitstempel. Danach in der Data Factory **Publish all**: Jetzt sind beide neuen Pipelines (`pl_bronze_to_silver`, `pl_bronze_new_to_archive`) und die neuen Datasets vollständig und werden zusammen veröffentlicht → ↻.



Abb. 57: Dateiname mit Zeitstempel in UTC – nichts wird mehr überschrieben.

Ausdrucksteil	Ergebnis
`replace(item().name, '.csv', '')`	flow_schwartau
`'_raw__'`	_raw__
`formatDateTime(utcNow(), 'yyyy-MM-dd_HHmmss')`	2026-09-24_154714
`'_UTC.csv'`	_UTC.csv
`concat(…)`	flow_schwartau_raw__2026-09-24_154714_UTC.csv

⚠ **Achtung:** `HH` (24 Stunden) statt `hh` (12 Stunden): Mit `hh` sind 03:00 und 15:00 nicht unterscheidbar – Dateien können sich überschreiben.

⚠ **Achtung:** Reihenfolge: Erst nach Silver kopieren, DANACH archivieren (verschieben). Wird zuerst verschoben, ist `bronze/new` leer und die Silver-Kopie findet nichts. Wird nur kopiert statt verschoben, bleibt `bronze/new` voll und jeder Lauf verarbeitet alles erneut.

> 💡 **Gut zu wissen:** UTC statt deutscher Zeit: UTC hat keine Sommerzeit-Umstellung und ist eindeutig sortierbar. Die Sensordaten zeigen genau dieses Problem: `temperature_schwartau.csv` hat 24 doppelte Zeitstempel an den Zeitumstellungen (29.10.2017 und 28.10.2018, 02 Uhr).
>
> 💡 **Gut zu wissen:** Idee aus dem Unterricht: Archiv zusätzlich nach Stadt gliedern, z. B. `bronze/archive/schwartau/flow/…` – dafür den Dateinamen zweimal teilen: `split(split(item().name, '_')[1], '.')[0]` = schwartau. So bleibt es auch mit vielen Standorten übersichtlich.

> ✅ **Prüfen:** `bronze/archive` enthält je Quelle einen Ordner `…_raw` mit Dateien `…_raw__<Zeitstempel>_UTC.csv`; `bronze/new` ist leer.

Schritt 14 – Silver aufteilen: processing (Arbeitskorb) und cleaned (bereinigt)

> 💡 **Gut zu wissen:** **Hintergrund:** `processing` ist der **Arbeitskorb** (wird nach jedem erfolgreichen Lauf geleert), `cleaned` enthält die geprüften Ergebnisse mit Zeitstempel und `_archiv`. Getrennte Ordner verhindern, dass halb verarbeitete Daten als fertig gelten.

> **Warum?** Die Bereinigung läuft in Python-Notebooks (eigene Anleitung „Datenbereinigung“). Dafür braucht Silver zwei Bereiche: `processing` = unbereinigte Arbeitskopie für die Notebooks, `cleaned` = bereinigte Ergebnisse. Die Pipeline schreibt nach `processing/<stadt>/<sensor>/` – so sortiert sich jeder neue Standort automatisch ein.

Teil A – Ordner anlegen (Azure-Portal: `mabeehavenlakehouse` → Speicherbrowser → `lakehouse` → `silver`)

1. **+ Add Directory (Verzeichnis hinzufügen)** → `processing` → **OK**.
2. Im selben Ordner `silver` noch einmal **+ Verzeichnis hinzufügen** → `cleaned` → **OK**. In der Liste stehen jetzt **processing** (Arbeitskorb für neue Rohdaten) und **cleaned** (bereinigte Ergebnisse) neben den Sensor-Ordnern aus Schritt 12.

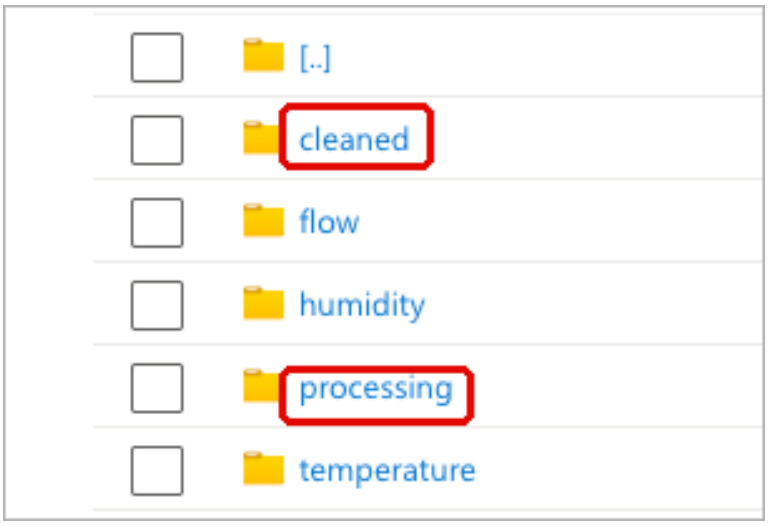


Abb. 58: silver enthält jetzt cleaned und processing (die alten Sensor-Ordner wurden danach gelöscht).

> 💡 **Gut zu wissen:** Eigentlich müsste man Ordner nicht von Hand anlegen: Pipeline, Notebook und der Upload-Dialog (**Advanced (Erweitert)** → **Upload to folder (In Ordner hochladen)**) erzeugen fehlende Ordner automatisch. Die beiden Hauptordner von Hand anzulegen macht die Struktur aber sofort sichtbar.

Teil B – Pipeline auf processing umstellen (ADF Studio: Author → `pl_bronze_to_silver` → in der ForEach-Box ✏️ → `copy_to_silver` → Reiter Sink (Senke))

1. Bei `folder_name` steht noch der alte Ausdruck `@split(item().name, '_')[0]` (schreibt nach `silver/<sensor>/`).



Abb. 59: Vorher: folder_name = @split(item().name, '_')[0] schreibt nach silver/<sensor>/.

2. In das Feld klicken → **Add dynamic content (Dynamischen Inhalt hinzufügen)** → alten Text löschen → `@concat('processing/', split(split(item().name,'_')[1],'.')[0], '/', split(item().name,'_')[0])` → **OK** → oben **Validate (Überprüfen)**. Der Ausdruck baut aus `flow_schwartau.csv` den Ordner `processing/schwartau/flow` – erst die Stadt, dann der Sensor, damit jede Stadt ihren eigenen Arbeitskorb hat.

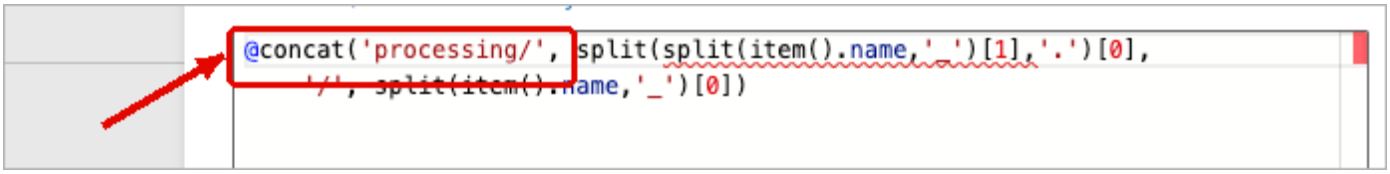


Abb. 60: Nachher: folder_name = @concat('processing/', Stadt, '/', Sensor).

> 💡 **Gut zu wissen:** Die rote Wellenlinie im Editor ist nur ein Hinweis – solange keine rote Fehlermeldung darunter steht, ist der Ausdruck in Ordnung.

Ausdrucksteil	Ergebnis für flow_schwartau.csv
`split(item().name,'_')[1]`	schwartau.csv
`split(…,'.')[0]`	schwartau (Stadt)
`split(item().name,'_')[0]`	flow (Sensor)
`concat('processing/', …)`	processing/schwartau/flow

Teil C – Testen: ingest → silver → archive (ADF Studio)

1. `bronze/new` ist leer (das Archiv hat verschoben), deshalb zuerst neue Rohdaten holen: `pl_ingest_raw_to_bronze` → **Debug** → unten **Ausgabe**: **Pipelinestatus Erfolgreich**, `copy_raw_to_bronze_new` erfolgreich.

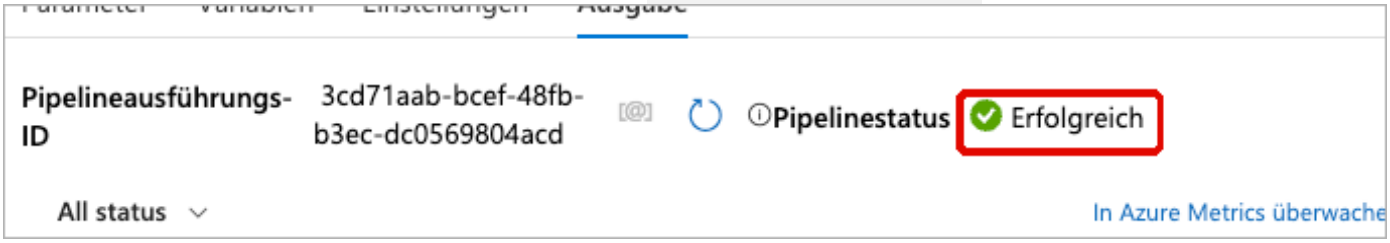


Abb. 61: pl_ingest_raw_to_bronze: bronze/new wieder befüllt.

2. `pl_bronze_to_silver` → **Debug** → warten (4× `copy_to_silver` stehen zuerst „in der Warteschlange“) → ↻ **Refresh** → alle **Erfolgreich**.
3. Gegenprobe im Portal: `lakehouse` → `silver` → `processing`: Ordner **schwartau** ist neu (Spalte **Zuletzt geändert** = Uhrzeit des Laufs).



Abb. 62: processing enthält automatisch den Ordner schwartau (Stadt).

4. Ordner **schwartau** öffnen: 4 Sensor-Ordner **flow**, **humidity**, **temperature**, **weight**, darin je eine CSV (flow 54,23 MiB).



Abb. 63: darin je Sensor ein Ordner – automatisch von der Pipeline angelegt.





Abb. 64: flow/flow_schwartau.csv mit 54,23 MiB – wie die Quelle. (Ausschnitte 1–2)

Teil D – Speichern und aufräumen

1. Oben **Publish all (Alle veröffentlichen)** → Liste prüfen (eine Änderung: `pl_bronze_to_silver`) → unten **Publish (Veröffentlichen)**.



Abb. 65: Alle veröffentlichen: genau eine Änderung (pl_bronze_to_silver).

> ⚠ **Achtung:** Erst unten im Fenster auf **Veröffentlichen** klicken – sonst bleibt der Punkt ● neben der Pipeline und oben „Alle veröffentlichen (1)“ stehen.

2. `pl_bronze_new_to_archive` → **Debug** → unten **Ausgabe**: **Pipelinestatus Erfolgreich**, viermal `move_file_to_archive` (eine je Datei), `foreach_file` und `get_files_bronze_new` erfolgreich – `bronze/new` ist wieder leer.

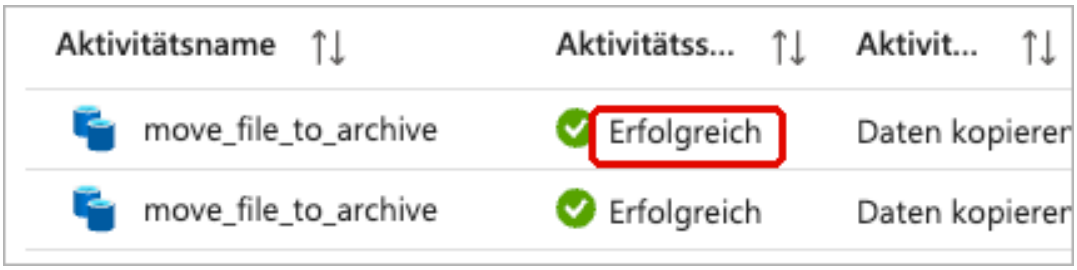


Abb. 66: pl_bronze_new_to_archive: alle 6 Aktivitäten erfolgreich – bronze/new ist wieder leer.

3. Die alten Ordner `silver/flow`, `humidity`, `temperature`, `weight` (Ausgabe der alten Pipeline-Version) markieren → **Delete (Löschen)** → bestätigen. Die Daten liegen weiterhin in `processing` und mit Zeitstempel in `bronze/archive`.

> ⚠ **Achtung:** In Azure gibt es ohne eingeschaltetes „Soft Delete“ keinen Papierkorb – vor dem Löschen die Namen zweimal lesen; `processing` und `cleaned` nicht anhaken.

> ✅ **Prüfen:** `silver` enthält nur noch `processing` und `cleaned`; `processing/schwartau/<sensor>/` hat je eine CSV; `bronze/new` ist leer.

Ordner	Zeitstempel	Warum
`bronze/archive`	✅ UTC	jede Rohdaten-Lieferung dauerhaft
`silver/processing`	❌	nur Arbeitskorb für die Notebooks; wird je Lauf überschrieben
`silver/cleaned`	✅ UTC	jede bereinigte Version – Rückgriff auf den letzten guten Stand

💡 **Gut zu wissen: Frage aus dem Unterricht: „Wäre es bei einer falschen Bereinigung nicht besser, den letzten Stand zu nehmen?“** Genau dafür hat `cleaned` Zeitstempel: Jede Bereinigung ist eine eigene Version; war die letzte falsch, nimmt man die davor. Die Notebooks lesen `processing` nur und überschreiben es nie; ältere Lieferungen liegen zusätzlich in `bronze/archive`.

💡 **Gut zu wissen: Frage: „Brauchen wir nicht eine Master-Pipeline, die alles auf einmal ausführt?“** Ja – sie folgt in Phase 3 (Schritt 24). Einzelne Pipelines zuerst ist Praxis: jede einzeln testbar, wiederverwendbar; die Master-Pipeline steuert nur die Reihenfolge (**Execute Pipeline**, „Wait on completion“). Die Notebook-Bereinigung kommt als Schritt 4 dazu, sobald sie in Azure läuft (z. B. Synapse-Notebook, Kurskapitel „Synapse Notebooks in Azure“).

💡 **Gut zu wissen: Kosten im Blick:** Datenmengen je Stufe (Rohdaten ca. 64 MB CSV, Silver cleaned ca. 82 MB, Gold ca. 0,04 MB) bestimmen Speicher- und Übertragungskosten – Parquet halbiert die Größe gegenüber CSV. Solche Zahlen gehören in jede Dokumentation (Budgetplanung).

Phase 2 – Notebooks: Überblick

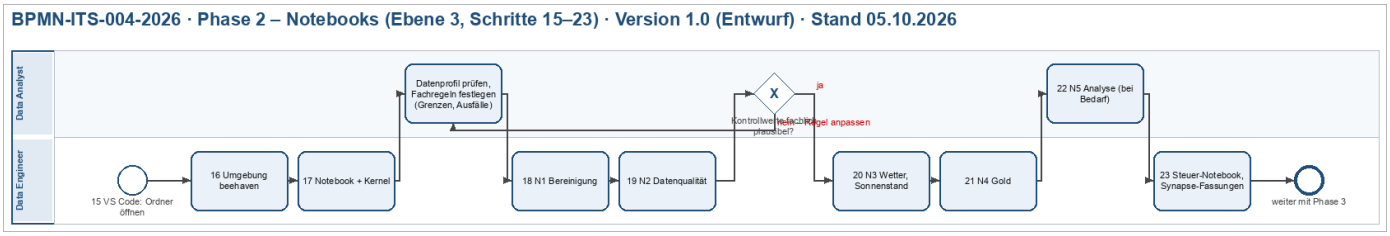


Abb. 67: Phase 2 im Überblick (Ebene 3): Schritte 15–23 nach Rollen.

Die Pipeline in Azure legt die Rohdaten unverändert in `silver/processing/<stadt>/<sensor>/` ab (lokaler Spiegel: `02_daten_ITS-Data/lakehouse/`). Sechs Notebooks machen daraus verlässliche Daten, eine Gold-Tabelle und Auswertungen:

Notebook	Aufgabe	Ergebnis	Läuft
00 `00_steuerung_silver`	startet je Stadt N1 → N2 → N3	–	täglich (Pipeline)
N1 `01_bereinigung`	technisch: Datentypen, UTC, Duplikate, Lücken, fehlende Werte	Zwischenstand (Parquet)	täglich
N2 `02_qualitaet`	fachlich: Messfehler markieren, flow trennen, Ausreißer	`silver/cleaned` (Zeitstempel)	täglich
N3 `03_wetter`	Außenwetter (DWD) und Sonnenstand ergänzen	`silver/cleaned/…/wetter`, `…/sonne`	täglich
N4 `04_gold`	alles zu einer Stundentabelle + Tageswerte, kurze Lücken auffüllen	`gold/<stadt>/` (Parquet)	täglich (Pipeline)
N5 `05_analyse`	Fragen beantworten, Grafiken	`04_images/<stadt>/`	bei Bedarf

> 💡 **Gut zu wissen: Einfach erklärt:** Wie in einer Küche: N1 wäscht die Zutaten, N2 sortiert Verdorbenes aus (markiert es), N3 holt die Beilage (Wetter), N4 richtet den Teller an (Gold), N5 schreibt die Speisekarte (Analyse). Das Steuer-Notebook ist der Küchenchef, der die Reihenfolge vorgibt.

Grundsätze

- **Markieren statt löschen:** Ungültige Werte werden geleert (NaN), die Spalte `qualitaet` nennt den Grund.
- **Silver bleibt Messung, Gold darf schätzen:** Kurze Lücken werden erst in Gold aufgefüllt – mit Kennzeichen-Spalte `…_aufgefuellt`.
- **Eingangsdaten nie überschreiben:** Die Notebooks lesen `processing` nur. Jede neue Version bekommt einen UTC-Zeitstempel; ältere wandern automatisch ins `_archiv` des Ordners.
- **Ein Standort = ein Parameter:** `STADT` und die Standort-Tabelle `konfiguration/standorte.csv` (Koordinaten, Zeitzone). Neue Stadt = neue Zeile, kein neuer Code.
- **Muster je Abschnitt:** Markdown (Regel, Warum, **Erwartung**) → Code (Zeile für Zeile kommentiert) → Markdown **„Ergebnis“**. Die Ergebnisse gehen ins Pflichtenheft.

Ordner und Zeitstempel

Ordner	Zeitstempel	Aufgabe
`quelle/` (lokal) bzw. `source-data`	–	neue Lieferungen `<sensor>_<stadt>.csv`
`bronze/archive`	UTC	jede Rohdaten-Lieferung, dauerhaft
`silver/processing/<stadt>/<sensor>/`	bewusst ohne	aktueller Arbeitskorb, wird je Lauf überschrieben
`silver/cleaned/<stadt>/`	UTC	neueste bereinigte Version; ältere im `_archiv`
`gold/<stadt>/`	im Archiv	Stundentabelle und Tageswerte
`konfiguration/standorte.csv`	–	Standort-Tabelle

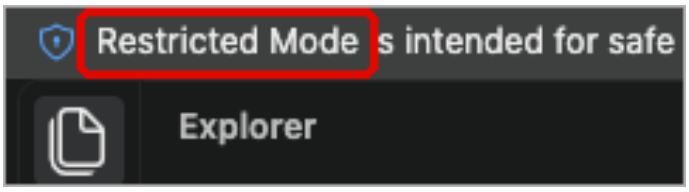
Schritt 15 – VS Code: Projektordner öffnen und vertrauen

> 💡 **Gut zu wissen:** **Hintergrund: VS Code** ist der Editor, in dem Notebooks lokal laufen. „Vertrauen“ (Workspace Trust) erlaubt das Ausführen von Code aus dem Projektordner – nur für eigene, bekannte Ordner geben.

> **Warum?** VS Code arbeitet immer mit einem Ordner. Alle Notebooks, Daten und Dokumente liegen im Projektordner `Schulprojekt` – so finden die Notebooks ihre Daten über relative Pfade.

Teil A – Ordner öffnen (VS Code)

1. Menüleiste **File (Datei)** → **Open Folder… (Ordner öffnen)**.
2. Zum Ordner gehen, in dem das Projekt liegt → `Schulprojekt` **einmal anklicken** (nicht hineingehen) → **Open (Öffnen)**.
3. Links im **Explorer** erscheinen die Projektordner. Oben steht der blaue Balken **Restricted Mode (eingeschränkter Modus)** mit dem Link **Manage (Verwalten)**.



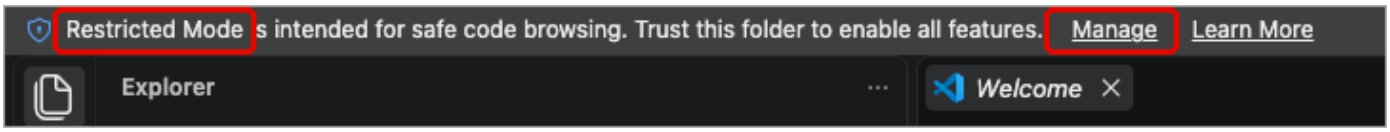


Abb. 68: Projektordner geöffnet – oben der Hinweis „Restricted Mode“ (eingeschränkter Modus). (Ausschnitte 1–2)

Teil B – Ordner vertrauen

1. Im blauen Balken auf **Manage (Verwalten)** klicken.
2. **Trust (Vertrauen)** wählen – erst dann dürfen Notebooks und Erweiterungen Code ausführen. Oben steht danach **You trust this folder (Du vertraust diesem Ordner)** → das Fenster mit **X** schließen.

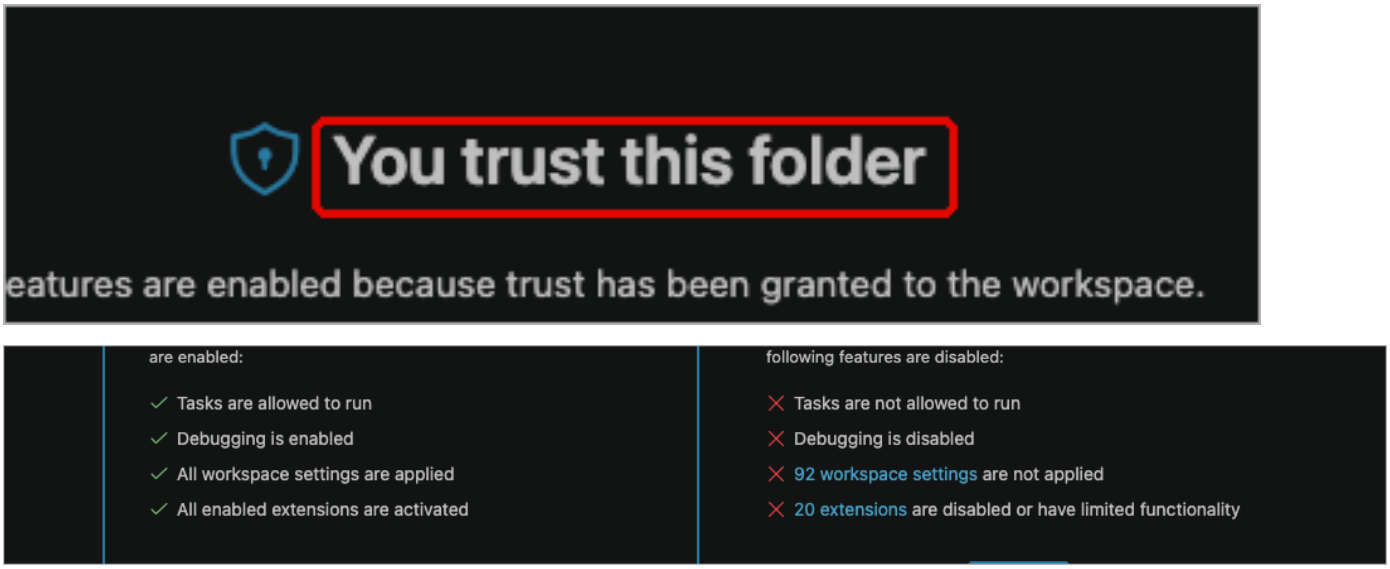


Abb. 69: Workspace Trust: Ordner als vertrauenswürdig markiert – jetzt darf Code laufen. (Ausschnitte 1–2)

> ✅ **Prüfen:** Der blaue Balken und unten links „Restricted Mode“ sind weg; unten links steht `main` (Git-Repo erkannt).

> ⚠️ **Achtung: Terminal-Fehler „Starting directory (cwd) … does not exist“:** VS Code hatte noch einen verschobenen oder gelöschten Ordner geöffnet → richtigen Ordner neu öffnen (Schritt 15, Teil A).

Schritt 16 – Python-Umgebung „beehaven“ mit Conda anlegen

> 💡 **Gut zu wissen: Hintergrund:** Eine **Conda-Umgebung** ist ein abgeschlossener Werkzeugkasten mit festen Python- und Bibliotheksversionen. So läuft das Projekt auf jedem Rechner gleich, unabhängig von anderen Projekten.

> **Warum?** Jedes Projekt bekommt eine **eigene Umgebung** mit festen Versionen der Bibliotheken – so stören sich Projekte nicht, und jeder kann die Umgebung genau nachbauen.

1. Menüleiste **Terminal** → **New Terminal (Neues Terminal)**.
2. Diesen Befehl einfügen und **Enter** drücken:

```
conda create -n beehaven python=3.12 pandas pyarrow matplotlib requests jupyter ipykernel nbclient -y
```

1. Im **Terminal** 1–3 Minuten warten, bis der Hinweis mit `conda activate beehaven` und wieder die Eingabezeile erscheint – dann ist die Umgebung angelegt (die Zeile `WARNING … conda-pypi` ist nur ein Werbehinweis und kann ignoriert werden).

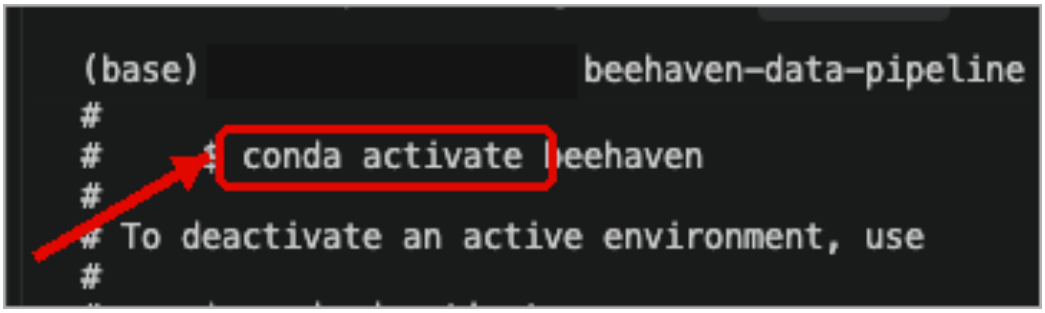


Abb. 70: Conda-Umgebung beehaven ist angelegt (Hinweis „conda activate beehaven“).

> ✅ **Prüfen:** Am Ende steht `done` bzw. der Hinweis `conda activate beehaven`.

Paket	Wofür
`pandas`, `pyarrow`	Tabellen, Parquet-Dateien
`matplotlib`	Diagramme
`requests`	Wetter-Schnittstelle abrufen (N3)
`jupyter`, `ipykernel`, `nbclient`	Notebooks ausführen, auch nacheinander (Steuer-Notebook)

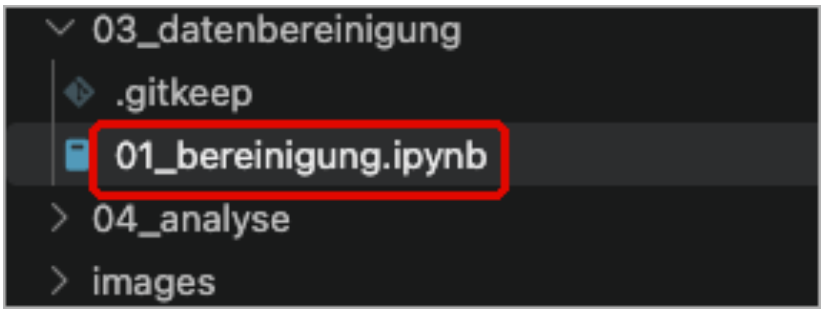
> 💡 **Gut zu wissen: Warum Python 3.12?** Mit 3.12 laufen alle Datenbibliotheken sicher. Die Warnung „Did you know? You can install many PyPI packages with conda“ ist nur ein Hinweis – ignorieren.

Schritt 17 – Notebook öffnen und Kernel wählen

> 💡 **Gut zu wissen: Hintergrund:** Der **Kernel** ist der Rechenmotor hinter dem Notebook. Er muss zur Umgebung „beehaven“ passen, sonst fehlen Bibliotheken oder Versionen weichen ab.

Warum? Ein Notebook besteht aus **Zellen**: Markdown für Text, Code für Python. Der **Kernel** ist die Python-Umgebung, in der der Code läuft – hier `beehaven`.

1. Im Explorer `03_notebooks_ITS-Data` öffnen und das Notebook anklicken (z. B. `01_bereinigung.ipynb`).
2. In der geöffneten Datei `01_bereinigung.ipynb` oben rechts **Select Kernel (Kernel auswählen)** → **Python Environments…** → **beehaven (Python 3.12)** – nur in dieser Umgebung sind pandas und pyarrow installiert. Oben rechts steht danach **beehaven (Python 3.12.14)**.



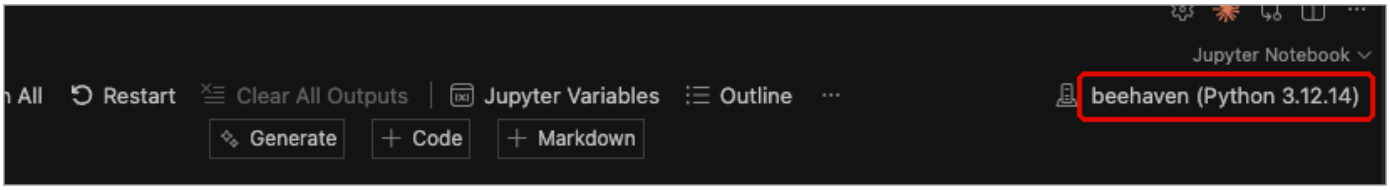


Abb. 71: Neues Notebook 01_bereinigung.ipynb, Kernel beehaven (Python 3.12.14) gewählt. (Ausschnitte 1–2)

3. Zelle ausführen: ⇧ **Shift + Enter** – oder oben **Run All (Alle ausführen)**. Speichern: ⌘ **Cmd + S**.

> ✅ **Prüfen:** Oben rechts steht `beehaven (Python 3.12…)`.

> ⚠️ **Achtung: Häufige Fehler:** Code in eine Markdown-Zelle kopiert (erscheint als große Überschrift) → Zelle löschen und als Code-Zelle neu einfügen. „NameError“ → vorherige Zellen wurden nicht ausgeführt → **Run All**.

Schritt 18 – N1 Bereinigung (technisch)

> 💡 **Gut zu wissen: Hintergrund: Technische Bereinigung**: Zeitstempel in UTC umrechnen, Duplikate und Lücken zählen, Datentypen prüfen – noch ohne fachliche Bewertung. Rohdaten bleiben unverändert, gearbeitet wird mit Kopien. Die Einheit je Stadt (z. B. Gewicht in g oder kg) steht in `konfiguration/sensoren.csv`.

Warum? N1 macht die Dateien technisch sauber: richtige Datentypen, eindeutige Zeitachse, Prüfung auf Duplikate, versteckte Lücken und fehlende Werte. Ergebnis ist ein Zwischenstand für N2.

Notebook auf einen Blick	
Ziel	Rohdaten eines Standorts technisch sauber machen: Datentypen, UTC, Duplikate, Lücken, fehlende Werte; Gewicht einheitlich in Gramm
Eingang	`silver/processing/<stadt>/<sensor>/<sensor>_<stadt>.csv` · `konfiguration/standorte.csv`, `sensoren.csv`
Ausgang	`02_daten_ITS-Data/zwischenstand/<sensor>_<stadt>_n1.parquet` + `n1_protokoll.csv`

Notebook auf einen Blick	
Abschnitte	0 Parameter/Pfade · 1 Einlesen · 2 Arbeitskopie · 3 Exploration · 4 Datentypen und UTC · 5 Duplikate · 5a Lücken · 6 fehlende Werte · 7 Speichern
Ergebnis	4 Parquet-Dateien; Kontrollwerte je Stadt stehen im Notebook (z. B. Schwartau flow 2.513.836 Zeilen, Würzburg 2.055.520)
Typische Fehler	Stadt fehlt in `standorte.csv` (Abbruch mit Meldung) · Gewicht-Einheit in `sensoren.csv` falsch · Restricted Mode / falscher Kernel

#	Regel	Warum
R1	Zeitstempel → Datum/Uhrzeit	als Text nicht sortier-, gruppier- oder berechenbar
R2	Zeit → UTC (erst danach Duplikate prüfen)	02:00 Uhr gibt es im Oktober doppelt
R3	fehlende Werte nur zählen	erfundene Werte täuschen Messungen vor
R4	gleiche flow-Zeilen behalten	viele Minuten mit 0 = echte Messungen

Kapitel 0 – Parameter, Bibliotheken, Pfade

```
# --- Parameter: Standort (neue Stadt = Zeile in lakehouse/konfiguration/standorte.csv + Ordner in silver/
processing) ---
STADT = "schwartau"                                    # Standort = Unterordner in bronze/silver/gold
und Teil der Dateinamen

# --- Bibliotheken laden ---
import pandas as pd                      # pandas: Tabellen (DataFrames) einlesen, bereinigen, speichern
from pathlib import Path                 # Path: Ordner- und Dateipfade unabhängig vom Betriebssystem
angeben
from datetime import datetime, timezone  # für den UTC-Zeitstempel im Dateinamen der Ausgabe

# --- Pfade festlegen (gleiche Struktur wie in Azure) ---
PROJEKT = Path.cwd().parent                                # Projektordner = eine Ebene über
03_notebooks_ITS-Data
SILVER = PROJEKT / "02_daten_ITS-Data" / "lakehouse" / "silver"    # lokaler Spiegel von lakehouse/silver
PROCESSING = SILVER / "processing" / STADT              # Eingang: unbereinigte Arbeitskopie (nur LESEN!)
CLEANED = SILVER / "cleaned" / STADT                    # Ausgang: bereinigte Dateien mit Zeitstempel

# --- Standortdaten aus der Konfiguration (Koordinaten, Zeitzone) ---
standorte = pd.read_csv(PROJEKT / "02_daten_ITS-Data" / "lakehouse" / "konfiguration" / "standorte.csv",
index_col="stadt")
if STADT not in standorte.index:                           # unbekannte Stadt -> sofort abbrechen statt mit
falschen Daten weiterrechnen
    raise ValueError(f"Standort '{STADT}' fehlt in konfiguration/standorte.csv – Zeile mit Koordinaten und
Zeitzone ergänzen")
ORT = standorte.loc[STADT]                                 # eine Zeile: name, breite, laenge, zeitzone,
quelle

# --- Sensor-Regeln je Standort (Einheit der Rohdaten, gültiger Bereich) und bekannte Ausfallzeiträume ---
KONFIG = PROJEKT / "02_daten_ITS-Data" / "lakehouse" / "konfiguration"                  # Ordner mit den
Konfigurationstabellen
sensoren_cfg = pd.read_csv(KONFIG / "sensoren.csv")                                     # eine Zeile je
Stadt und Sensor
REGELN = sensoren_cfg[sensoren_cfg["stadt"] == STADT].set_index("sensor")               # Regeln dieser
Stadt: einheit_roh, min, max
ausschluesse = pd.read_csv(KONFIG / "ausschluesse.csv")                                 # bekannte
Ausfallzeiträume (z. B. Stock nicht auf Waage)
AUSSCHLUESSE = ausschluesse[ausschluesse["stadt"] == STADT].copy()                      # nur Zeiträume
dieser Stadt
AUSSCHLUESSE[["von_utc", "bis_utc"]] = AUSSCHLUESSE[["von_utc", "bis_utc"]].apply(pd.to_datetime,
utc=True)  # Text -> UTC-Zeit
ZEITZONE = ORT["zeitzone"]                                 # Ortszeit der Rohdaten (Deutschland: Europe/
Berlin)

# --- Kontrolle: Welche Dateien liegen im Eingang? ---
for datei in sorted(PROCESSING.glob("*/*.csv")):           # alle CSV in den Sensor-Unterordnern suchen
    groesse_mb = datei.stat().st_size / 1_000_000          # Dateigröße von Byte in Megabyte umrechnen
    print(datei.relative_to(PROJEKT), f"{groesse_mb:.1f} MB")  # Pfad (ab Projektordner) und Größe
ausgeben
```

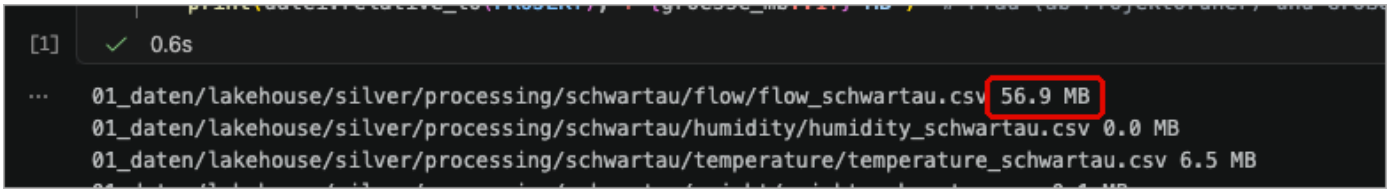


Abb. 72: Kapitel 0: Pfade gesetzt – die 4 CSV-Dateien im Eingang werden gefunden.

> 💡 **Gut zu wissen: Parameter `STADT`:** Das Notebook liest Koordinaten und Zeitzone aus `konfiguration/standorte.csv`. Fehlt die Stadt, bricht es mit einer klaren Meldung ab – lieber kein Ergebnis als ein falsches.

Kapitel 1 bis 3 – Einlesen, Arbeitskopie, Exploration

```
# --- Alle 4 CSV-Dateien einlesen ---
sensoren = ["flow", "humidity", "temperature", "weight"]  # Liste der Sensoren (= Ordnernamen in
processing)
df = {}                                                   # leeres Dictionary: Sensorname -> Tabelle
(DataFrame)
for s in sensoren:                                        # für jeden Sensor nacheinander ...
    pfad = PROCESSING / s / f"{s}_{STADT}.csv"          # ... den Dateipfad zusammensetzen
    df[s] = pd.read_csv(pfad)                             # ... die CSV als Tabelle einlesen (nur lesen!)
    print(f"{s:12} {len(df[s]):>10,} Zeilen   Spalten: {list(df[s].columns)}")  # Kontrolle ausgeben
```

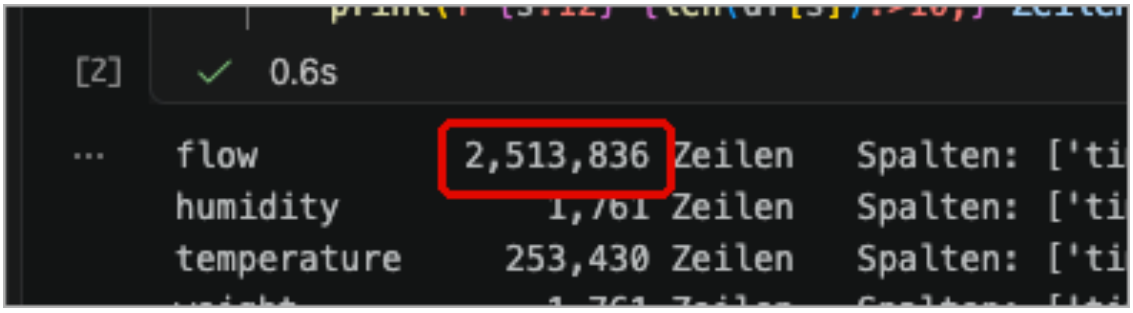


Abb. 73: Kapitel 1 und 2: Dateien eingelesen und Arbeitskopien angelegt.

> ✅ **Prüfen:** flow 2.513.836 · humidity 1.761 · temperature 253.430 · weight 1.761 Zeilen; `timestamp` ist Text → R1; Minimum humidity −100 und weight −172 → Aufgaben für N2.

Kapitel 4 – Datentypen und Zeitzone (R1, R2)

```
# --- Funktion: deutsche Ortszeit (Text) -> eindeutige UTC-Zeit ---
def zu_utc(zeit_text):
    """Wandelt Zeitstempel-Text in deutscher Ortszeit (mit Sommerzeit) in UTC um."""
    zeit = pd.to_datetime(zeit_text, format="%Y-%m-%d %H:%M:%S")  # R1: Text -> Datum/Uhrzeit (Format fest
vorgegeben)
    neustart = (zeit.diff() < pd.Timedelta(hours=-2)).cumsum()  # Zeit springt weit zurück = neue
Messreihe (flow hat 2)
    wiederholung = zeit.groupby([neustart, zeit]).cumcount()    # 0 = Uhrzeit kommt zum 1. Mal vor, 1 =
zum 2. Mal
    sommerzeit = (wiederholung == 0).to_numpy()                 # 1. Vorkommen = noch Sommerzeit, 2. =
schon Winterzeit
    zeit = zeit.dt.tz_localize(ZEITZONE,                        # R2: Zeitzone des Standorts zuordnen
(Standort-Tabelle) ...
                               ambiguous=sommerzeit,            # ... doppelte Oktober-Stunde nach der
Regel oben auflösen
                               nonexistent="raise")             # ... Fehler melden, falls eine Zeit in
der März-Lücke liegt
    return zeit.dt.tz_convert("UTC")                            # ... und in UTC umrechnen (keine
Sommerzeit)
```

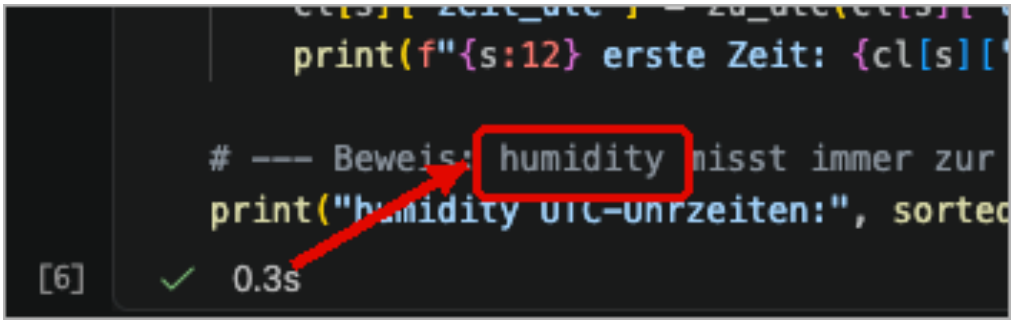


Abb. 74: Kapitel 4: UTC-Umrechnung – humidity misst danach immer 00:00 und 12:00 Uhr UTC.

> 💡 **Gut zu wissen:** **Einfach erklärt:** Deutschland stellt zweimal im Jahr die Uhr um. Im Oktober gibt es 02:00–02:59 **zweimal**, im März **gar nicht**. UTC ist die Einheitszeit ohne Umstellung – jede Minute gibt es genau einmal. Deshalb erst nach UTC umrechnen, dann Duplikate prüfen (abweichend von der Kursvorlage, Befund E1).

Kapitel 5 und 5a – Duplikate und versteckte Lücken

```
# --- Duplikate prüfen (nach der UTC-Umrechnung) ---
for s in sensoren:                                                    # für jeden Sensor ...
    reihe = (cl[s]["zeit_utc"].diff() < pd.Timedelta(hours=-2)).cumsum()  # ... Messreihe bestimmen (flow:
0 und 1)
    doppelte_zeit = cl[s].assign(reihe=reihe).duplicated(subset=["reihe", "zeit_utc"]).sum()  # ...
gleiche Zeit in derselben Reihe zählen
    gleiche_rohzeilen = df[s].duplicated().sum()                      # ... komplett gleiche Zeilen in
den ROHdaten zählen
    print(f"{s:12} doppelte UTC-Zeiten: {doppelte_zeit:>3}   gleiche Rohzeilen: {gleiche_rohzeilen:>9,}")
# ... Ergebnis ausgeben
```

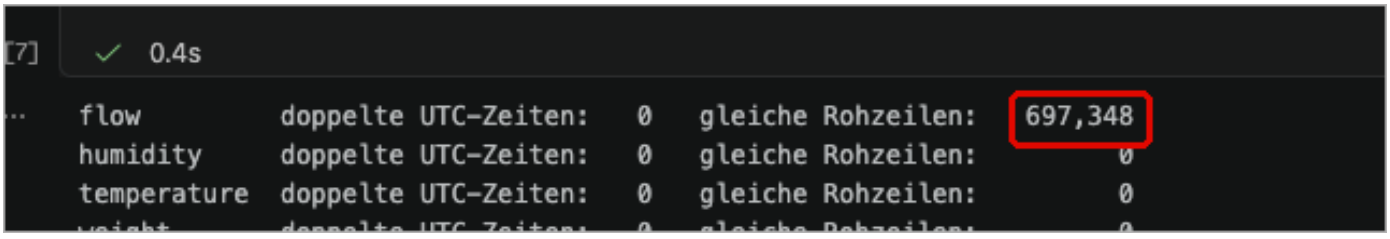


Abb. 75: Kapitel 5: keine doppelten UTC-Zeiten; 697.348 gleiche flow-Zeilen = echte Nullwerte.

```
# --- Zeitabstände je Messreihe prüfen (nach UTC) ---
luecken_protokoll = []                                                # Sammlung der Ergebnisse
for s in sensoren:                                                    # für jeden Sensor ...
    zeit = cl[s]["zeit_utc"]                                          # ... UTC-Zeitspalte
    reihe = (zeit.diff() < pd.Timedelta(hours=-2)).cumsum()           # ... Messreihe (flow: 0 =
Ausflug, 1 = Einflug)
    abstand = zeit.groupby(reihe).diff().dropna()                     # ... Abstand zur vorherigen Zeile
derselben Reihe
    normal = abstand.mode()[0]                                        # ... häufigster Abstand =
normales Messintervall
    zu_gross = abstand[abstand > normal]                              # ... alle Abstände größer als
normal = Lücken
    luecken_protokoll.append({"sensor": s, "normal": str(normal),     # ... Ergebnis festhalten:
                              "luecken": len(zu_gross),               #     Anzahl Lücken
                              "davon_ueber_1_tag": int((zu_gross > pd.Timedelta(days=1)).sum()),
                              "groesste": str(zu_gross.max()) if len(zu_gross) else "-"})
pd.DataFrame(luecken_protokoll)                                       # Übersicht als Tabelle
```

> ✅ **Prüfen:** Keine doppelten UTC-Zeiten. flow: 120 versteckte Lücken (Zeilen fehlen, obwohl nirgends NaN steht), größte 5 Tage.

Kapitel 6 und 7 – Fehlende Werte zählen, Zwischenstand speichern

```
# --- N1-Ergebnis speichern (Zwischenstand für N2) ---
ZWISCHENSTAND = PROJEKT / "02_daten_ITS-Data" / "zwischenstand"       # Ordner für Zwischenergebnisse
zwischen den Notebooks
ZWISCHENSTAND.mkdir(parents=True, exist_ok=True)             # Ordner anlegen, falls er noch nicht
existiert
for s in sensoren:                                           # für jeden Sensor ...
    ziel = ZWISCHENSTAND / f"{s}_{STADT}_n1.parquet"        # ... Dateiname festlegen
    cl[s].to_parquet(ziel, index=False)                      # ... als Parquet speichern (behält
Datentypen inkl. UTC)
    groesse_mb = ziel.stat().st_size / 1_000_000             # ... Dateigröße in MB
    print(f"{s:12} gespeichert: {ziel.relative_to(PROJEKT)}  ({groesse_mb:.1f} MB)")  # ... Kontrolle
ausgeben
pd.DataFrame(protokoll).to_csv(ZWISCHENSTAND / "n1_protokoll.csv", index=False)       # Protokoll
(fehlende Werte) sichern
print("Protokoll gespeichert: n1_protokoll.csv")                                       # Kontrolle
```

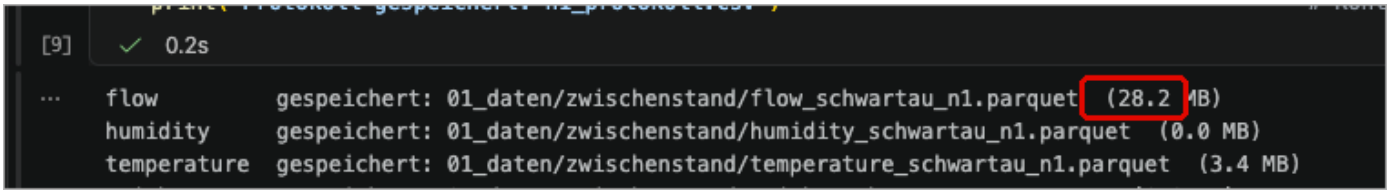


Abb. 76: Kapitel 7: Zwischenstand als Parquet gespeichert (flow 28,2 MB).

> 💡 **Gut zu wissen: Warum Parquet?** Parquet behält die Datentypen (UTC-Zeit bleibt Zeit) und ist viel kleiner als CSV (Zusatz Z1: 13–18 % des Speichers).

Schritt 19 – N2 Datenqualität (fachlich)

> 💡 **Gut zu wissen: Hintergrund: Fachliche Prüfung**: Werte, die physikalisch oder fachlich nicht stimmen können, werden **markiert statt gelöscht** (Spalte `qualitaet`). Grenzen und bekannte Ausfallzeiten kommen je Stadt aus der Konfiguration – nicht aus dem Code.

> **Warum?** N2 prüft, ob die Werte fachlich sinnvoll sind, markiert Messfehler und Ausreißer und bringt flow in zwei Spalten. Ergebnis: `silver/cleaned` mit UTC-Zeitstempel.

Notebook auf einen Blick	
Ziel	Fachlich unplausible Werte markieren (nicht löschen), flow in Ausflug/Einflug trennen, Ausreißer kennzeichnen
Eingang	Zwischenstand aus N1 · `konfiguration/sensoren.csv` (Grenzen) · `ausschluesse.csv` (Ausfallzeiträume)
Ausgang	`silver/cleaned/<stadt>/<sensor>/<sensor>_<stadt>_cleaned__<UTC>_UTC.csv` (neue Version je Lauf) · Grafik `04_images/<stadt>/d2_…`
Abschnitte	R5 Gewicht unter Grenze · R5a Ausfallzeiträume · R6 Feuchte 0–100 % · R7a Temperatur –40 … 60 °C · R7 Frost behalten · R8 flow trennen · R9 z-Score · D2 Mindestabweichung · Speichern
Ergebnis	Spalte `qualitaet` mit Grund je Wert, Spalten `…_auffaellig`; Zusammenfassung je Stadt am Ende des Notebooks
Typische Fehler	N1 nicht gelaufen (kein Zwischenstand) · Grenze in `sensoren.csv` in falscher Einheit · Ausfallzeitraum in Ortszeit statt UTC eingetragen

#	Regel	Ergebnis
R5	weight unter der Grenze aus `sensoren.csv` (20 kg) → ungültig	Schutzregel; Würzburg: leere Waage
R5a	Zeiträume aus `ausschluesse.csv` → ungültig (Schwartau 28.04.–02.07.2018, Stock nicht auf der Waage, E5)	Schwartau 130 ungültig
R6	humidity außerhalb 0–100 % → ungültig (−100 = Fehlercode)	Schwartau 31, Würzburg 6 ungültig
R7a	temperature außerhalb −40 … 60 °C → ungültig (z. B. −227 °C)	Würzburg 1 ungültig
R7	temperature < 0 °C im gültigen Bereich behalten (Frost ist real)	Schwartau 4.033, Würzburg 23.755 behalten
R8	flow in Ausflug (negativ) und Einflug (positiv) trennen	1.256.918 Minuten
R9	Ausreißer mit robustem z-Score nur markieren; Temperatur erst ab 1 °C Abweichung (D2)	Spalte `…_auffaellig`

Beispiel – Zeitraum E5 markieren (R5a)

```
# --- R5a: bekannte Ausfallzeiträume (konfiguration/ausschluesse.csv, z. B. Schwartau E5 „Stock nicht auf
Waage“) als ungültig markieren ---
w = q["weight"]                                                    # kurzer Name für die weight-Tabelle
for _, a in AUSSCHLUESSE[AUSSCHLUESSE["sensor"] == "weight"].iterrows():   # je eingetragenem Zeitraum
dieser Stadt ...
    im_zeitraum = w["zeit_utc"].between(a["von_utc"], a["bis_utc"])  # ... True für alle Messungen im
Zeitraum (inkl. Grenzen)
    betroffen = im_zeitraum & (w["qualitaet"] != "fehlend")         # fehlende Werte behalten ihren Grund
"fehlend"
    print(a["grund"], "| im Zeitraum:", im_zeitraum.sum(), "| davon bisher ok:", (betroffen &
(w["qualitaet"] == "ok")).sum())  # Kontrolle (Schwartau: 130 | 28)
    w.loc[betroffen, "qualitaet"] = f"ungueltig: {a['grund']}"      # einheitlicher Grund für den ganzen
Zeitraum
    w.loc[betroffen, "weight"] = float("nan")                       # Messwert leeren (markieren statt
Zeile löschen)
w["qualitaet"].value_counts()                                      # Kontrolle: Anzahl je Qualitätsstufe
```

> 💡 **Gut zu wissen:** **Einfach erklärt:** Vom 28.04. bis 02.07.2018 zeigte die Waage zeitweise 0 kg – der Stock stand nicht (vollständig) darauf. Eine Honigernte erklärt das nicht (Kasten und Bienen bleiben auf der Waage). Die Werte bleiben in der Datei, sind aber als ungültig markiert.

flow trennen (R8)

```
# --- R8: flow in Ausflug (Reihe 0) und Einflug (Reihe 1) trennen ---
f = q["flow"]                                                          # kurzer Name für die flow-
Tabelle
reihe = (f["zeit_utc"].diff() < pd.Timedelta(hours=-2)).cumsum()       # Zeitsprung zurück = 2.
Messreihe beginnt
print(f.groupby(reihe)["flow"].agg(["min", "max", "count"]))           # Kontrolle: Wertebereich + Anzahl
je Reihe
aus = f[reihe == 0][["zeit_utc", "timestamp", "flow"]].rename(columns={"flow": "flow_aus"})  # Reihe 0 =
Ausflug
ein = f[reihe == 1][["zeit_utc", "flow"]].rename(columns={"flow": "flow_ein"})             # Reihe 1 =
Einflug
q["flow"] = aus.merge(ein, on="zeit_utc", how="outer", indicator=True)   # je Minute nebeneinanderstellen
print(q["flow"]["_merge"].value_counts())                              # Kontrolle: "both" = beide Werte
vorhanden
q["flow"] = q["flow"].drop(columns="_merge")                           # Hilfsspalte wieder entfernen
q["flow"]["qualitaet"] = "ok"                                          # Qualitätsspalte (flow hatte
keine Lücken)
```

> 💡 **Gut zu wissen:** **Vorzeichen belegt:** Laut Datensatz-Beschreibung (Kaggle, HOBOS) steht eine positive Zahl für Ankünfte, eine negative für Abflüge. Das Kurs-Notebook ordnet umgekehrt zu – wir folgen der Quelle.

Ausreißer (R9) und Mindestabweichung (D2)

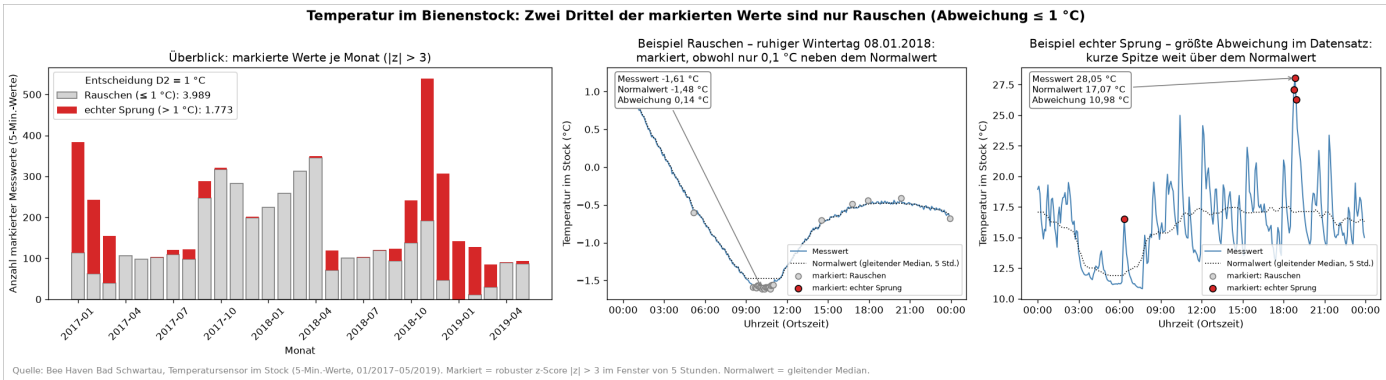


Abb. 77: D2: Zwei Drittel der markierten Temperaturwerte sind nur Rauschen (Abweichung ≤ 1 °C).

Speichern nach silver/cleaned

```
# --- Bereinigte Daten mit UTC-Zeitstempel speichern ---
stempel = datetime.now(timezone.utc).strftime("%Y-%m-%d_%H%M%S")     # aktueller Zeitpunkt in UTC (HH = 24
Stunden)
protokoll = []                                                        # leeres Protokoll
for s in sensoren:                                                    # für jeden Sensor ...
    ordner = CLEANED / s                                              # ... Zielordner cleaned/<stadt>/
<sensor>/
    ordner.mkdir(parents=True, exist_ok=True)                         # ... anlegen, falls nicht vorhanden
    ziel = ordner / f"{s}_{STADT}_cleaned__{stempel}_UTC.csv"       # ... Dateiname mit Zeitstempel
    q[s].to_csv(ziel, index=False)                                    # ... als CSV speichern (neue
Version, nichts wird überschrieben)
    for alt in sorted(ordner.glob(f"{s}_{STADT}_cleaned__*_UTC.csv")):  # ... ältere Versionen ins _archiv
des Ordners (nichts wird gelöscht)
        if alt != ziel:
            (ordner / "_archiv").mkdir(exist_ok=True)
            alt.rename(ordner / "_archiv" / alt.name)
    protokoll.append({"sensor": s, "zeilen": len(q[s]),               # ... Protokollzeile: Zeilen +
                      **q[s]["qualitaet"].value_counts().to_dict()}) #     Anzahl je Qualitätsstufe
    print("gespeichert:", ziel.relative_to(PROJEKT))                  # ... Kontrolle
pd.DataFrame(protokoll).fillna(0)                                     # Protokoll als Tabelle anzeigen
```

> ✅ **Prüfen:** Neue Datei `<sensor>_<stadt>_cleaned__<UTC>.csv`; ältere Versionen liegen im `_archiv` des Sensor-Ordners.

Schritt 20 – N3 Wetter und Sonnenstand

> 💡 **Gut zu wissen: Hintergrund:** Eine **API** (Programmierschnittstelle) liefert Daten auf Anfrage – hier Wetter von Bright Sky (Deutscher Wetterdienst) als **JSON** (halbstrukturiert). Die Rohantwort wird unverändert in Bronze archiviert, die aufbereitete Tabelle in Silver.

Warum? Der Temperatursensor misst **im** Stock (Sommer konstant 34–35 °C, Brutnest). Für Fragen zum Flugverhalten braucht man das **Außenwetter** und die Tageszeit (Tag, Dämmerung, Nacht).

Notebook auf einen Blick	
Ziel	Außenwetter (stündlich) und Sonnenstand (Tag/Dämmerung/Nacht) für den Standort bereitstellen
Eingang	Koordinaten aus `standorte.csv` · Zeitraum aus N1 · Schnittstelle Bright Sky (DWD)
Ausgang	Rohantwort `bronze/archive/<stadt>/wetter/` · `silver/cleaned/<stadt>/wetter/` und `…/sonne/` · Grafik `04_images/<stadt>/n3_…`
Abschnitte	1 Zeitraum · 2 Abruf und Archiv · 3 Stationen (W1) · 4 Lücken (W2) · 5 Sonnenstand · 6 Gegenprüfung Open-Meteo · 7 Aufbereiten · 8 Grafik · 9 Speichern
Ergebnis	Stundenwerte lückenlos, Sonnenschein nachts = 0; Hauptstation je Messgröße mit Entfernung
Typische Fehler	Schnittstelle kurz nicht erreichbar (Notebook wiederholt bis zu 4-mal) · Wetterstation weit entfernt (Entfernung in der Ausgabe prüfen)

Wetter abrufen (Bright Sky / Deutscher Wetterdienst)

```
# --- Wetterdaten von Bright Sky (DWD) abrufen ---
def wetter_abrufen(breite, laenge, von, bis, versuche=4):
    """Stündliches Wetter als JSON holen; bei Fehlern mehrfach versuchen."""
    parameter = {"lat": breite, "lon": laenge,           # Standort
                 "date": von, "last_date": bis,          # Zeitraum (ganze Tage)
                 "tz": "UTC", "units": "dwd",            # Zeiten in UTC, Einheiten des DWD
                 "max_dist": 50_000}                     # Stationen im Umkreis von 50 km
    fehler = None
    for versuch in range(1, versuche + 1):               # mehrere Versuche (kurze Störungen überbrücken)
        try:
            antwort = requests.get("https://api.brightsky.dev/weather", params=parameter, timeout=120)
            if antwort.status_code == 200:               # 200 = erfolgreich
                return antwort.json()                    # leere/kaputte Antwort -> Fehler -> nächster
Versuch
            fehler = f"Status {antwort.status_code}"
        except (requests.RequestException, ValueError) as e:   # Netzwerkfehler oder ungültiges JSON
            fehler = repr(e)
        time.sleep(5 * versuch)                          # 5, 10, 15 … Sekunden warten, dann erneut
versuchen
    raise RuntimeError(f"Bright Sky nach {versuche} Versuchen nicht erreichbar ({fehler})")

roh = wetter_abrufen(BREITE, LAENGE, start.strftime("%Y-%m-%d"), (ende +
pd.Timedelta(days=1)).strftime("%Y-%m-%d"))
wetter_roh = pd.json_normalize(roh["weather"])           # Liste der Stundenwerte → Tabelle
stationen = pd.json_normalize(roh["sources"])            # Liste der Wetterstationen → Tabelle

# --- Rohdaten unverändert archivieren (Parquet, mit UTC-Zeitstempel) ---
stempel = datetime.now(timezone.utc).strftime("%Y-%m-%d_%H%M%S")        # Zeitpunkt des Abrufs
BRONZE_WETTER.mkdir(parents=True, exist_ok=True)                         # Ordner anlegen, falls nötig
wetter_roh.to_parquet(BRONZE_WETTER / f"wetter_roh__{stempel}_UTC.parquet", index=False)
stationen.to_parquet(BRONZE_WETTER / f"stationen_roh__{stempel}_UTC.parquet", index=False)
print(f"{len(wetter_roh):,} Stundenwerte · {wetter_roh.shape[1]} Spalten · {len(stationen)} Stationen")
```

> 💡 **Gut zu wissen: Wiederholversuche:** Bleibt die Schnittstelle kurz stumm, versucht das Notebook es bis zu viermal mit Wartezeit – eine kurze Störung bricht die tägliche Pipeline nicht ab. Die Rohantwort wird unverändert in `bronze/archive/<stadt>/wetter/` archiviert.

Sonnenstand berechnen (NOAA) und gegenprüfen (Open-Meteo)

```
# --- Sonnenhöhe nach NOAA ("General Solar Position Calculations") ---
def sonnenhoehe(zeit_utc, breite=BREITE, laenge=LAENGE):
    """Sonnenhöhe in Grad für UTC-Zeitpunkte (Genauigkeit ca. ±2 Minuten bei Auf-/Untergang)."""
    tag = zeit_utc.dt.dayofyear.to_numpy()                                  # Tag im Jahr (1–366)
    std = (zeit_utc.dt.hour + zeit_utc.dt.minute / 60).to_numpy()           # Uhrzeit UTC als Dezimalstunde
    g = 2 * np.pi / 365 * (tag - 1 + (std - 12) / 24)                       # Jahreswinkel (Bogenmaß)
    zeitgl = 229.18 * (0.000075 + 0.001868 * np.cos(g) - 0.032077 * np.sin(g)
                       - 0.014615 * np.cos(2 * g) - 0.040849 * np.sin(2 * g))   # Zeitgleichung in Minuten
    dekl = (0.006918 - 0.399912 * np.cos(g) + 0.070257 * np.sin(g) - 0.006758 * np.cos(2 * g)
            + 0.000907 * np.sin(2 * g) - 0.002697 * np.cos(3 * g) + 0.00148 * np.sin(3 * g))  #
Deklination der Sonne (Bogenmaß)
    ortszeit_sonne = std * 60 + zeitgl + 4 * laenge                         # wahre Sonnenzeit in Minuten
    stundenwinkel = np.radians(ortszeit_sonne / 4 - 180)                    # Stundenwinkel (Bogenmaß)
    b = np.radians(breite)                                                  # Breitengrad (Bogenmaß)
    return np.degrees(np.arcsin(np.sin(b) * np.sin(dekl) + np.cos(b) * np.cos(dekl) *
np.cos(stundenwinkel)))

def tageszeit(hoehe):
    """Sonnenhöhe → Tag / Dämmerung / Nacht."""
    return np.select([hoehe > -0.833, hoehe > -6], ["Tag", "Dämmerung"], "Nacht")

# --- Sonnenzeiten je Tag aus einer Minutenreihe ableiten ---
minuten = pd.Series(pd.date_range(start.floor("D"), ende.ceil("D"), freq="min", tz="UTC"))  # jede Minute
des Zeitraums
m = pd.DataFrame({"zeit_utc": minuten, "hoehe": sonnenhoehe(minuten)})
m["datum"] = m["zeit_utc"].dt.tz_convert(ZEITZONE).dt.date                          # Kalendertag
(Ortszeit)
tag = m[m["hoehe"] > -0.833].groupby("datum")["zeit_utc"]                                  # Minuten mit
Sonne über dem Horizont
hell = m[m["hoehe"] > -6].groupby("datum")["zeit_utc"]                                      # Minuten
inkl. Dämmerung
sonne = pd.DataFrame({"daemmerung_beginn_utc": hell.min(), "sonnenaufgang_utc": tag.min(),
                      "sonnenuntergang_utc": tag.max(), "daemmerung_ende_utc": hell.max()})
sonne["tageslaenge_h"] = ((sonne["sonnenuntergang_utc"] - sonne["sonnenaufgang_utc"]).dt.total_seconds() /
3600).round(2)
sonne = sonne.reset_index()

# --- Kontrolle an Stichtagen (Ortszeit) ---
zeige = sonne.set_index("datum").loc[[pd.Timestamp(d).date() for d in ["2018-03-20", "2018-06-21",
"2018-09-23", "2018-12-21"]]]
zeige[["sonnenaufgang_utc", "sonnenuntergang_utc"]].apply(lambda s:
s.dt.tz_convert(ZEITZONE).dt.strftime("%H:%M")) \
    .set_axis(["sonnenaufgang_ortszeit", "sonnenuntergang_ortszeit"],
axis=1).assign(tageslaenge_h=zeige["tageslaenge_h"])
```

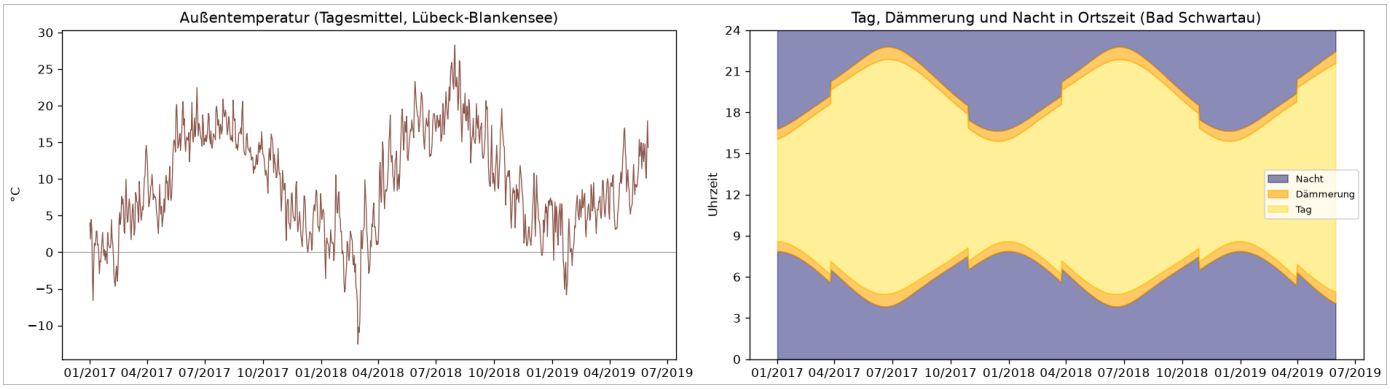


Abb. 78: N3: Außentemperatur und Sonnenzeiten im Jahresverlauf – Plausibilitätsprüfung.

✅ **Prüfen:** 21.145 Stundenwerte; Außentemperatur überwiegend von Lübeck-Blankensee (13 km, Spalte `station_temperatur`); Sonnenzeiten weichen von Open-Meteo im Mittel ca. 2 Minuten ab.

⚠️ **Achtung: Quellenangabe Pflicht:** „Datenbasis: Deutscher Wetterdienst“ (über Bright Sky).

Schritt 21 – N4 Gold: eine Tabelle je Stunde

> 💡 **Gut zu wissen:** **Hintergrund: Gold** fasst alle Quellen zu einer Tabelle zusammen, die ohne weiteres Wissen auswertbar ist (je Stunde und je Tag). Kurze Lücken werden aufgefüllt und **gekennzeichnet** – Analyse und Machine Learning brauchen lückenlose Reihen.

Warum? Dashboard, Analyse und maschinelles Lernen brauchen **eine** Tabelle, in der alle Messungen zeitlich nebeneinander stehen. N4 legt alles auf ein gemeinsames Stundenraster (Vorbild: Kurs-Notebook 05).

Notebook auf einen Blick	
Ziel	Eine analysebereite Gold-Tabelle je Stunde und Tageswerte je Standort bauen; kurze Lücken gekennzeichnet auffüllen
Eingang	neueste Versionen in `silver/cleaned/<stadt>/` (Sensoren, Wetter, Sonne)
Ausgang	`gold/<stadt>/stunden_<stadt>` und `tageswerte_<stadt>` (Parquet + CSV)
Abschnitte	1 Einlesen · 2 nur gültige Werte · 3 Stundentabelle · 4 kurze Lücken auffüllen · 5 Tageswerte · 6 Speichern
Ergebnis	Schwartau 21.121 Stunden / 881 Tage, Würzburg 20.864 Stunden / 870 Tage; Spalten `…_aufgefuellt` zeigen geschätzte Werte
Typische Fehler	`silver/cleaned` leer (`IndexError`) → erst Silver-Stufe · flow wird nie aufgefüllt (gewollt)

Stundentabelle bauen

```
# --- Gemeinsames Zeitraster: jede volle Stunde (UTC) vom ersten bis zum letzten Messwert ---
start = min(daten[s]["zeit_utc"].min() for s in sensoren).floor("h")    # früheste Messung, auf volle
Stunde abgerundet
ende = max(daten[s]["zeit_utc"].max() for s in sensoren).floor("h")     # späteste Messung
gold = pd.DataFrame(index=pd.date_range(start, ende, freq="h", name="zeit_utc"))  # leere Tabelle, eine
Zeile je Stunde

# --- flow: Flugbewegungen je Minute (Einflug + Ausflug als Betrag), auffällige Werte (R9) ausgeschlossen
---
fl = gueltig["flow"].set_index("zeit_utc")
ein = fl["flow_ein"].where(~fl["flow_ein_auffaellig"])                  # Einflug (positiv laut Datensatz-
Beschreibung)
aus = fl["flow_aus"].abs().where(~fl["flow_aus_auffaellig"])            # Ausflug (negativ gemessen ->
Betrag)
je_std = pd.DataFrame({"ein": ein, "aus": aus}).resample("h")           # nach Stunden gruppieren
gold["flug_ein_summe"] = je_std["ein"].sum(min_count=1)                 # Einflüge in der Stunde
gold["flug_aus_summe"] = je_std["aus"].sum(min_count=1)                 # Ausflüge in der Stunde
gold["flug_minuten"] = je_std["ein"].count()                            # wie viele Minuten gemessen
wurden (max. 60)
gold["flug_bewegungen_pro_min"] = (gold["flug_ein_summe"] + gold["flug_aus_summe"]) / gold["flug_minuten"]
# Mittel je Minute

# --- Stocktemperatur: Stundenmittel der 5-Minuten-Werte ---
gold["stock_temperatur_c"] = gueltig["temperature"].set_index("zeit_utc")
["temperature"].resample("h").mean()

# --- Feuchte und Gewicht: Messwert in seiner Messstunde (00:00 und 12:00 UTC) ---
gold["stock_feuchte_pct"] = gueltig["humidity"].set_index("zeit_utc")["humidity"].resample("h").mean()
gold["gewicht_kg"] = gueltig["weight"].set_index("zeit_utc")["weight"].resample("h").mean() / 1000   #
Gramm -> kg

# --- Außenwetter der Stunde (N3) anhängen ---
spalten_wetter = ["aussentemperatur_c", "niederschlag_mm", "sonnenschein_min", "wind_kmh",
"windboeen_kmh",
                  "luftdruck_hpa", "taupunkt_c", "sonnenstrahlung_kwh_m2", "wetterlage",
"sonnenhoehe_grad", "tageszeit"]
gold = gold.join(wetter.set_index("zeit_utc")[spalten_wetter], how="left")
gold.insert(0, "stadt", STADT)                                          # Standort als Spalte (mehrere
Städte in einer Tabelle möglich)
print(f"{len(gold):,} Stunden von {gold.index.min()} bis {gold.index.max()}")
gold.notna().mean().mul(100).round(1).to_frame("gefuellt_%").T         # Kontrolle: Anteil vorhandener
Werte je Spalte
```

Kurze Lücken auffüllen – mit Kennzeichen

```
def kurze_luecken_fuellen(reihe, max_stunden):
    """Nur Lücken bis max_stunden aufeinanderfolgender fehlender Stunden zeitlich interpolieren.
    Gibt (gefüllte Reihe, Kennzeichen True = aufgefüllt) zurück; längere Lücken bleiben leer."""
    fehlt = reihe.isna()
    block = (fehlt != fehlt.shift()).cumsum()                              # Nummer je zusammenhängendem
Block (fehlend / vorhanden)
    laenge = fehlt.groupby(block).transform("sum")                         # Länge des fehlenden Blocks, zu
dem die Stunde gehört
    geschaetzt = reihe.interpolate(method="time", limit_area="inside")    # nur zwischen zwei echten
Werten, nie am Rand
    erlaubt = fehlt & (laenge <= max_stunden) & geschaetzt.notna()        # kurze Lücke mit Nachbarwerten
auf beiden Seiten
    return reihe.where(~erlaubt, geschaetzt), erlaubt

regeln = {"stock_temperatur_c": 2, "stock_feuchte_pct": 11, "gewicht_kg": 11}   # Spalte -> höchstens
aufgefüllte Stunden
for spalte, max_std in regeln.items():
    gold[spalte], gold[f"{spalte}_aufgefuellt"] = kurze_luecken_fuellen(gold[spalte], max_std)
    print(f"{spalte:22} aufgefüllt: {gold[f'{spalte}_aufgefuellt'].sum():>6,} Stunden · weiter leer:
{gold[spalte].isna().sum():>6,}")
```

Spalte	höchstens aufgefüllt	Warum
Stocktemperatur	2 Stunden	ändert sich langsam
Stockfeuchte, Gewicht	11 Stunden	nur zwischen zwei 12-Stunden-Messungen
flow	nie	Zählwerte sind nicht schätzbar

> 💡 **Gut zu wissen:** **Einfach erklärt:** Silver ist das Kassenbuch – nur echte Buchungen. Gold ist der Monatsbericht – dort darf man kurze Lücken glätten, solange es gekennzeichnet ist (`…_aufgefuellt = True`). Lange Lücken wie E5 bleiben leer.

Speichern nach gold/<stadt>/

```
# --- Gold-Tabellen speichern ---
GOLD.mkdir(parents=True, exist_ok=True)                                    # Gold-Ordner anlegen, falls nicht
vorhanden
stempel_gold = pd.Timestamp.now(tz="UTC").strftime("%Y-%m-%d_%H%M%S")     # Zeitpunkt, bis zu dem die alte
Version galt
for name in ["stunden", "tageswerte"]:
    for alt in sorted(GOLD.glob(f"{name}_{STADT}.*")):                     # alte Version vor dem
Überschreiben ins _archiv
        (GOLD / "_archiv").mkdir(exist_ok=True)
        alt.rename(GOLD / "_archiv" / f"{alt.stem}__bis_{stempel_gold}_UTC{alt.suffix}")
gold.to_parquet(GOLD / f"stunden_{STADT}.parquet")                         # Stundentabelle (Hauptergebnis)
gold.to_csv(GOLD / f"stunden_{STADT}.csv")                                 # zum Ansehen
tageswerte.to_parquet(GOLD / f"tageswerte_{STADT}.parquet")               # Tageswerte
tageswerte.to_csv(GOLD / f"tageswerte_{STADT}.csv")
for datei in sorted(GOLD.glob(f"*_{STADT}.*")):                            # Kontrolle: Dateien und Größen
    print(datei.name, f"{datei.stat().st_size / 1000:.0f} KB")
```

> ✅ **Prüfen:** `stunden_<stadt>.parquet` (Schwartau: 21.121 Stunden, ca. 1 MB) und `tageswerte_<stadt>.parquet` (881 Tage); alte Versionen im `_archiv`.

Schritt 22 – N5 Analyse (bei Bedarf)

> 💡 **Gut zu wissen:** **Hintergrund:** Die **Analyse** beantwortet fachliche Fragen mit Grafiken (Titel = Aussage). Sie liest nur aus Silver/Gold und verändert keine Daten.

> **Warum?** N5 beantwortet die Fragen zum Flugverhalten und zur Honigproduktion und erstellt die Grafiken nach Grafik-Standard. Es läuft **nicht** täglich – Grafiken braucht man nur, wenn sich etwas ändert.

Notebook auf einen Blick	
Ziel	Fragen zu Gewicht, Flugverhalten, Wetter und Honigproduktion beantworten; Grafiken nach Grafik-Standard
Eingang	`silver/cleaned/<stadt>/` (Minutenwerte) · Wetter und Sonne aus N3
Ausgang	Grafiken `04_images/<stadt>/` (f1–f3, a2–a10, h1)
Abschnitte	F1 Gewicht · F2 Flugsaison · F3 Temperatur · A2 Tageszeit · A3/A5 Außentemperatur · A4/A6 Regen, Sonne · A7–A10 Feuchte, Strahlung, Wind, Luftdruck · H1 Honig
Ergebnis	Tabellen „Kernaussagen je Stadt“ im Notebook; Vergleich der Standorte im Notebook `06_vergleich_standorte`
Typische Fehler	läuft nicht täglich (nur bei Bedarf) · Ergebnisse unter gleichen Bedingungen lesen, Korrelation ist keine Ursache

- **F1–F3:** Gewicht, Flugsaison, Temperatur
- **A1–A10:** Tageszeit, Außentemperatur, Regen, Sonne, Feuchte, Strahlung, Wind, Luftdruck
- **H1:** Honigproduktion

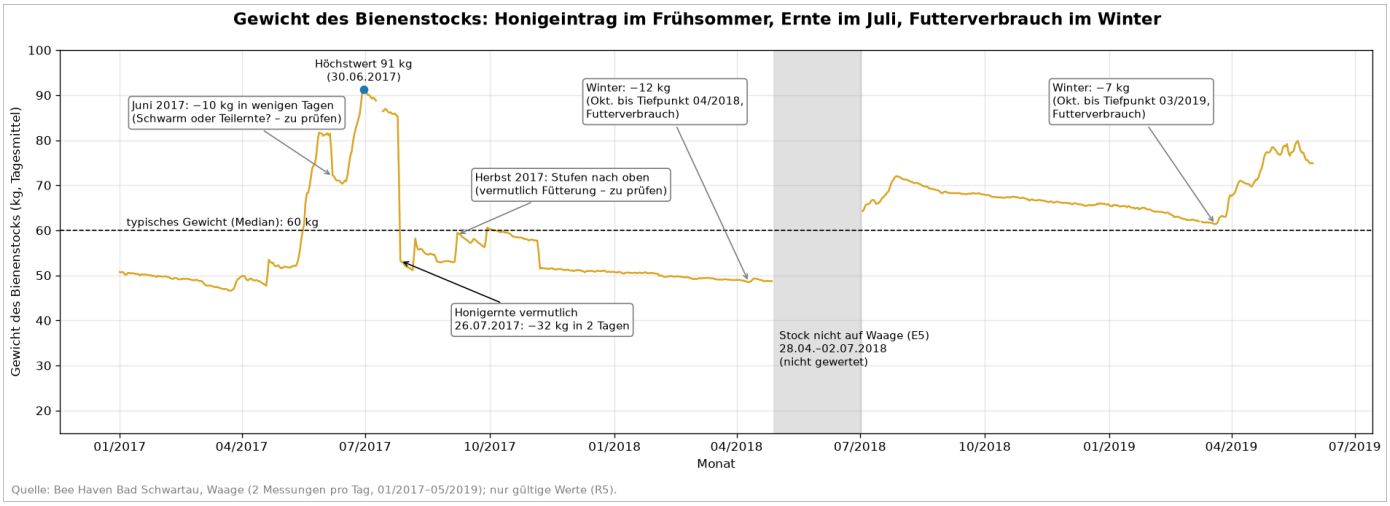


Abb. 79: F1: Gewicht des Bienenstocks (Tagesmittel) – Abfall im Juli = Honigernte, grau = Zeitraum E5 (Stock nicht auf der Waage).

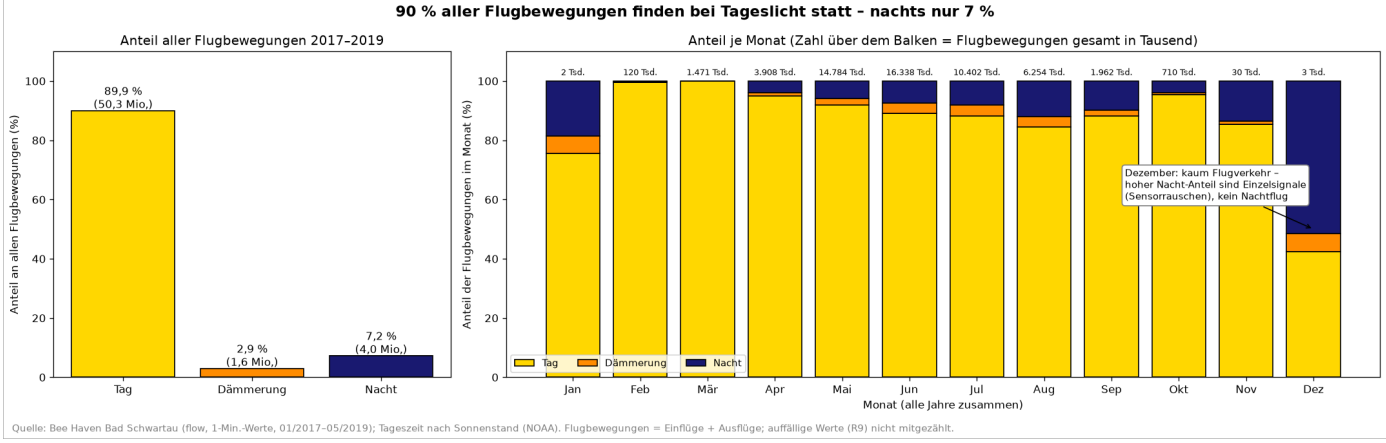


Abb. 80: A2: 90 % der Flugbewegungen bei Tag.

> 💡 **Gut zu wissen:** Jede Frage folgt demselben Muster: **eigene Erwartung** → **Erwartung laut Recherche** (mit Quelle) → Code → **Ergebnis**. Unter gleichen Bedingungen (Tag, kein Regen, 15–25 °C, April–September) werden Scheineffekte vermieden.

Schritt 23 – Steuer-Notebook und Vorbereitung für Azure

> 💡 **Gut zu wissen:** **Hintergrund:** Ein **Steuer-Notebook** startet mehrere Notebooks nacheinander in **einer** Spark-Sitzung (`runMultiple`). Der **Generator** erzeugt aus den lokalen Notebooks die Synapse-Fassungen – Änderungen immer lokal, nie von Hand in Synapse.

Warum? In Azure startet jede Notebook-Aktivität einen eigenen Rechner (Spark-Sitzung) – das kostet Zeit und Geld. Das Steuer-Notebook startet N1 → N2 → N3 für jede Stadt in **einer** Sitzung.

Notebook auf einen Blick	
Ziel	N1 → N2 → N3 für alle Städte aus `standorte.csv` in **einer** Spark-Sitzung starten (Azure) bzw. nacheinander (lokal)
Eingang	`konfiguration/standorte.csv` · Notebooks N1–N3
Ausgang	wie N1–N3; Ausgabe je Teil-Notebook „erfolgreich“ oder Fehler
Abschnitte	Teil A Ablaufplan (DAG) · Teil B lokal testen · Teil C Synapse-Fassungen erzeugen · Stadt-Fassungen (`stadt_fassungen.py <stadt>`)
Ergebnis	Stufe Silver aller Städte in einer Sitzung (Azure 02.10.: ca. 8 Min. für 2 Städte inkl. Start)
Typische Fehler	zweite Spark-Sitzung offen (`AVAILABLE_WORKSPACE_CAPACITY_EXCEEDED`) · Synapse-Fassung von Hand geändert statt neu erzeugt

Teil A – Ablaufplan

```
# --- Ablaufplan (DAG = gerichteter Ablaufgraph, Format von mssparkutils.notebook.runMultiple) ---
SCHRITTE = [("n1", "01_bereinigung"), ("n2", "02_qualitaet"), ("n3", "03_wetter")]   # Kurzname, Notebook
aktivitaeten = []
for stadt in staedte:
    vorher = None
    for kurz, notebook in SCHRITTE:
        aktivitaeten.append({
            "name": f"{kurz}_{stadt}",                                  # eindeutiger Name je Schritt
und Stadt
            "path": f"{notebook}_synapse" if AZURE else notebook,       # Synapse-Kopie bzw. lokales
Notebook
            "timeoutPerCellInSeconds": 1800,                            # eine Zelle höchstens 30
Minuten
            "args": {"STADT": stadt},                                   # überschreibt die
Parameterzelle
            "dependencies": [vorher] if vorher else [],                 # erst nach dem vorigen Schritt
        })
        vorher = f"{kurz}_{stadt}"
ablauf = {"activities": aktivitaeten, "timeoutInSeconds": ZEITLIMIT_S, "concurrency": 1}
for a in aktivitaeten:
    print(f"{a['name']:16} {a['path']:26} nach: {a['dependencies'] or '-'}")
```

💡 **Gut zu wissen: Einfach erklärt:** Statt für jedes Notebook einen Rechner hochzufahren, fährt das Steuer-Notebook einmal hoch und arbeitet die Liste ab – Stadt für Stadt, Schritt für Schritt. Ein Schritt startet erst, wenn der vorige erfolgreich war.

Teil B – lokal testen (VS Code)

1. `00_steuerung_silver.ipynb` öffnen, Kernel `beehaven` wählen.
2. **Run All (Alle ausführen)** – das Notebook erkennt „lokal“ und führt N1, N2, N3 nacheinander aus.

✅ **Prüfen:** Ausgabe „n1_schwartau erfolgreich … n3_schwartau erfolgreich · Stufe Silver fertig in … s“ (Schwartau ca. 1,5 Minuten).

Teil C – Azure-Kopien erzeugen (Generator)

1. Im Terminal (Umgebung `beehaven`) den Generator aufrufen:

```
python Privat/03_notebooks_ITS-Data/generator/make_synapse_notebooks.py
```

✅ **Prüfen:** Im Ordner `03_notebooks_ITS-Data/synapse/` liegen `01_…` bis `04_gold_synapse.ipynb` und das Steuer-Notebook. Unterschiede zu lokal: Parameterzelle, `abfss://`-Pfade, Silver als **Parquet**, kein Kontoschlüssel im Code.

⚠️ **Achtung: Synapse-Kopien nie von Hand ändern** – immer das lokale Notebook ändern und den Generator erneut ausführen. Sonst laufen lokal und Azure auseinander.

Phase 3 – Automatisierung und Betrieb: Überblick

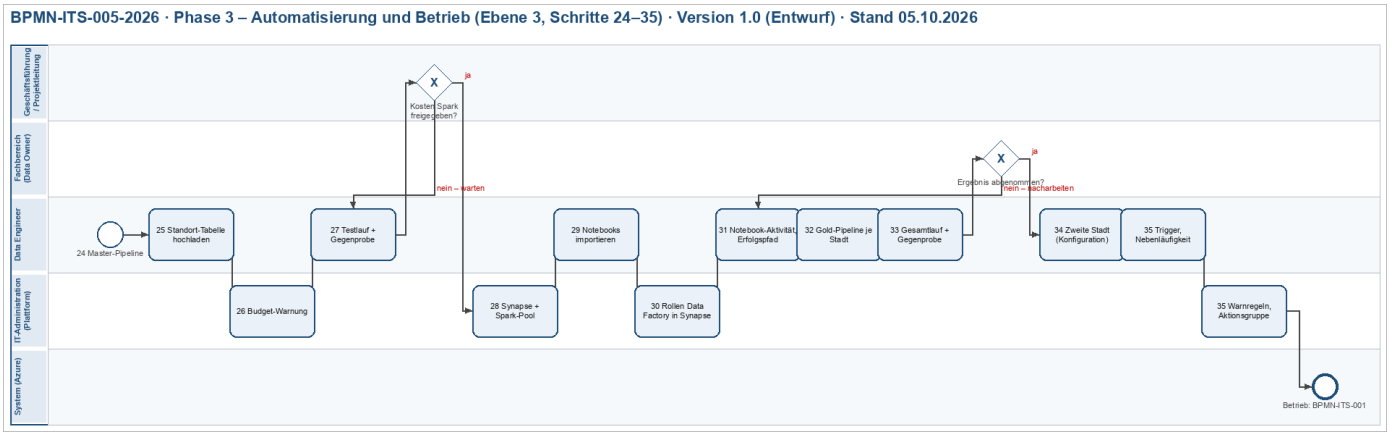


Abb. 81: Phase 3 im Überblick (Ebene 3): Schritte 24–35 mit Freigaben.

Phase 1 hat die einzelnen Pipelines gebaut, Phase 2 die Notebooks. Phase 3 verbindet alles zu **einer Kette mit einem Start** und sichert den Betrieb ab (Kosten, Test, Standorte).

Stufe in pl_daily_processing	ruft auf	Inhalt
`rohdaten_nach_bronze`	`pl_ingest_raw_to_bronze`	Rohdaten aus der Quelle nach `bronze/new`
`bronze_nach_silver`	`pl_bronze_to_silver` mit `silver_automatisierung` und `processing_leeren` (Schritte 30–31)	je Stadt/Sensor nach `silver/processing`, dann N1 → N2 → N3, Arbeitskorb leeren
`silver_nach_gold`	`pl_silver_to_gold` (Schritt 32)	N4 Gold je Stadt aus der Standort-Tabelle
`bronze_archivieren`	`pl_bronze_new_to_archive`	Rohdaten mit UTC-Zeitstempel ins Archiv – immer zuletzt

Schritt 24 – Master-Pipeline pl_daily_processing

> **Warum?** Die drei Pipelines aus Phase 1 sollen in der richtigen Reihenfolge mit **einem** Start laufen – und nur weiter, wenn der vorige Schritt erfolgreich war. Bei einem Fehler bleiben die Rohdaten in `bronze/new`, weil das Archiv erst am Ende läuft.

Teil A – Pipeline anlegen (ADF Studio → Author (Erstellen))

1. Links **Pipelines** → **… (Aktionen)** → **New pipeline (Neue Pipeline)** → Name `pl_daily_processing`.
2. Unter **Aktivitäten** → **Allgemein** dreimal **Execute Pipeline (Pipeline ausführen)** auf die Fläche ziehen.
3. Die Kästen nacheinander mit dem **grünen Haken (bei Erfolg)** verbinden.

Teil B – jede Aktivität einstellen

1. Reiter **Allgemein (General)**: im Feld **Name** den Namen (z. B. `bronze_archivieren`) und im Feld **Beschreibung** den Zweck eintragen (Tabelle unten) – so erkennt man jede Stufe im Monitor ohne die Pipeline zu öffnen. **Aktivitätsstatus** bleibt **Aktiviert** (deaktivierte Aktivitäten werden übersprungen), **Sichere Eingabe (Secure input)** bleibt leer – sonst fehlen die Eingaben im Monitor.

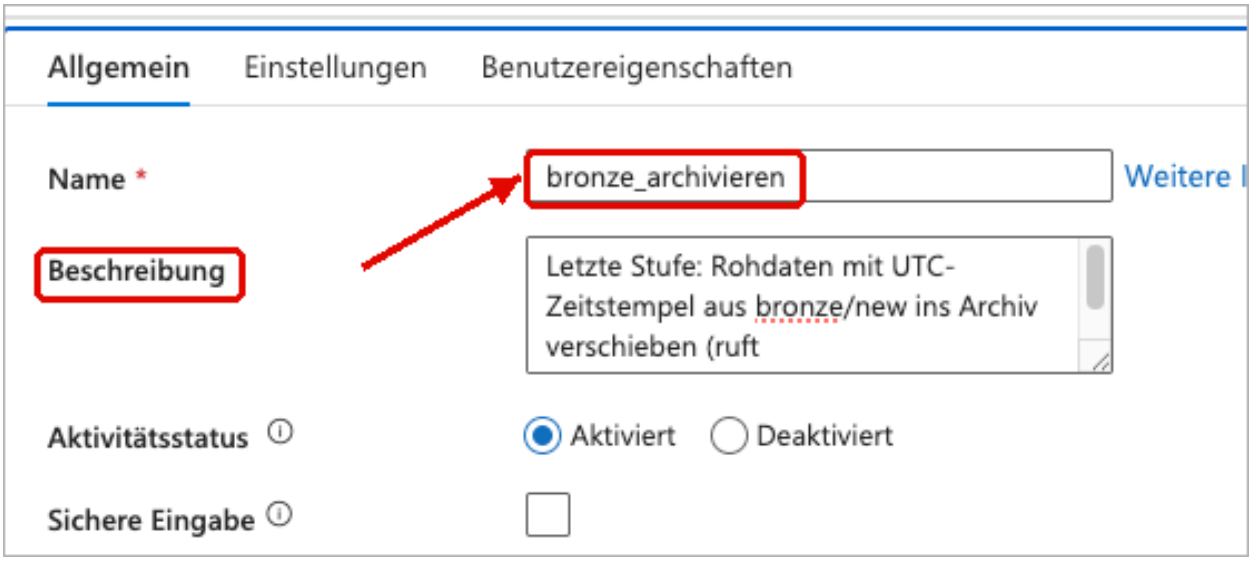


Abb. 82: Master-Pipeline pl_daily_processing: jede Aktivität mit sprechendem Namen und Beschreibung.

2. Reiter **Einstellungen (Settings)**: **Aufgerufene Pipeline (Invoked pipeline)** wählen und **Auf Abschluss warten (Wait on completion)** anhaken – nur dann wartet die Master-Pipeline, bis die Stufe fertig ist, und der grüne Haken bedeutet wirklich „erfolgreich".

Name	Aufgerufene Pipeline	Beschreibung
`rohdaten_nach_bronze`	`pl_ingest_raw_to_bronze`	Stufe 1: Rohdaten aus der Quelle nach bronze/new kopieren
`bronze_nach_silver`	`pl_bronze_to_silver`	Stufe 2: Rohdaten je Stadt und Sensor nach silver/processing kopieren
`bronze_archivieren`	`pl_bronze_new_to_archive`	Letzte Stufe: Rohdaten mit UTC-Zeitstempel ins Archiv verschieben

> 💡 **Gut zu wissen: Namensregeln (Microsoft Learn):** Aktivitätsnamen höchstens 55 Zeichen, Beginn mit Buchstabe, Ziffer oder `_`; nicht erlaubt: `. + ? / < > * % & : \`. Pipelines englisch mit `pl_`, Aktivitäten deutsch und sprechend, ohne Umlaute.

Teil C – prüfen und veröffentlichen

1. Oben **Überprüfen (Validate)** → „Es wurden keine Fehler gefunden".
2. Oben **Alle überprüfen (Validate all)** – keine Fehler – → **Alle veröffentlichen (Publish all)** → **Veröffentlichen (Publish)**. Danach ist **Alle veröffentlichen** grau: Es gibt keine offenen Änderungen mehr, links steht `pl_daily_processing` unter **Pipelines**.

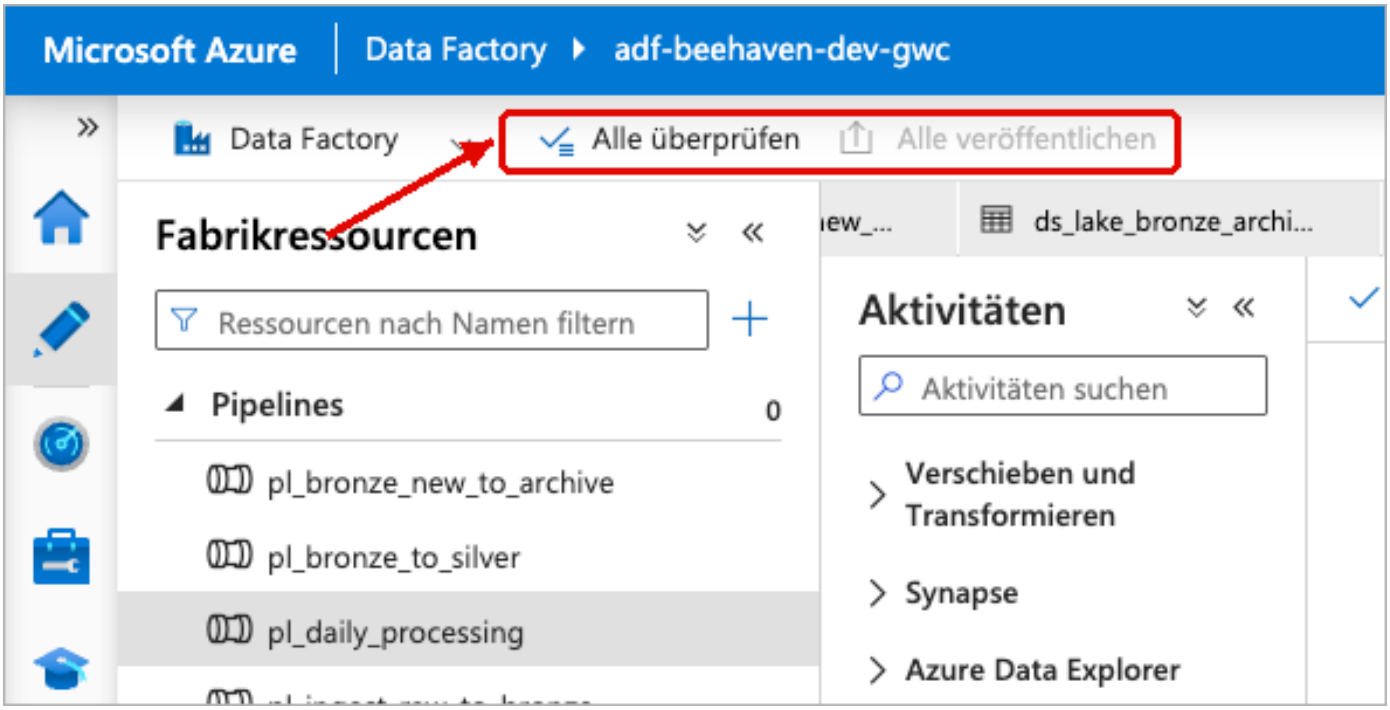


Abb. 83: Nach „Alle veröffentlichen“: kein Zähler mehr an der Schaltfläche, kein Punkt am Pipeline-Namen.

✅ **Prüfen:** An **Alle veröffentlichen** steht kein Zähler mehr, neben `pl_daily_processing` kein Punkt ●.

⚠️ **Achtung: Nur „Überprüfen“ reicht nicht:** Solange **Alle veröffentlichen** ① eine Zahl zeigt, ist die Änderung nur ein Entwurf – ein Trigger würde die alte Fassung starten.

Schritt 25 – Standort-Tabelle in den Data Lake laden

Warum? Die Notebooks lesen Koordinaten und Zeitzone jeder Stadt aus `konfiguration/standorte.csv`. In Azure muss die Datei deshalb im Data Lake liegen. Neue Stadt = neue Zeile, kein neuer Code.

1. Speicherkonto `mabeehavenlakehouse` → **Container** → `lakehouse` → oben **+ Verzeichnis hinzufügen** → rechts im Feld **Name** `konfiguration` eintragen → **Speichern**. Eigener Ordner, weil Einstellungen (Standorte, Sensorgrenzen) keine Messdaten sind und nicht durch Bronze/Silver/Gold wandern.

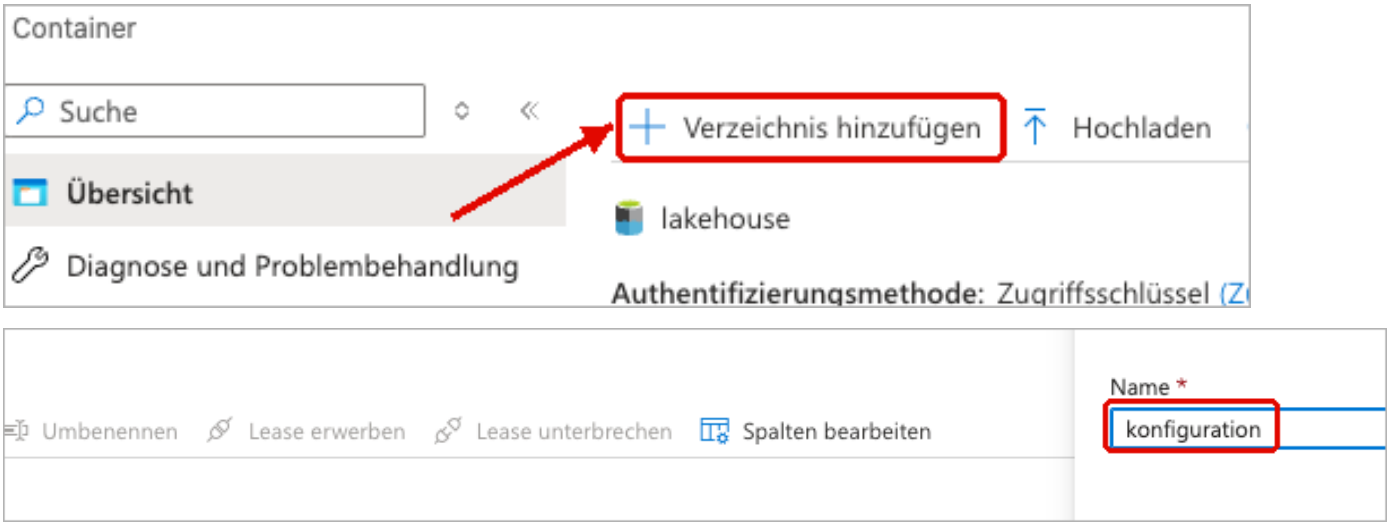


Abb. 84: Container lakehouse: Verzeichnis „konfiguration“ anlegen. (Ausschnitte 1–2)

2. Ordner `konfiguration` öffnen → **Hochladen** → **Nach Dateien suchen** → `standorte.csv` (lokal: `02_daten_ITS-Data/lakehouse/konfiguration/`) → **Hochladen**. Danach steht **standorte.csv** in der Liste.



Abb. 85: Standort-Tabelle standorte.csv hochgeladen (142 B).

> ✅ **Prüfen:** `standorte.csv` liegt in `konfiguration`; die Spalte **Größe** (rechts, im Ausschnitt nicht zu sehen) stimmt mit der lokalen Datei überein (Schwartau: 142 B).

> 💡 **Gut zu wissen:** Bereinigte Silver-Daten werden **nicht** von Hand hochgeladen – die Pipeline erzeugt sie in Azure selbst (als Parquet).

Schritt 26 – Budget-Warnung einrichten

> **Warum?** Bevor Notebooks in Azure laufen (Spark kostet je Minute), soll eine E-Mail warnen, wenn die Kosten steigen. **Die Warnung stoppt nichts** – sie informiert nur.

Teil A – zu den Budgets (Portal)

1. Oben in der Suche `Abonnements` eingeben → unter **Dienste Abonnements (Subscriptions)** öffnen → eigenes Abonnement anklicken. Budgets hängen am Abonnement, weil dort alle Kosten zusammenlaufen.



Abb. 86: Portal: in der Suche „Abonnements“ eingeben und den Dienst öffnen.

2. Links **Kostenverwaltung (Cost Management)** → **Budgets** → **+ Hinzufügen (+ Add)**.

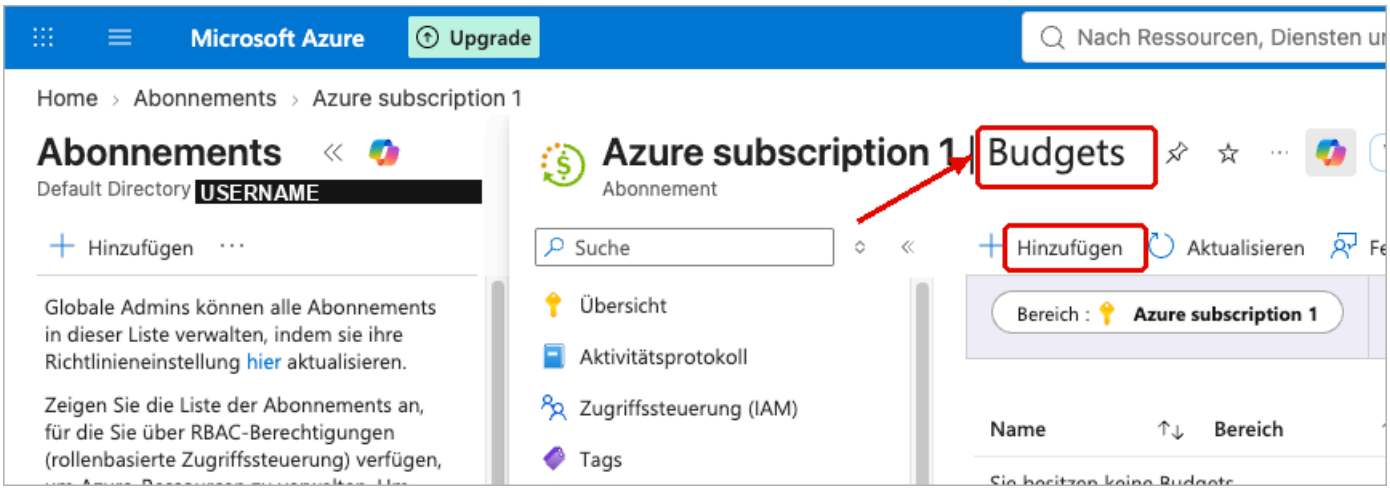


Abb. 87: Abonnement → Kostenverwaltung → Budgets → + Hinzufügen.

Teil B – Budget anlegen

1. Felder ausfüllen (Tabelle): **Name** `budget-beehaven-dev`, **Zurücksetzungszeitraum Monatlich**, **Erstellungsdatum** bleibt der aktuelle Monat (2026, September), **Ablaufdatum Dezember** 2026 (Projektende), darunter **Budgetbetrag** `10` → **Weiter >**. Monatlich, weil Azure monatlich abrechnet – jeden Monat beginnt die Zählung neu.

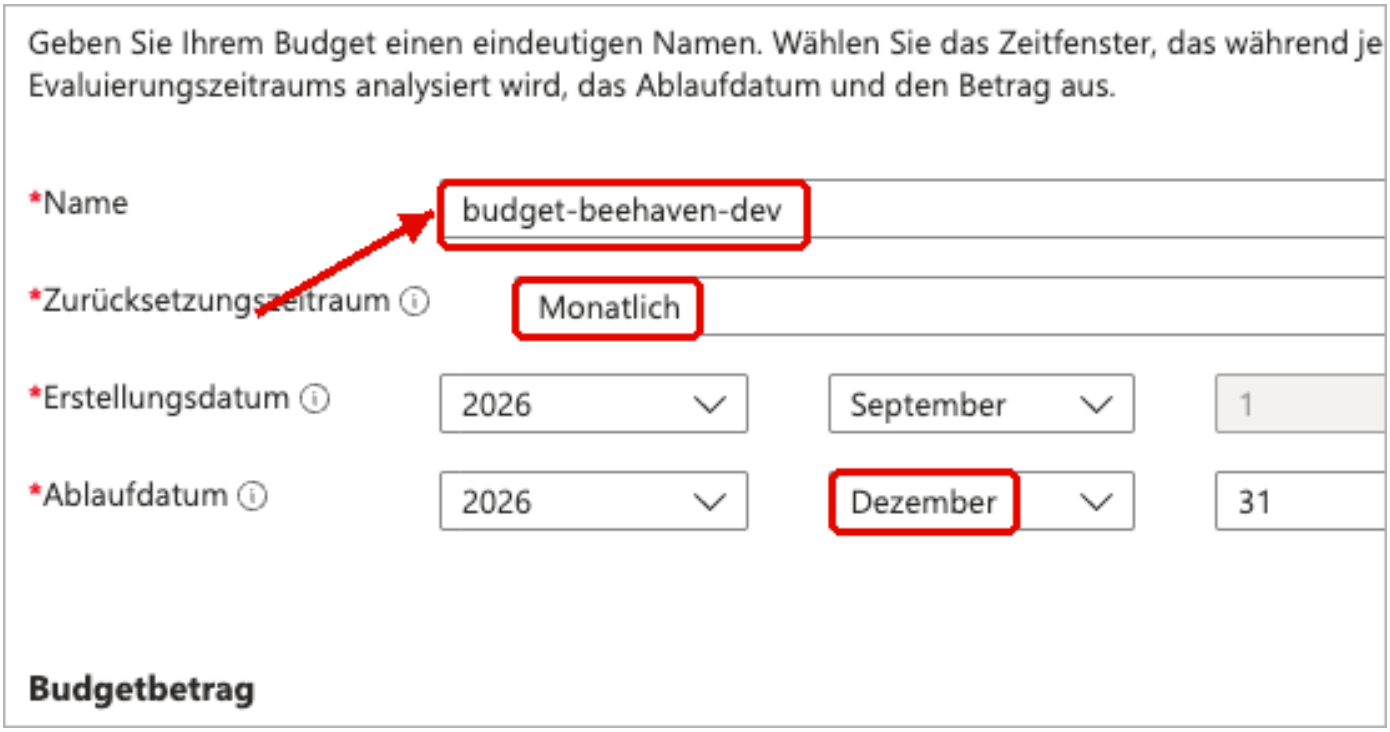
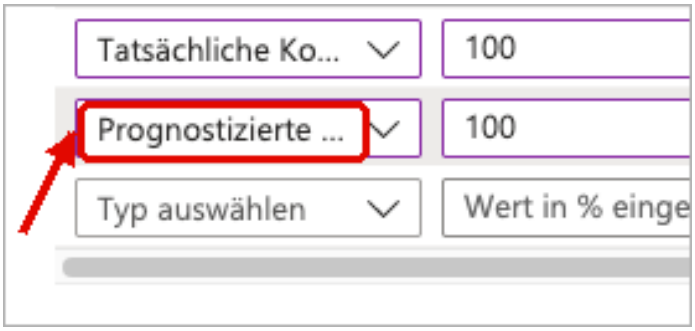


Abb. 88: Budget: Name, monatliche Zurücksetzung, Ablaufdatum, Betrag 10 €.

Feld	Inhalt
Filter	leer lassen (gilt für das ganze Abonnement)
Name	`budget-beehaven-dev`
Zurücksetzungszeitraum	Monatlich
Erstellungs-/Ablaufdatum	aktueller Monat bis Projektende (hier 31.12.2026)
Betrag	`10` (€)

Teil C – Warnungen festlegen

1. Vier Warnungen eintragen (Tabelle unten; im Bild die Zeilen 80 %, 100 % und **Prognostizierte** 100 %), **Aktionsgruppe** bleibt **Keine** (E-Mail reicht). Unter **Warnungsempfänger (E-Mail)** die eigene Adresse, unter **Sprachen Deutsch (Deutschland)** → **Erstellen**. Die Prognose-Warnung meldet sich schon, wenn Azure für den Monat mehr als 10 € erwartet – vor dem tatsächlichen Überschreiten.



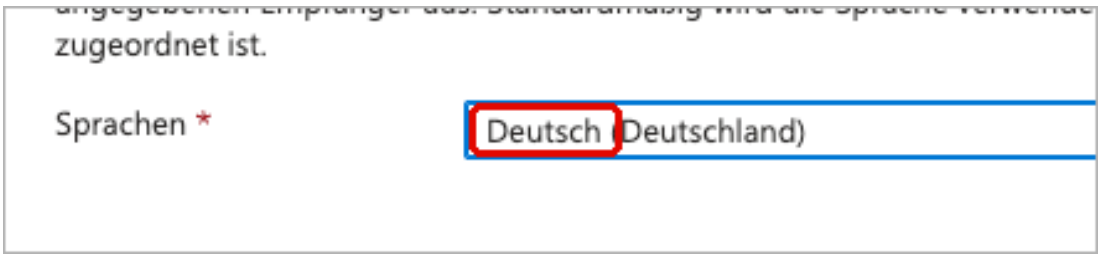


Abb. 89: Warnungen bei 50/80/100 % der tatsächlichen Kosten und bei 100 % Prognose; Empfänger und Sprache. (Ausschnitte 1–2)

2. Zurück in **Budgets**: In der Liste steht **budget-beehaven-dev**; die Spalte **Bereich** zeigt das Abonnement (im Bild geschwärzt) – das Budget gilt für alle Ressourcen darin.

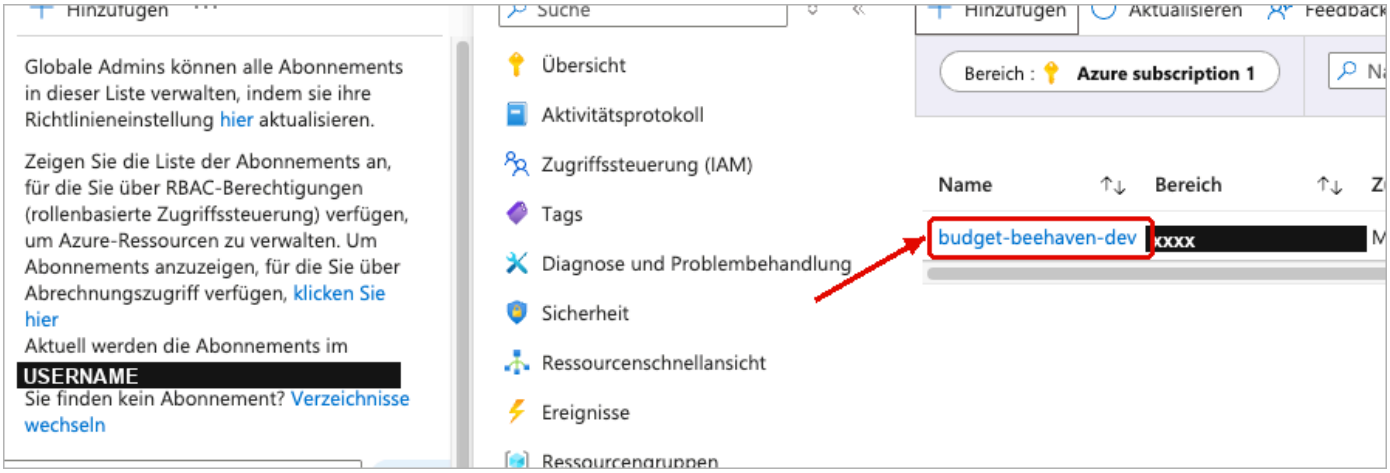


Abb. 90: Fertig: Budget budget-beehaven-dev mit 10 € je Monat.

Typ	% des Budgets	entspricht
Tatsächlich	50	5 €
Tatsächlich	80	8 €
Tatsächlich	100	10 €
Prognostiziert	100	Azure sagt 10 € für den Monat voraus

> ✅ **Prüfen:** In der Budget-Liste steht `budget-beehaven-dev` mit 10,00 €. Die Auswertung beginnt erst einige Stunden nach dem Anlegen.

> 💡 **Gut zu wissen:** **Einfach erklärt:** Wie eine Tankanzeige mit Warnlampe: Sie meldet sich, bevor der Tank leer ist – anhalten muss man selbst.
>
> 💡 **Gut zu wissen:** `azure-noreply@microsoft.com` ins Adressbuch aufnehmen, damit die Warnungen nicht im Spam landen.

Schritt 27 – Testlauf und Gegenprobe

> **Warum?** „Erfolgreich“ im Monitor heißt nur: nichts ist abgebrochen. Ob die Dateien am richtigen Ort liegen, prüft man zusätzlich im Data Lake.

1. ADF Studio → `pl_daily_processing` öffnen → **Debuggen (Debug)**.
2. Unten Reiter **Ausgabe (Output)**, zwischendurch **↻ Aktualisieren** (Kreispfeil neben der **Pipelineausführungs-ID**), bis **Pipelinestatus Erfolgreich** zeigt. In der Liste stehen die drei Stufen `rohdaten_nach_bronze`, `bronze_nach_silver` und `bronze_archivieren` je mit **Erfolgreich**; die Spalte **Dauer** zeigt, wie lange jede Stufe brauchte.

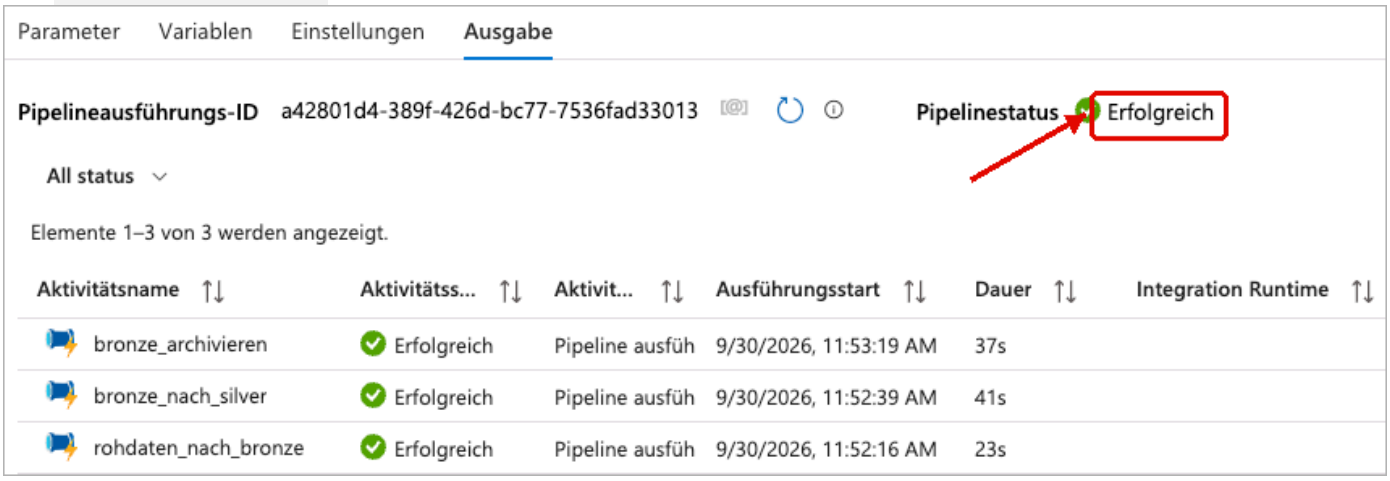


Abb. 91: Testlauf (Debuggen): alle drei Stufen erfolgreich.

Gegenprobe im Data Lake (Portal)

Ordner	Erwartung
`bronze/new`	leer (alles ins Archiv verschoben)
`bronze/archive/<sensor>_<stadt>_raw/`	neue Datei mit heutigem UTC-Zeitstempel, ältere Lieferungen bleiben
`silver/processing/<stadt>/<sensor>/`	Datei heute aktualisiert

1. Speicherkonto `mabeehavenlakehouse` → Container `lakehouse` → `bronze/new` öffnen: Der Ordner ist leer – die Archiv-Stufe hat alle Rohdaten verschoben.
2. `bronze/archive/flow_schwartau_raw/` öffnen: Oben steht die erste Lieferung **flow_schwartau_raw__2026-09-24_154714_UTC.csv**, unten die neue Datei **flow_schwartau_raw__2026-09-30_095330_UTC.csv** mit dem UTC-Zeitstempel des Testlaufs – alle Lieferungen dazwischen bleiben erhalten (Archiv = Nachweis jeder Lieferung).



Abb. 92: Gegenprobe Archiv: neue Datei mit UTC-Zeitstempel, ältere Lieferungen bleiben.

3. `silver/processing/schwartau/flow/` öffnen: **flow_schwartau.csv** liegt dort; in der Spalte **Zuletzt geändert** (rechts, im Ausschnitt nicht zu sehen) steht das heutige Datum – die Silver-Stufe hat die Datei neu geschrieben.



Abb. 93: Gegenprobe silver/processing: Datei heute aktualisiert.

> ✅ **Prüfen:** Schwartau: alle drei Stufen erfolgreich (ca. 23 s, 41 s, 37 s); Archiv und processing aktuell, `bronze/new` leer.

> ⚠️ **Achtung:** **Archiv-Uhrzeit:** Der Dateiname nutzt UTC (z. B. 09:53), das Portal zeigt „Zuletzt geändert“ in Ortszeit (11:53) – beides passt zusammen.

Schritt 28 – Synapse-Arbeitsbereich und Spark-Pool anlegen

> **Warum?** Die Notebooks sollen in Azure laufen. Dafür braucht es einen **Synapse-Arbeitsbereich** (Workspace) und einen **Spark-Pool** (die Rechner, die die Notebooks ausführen). Spark kostet je Minute – deshalb kleinster Pool, automatisches Pausieren und eine Freigabe vor dem ersten Lauf (Human Loop).

Teil A – Arbeitsbereich anlegen (Portal)

1. Oben in der Suche `Azure Synapse Analytics` eingeben → Dienst öffnen → **+ Erstellen**.
2. Reiter **Allgemeine Informationen**: **Abonnement** bleibt `Azure subscription 1`, **Ressourcengruppe** `rg-beehaven-dev-gwc` (alle Projektressourcen in einer Gruppe), **Verwaltete Ressourcengruppe** leer lassen (Azure legt sie selbst an).

Unter **Details zum Arbeitsbereich**: **Arbeitsbereichsname** `syn-beehaven-dev-gwc` (Schema `<dienst>-<projekt>-<umgebung>-<region>`), **Region (Europe) Germany West Central** – dieselbe Region wie der Data Lake (Daten bleiben in Deutschland, keine Übertragungskosten). **Data Lake Storage Gen2 auswählen** = **Aus Abonnement**, **Kontoname** `mabeehavenlakehouse`, **Dateisystemname** → **Neu erstellen** → `synapse` (eigener Container für Protokolle und Ausgaben von Synapse, getrennt von den Projektdaten in `lakehouse`). Den Haken **„Mir selbst die Rolle Mitwirkender an Storage-Blobdaten … zuweisen“** gesetzt lassen – sonst kann man im Synapse Studio die Dateien nicht lesen.

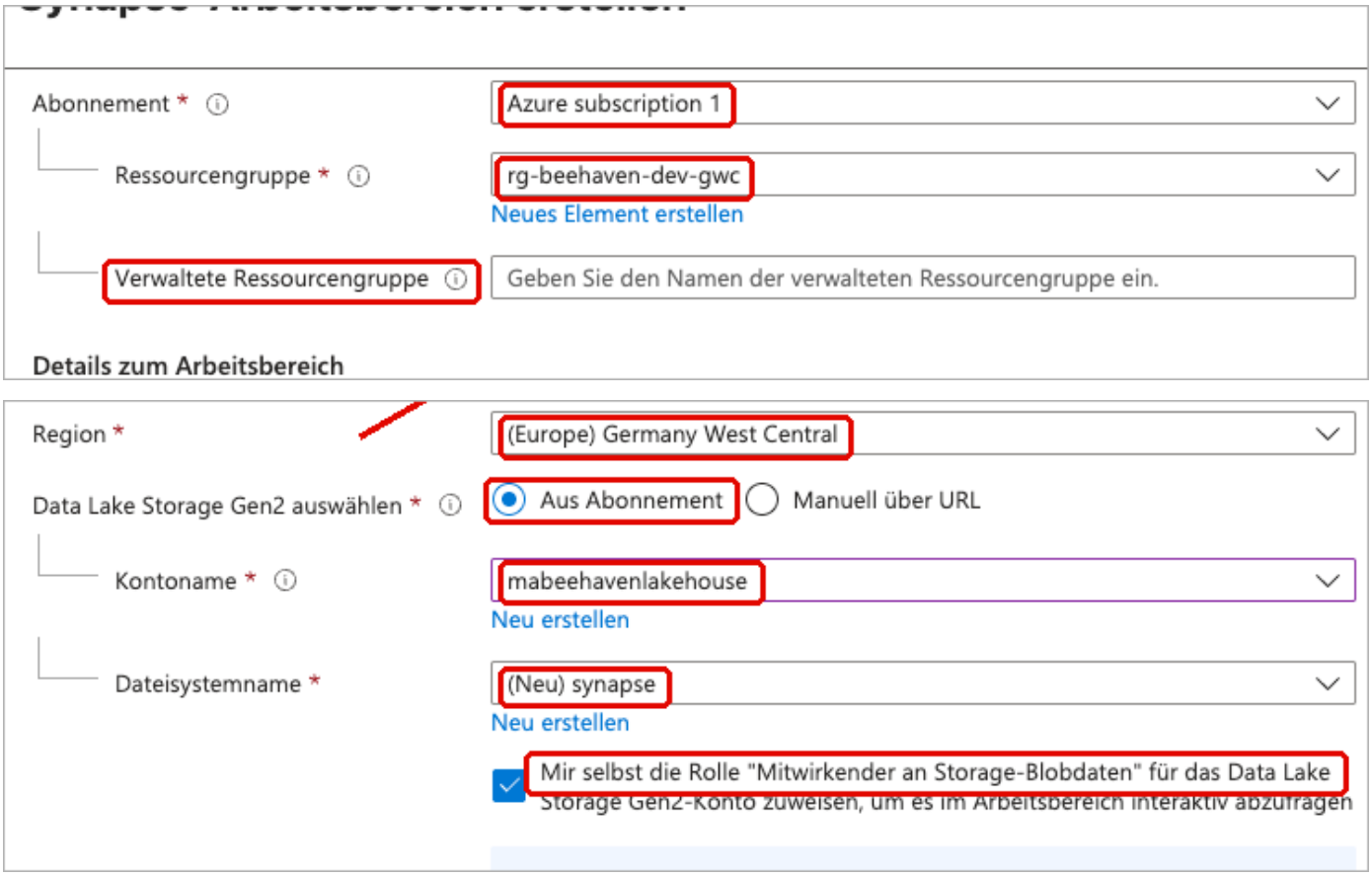
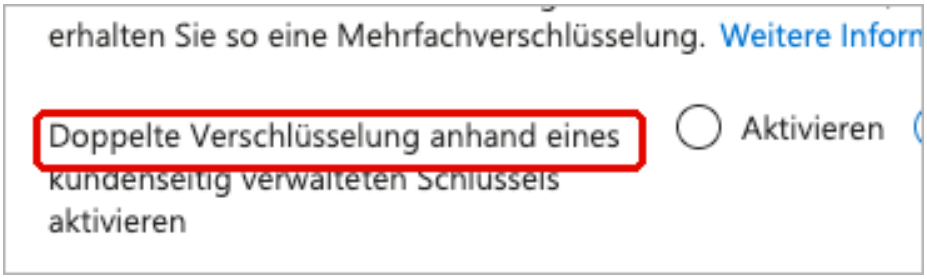


Abb. 94: Arbeitsbereich – Grundlagen: Name, Region, Data Lake mabeehavenlakehouse, neues Dateisystem synapse. (Ausschnitte 1–2)

3. Reiter **Sicherheit**: **Authentifizierungsmethode** = **Nur Microsoft Entra ID-Authentifizierung verwenden** – Anmeldung wie im Portal, kein zusätzliches SQL-Kennwort, das verloren gehen kann. **Netzwerkzugriff auf Data Lake Storage Gen2-Konto erlauben** bleibt leer (der Data Lake hat keine Netzwerkregeln), **Doppelte Verschlüsselung …** bleibt **Deaktivieren** (die Verschlüsselung durch Microsoft reicht; eigene Schlüssel wären zusätzlicher Verwaltungsaufwand). Reiter **Netzwerk** unverändert lassen.



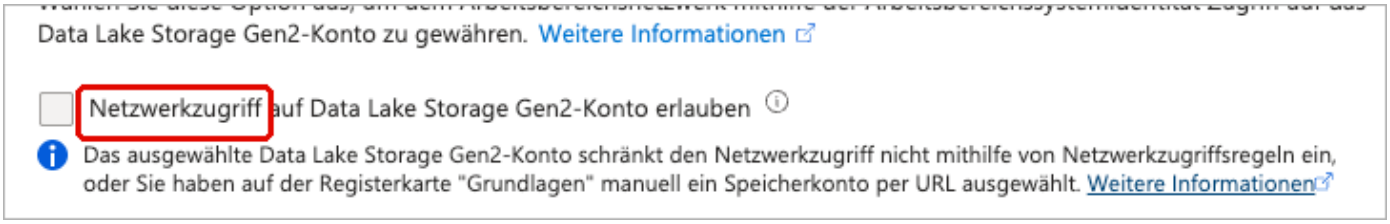


Abb. 95: Sicherheit: nur Microsoft Entra ID (keine SQL-Anmeldung mit Kennwort). (Ausschnitte 1–2)

4. Reiter **Tags**: **projekt** = `beehaven`, **umgebung** = `dev`, **verantwortlich** = `projektleitung`, **zweck** = `Notebooks Bereinigung und Gold (Spark)` – dieselben Tags wie bei allen Projektressourcen, damit die Kostenanalyse nach Projekt filtern kann. Im Bild steht **Umgebung** großgeschrieben; Azure unterscheidet bei Tag-Namen keine Groß-/Kleinschreibung, einheitlich klein ist trotzdem besser.

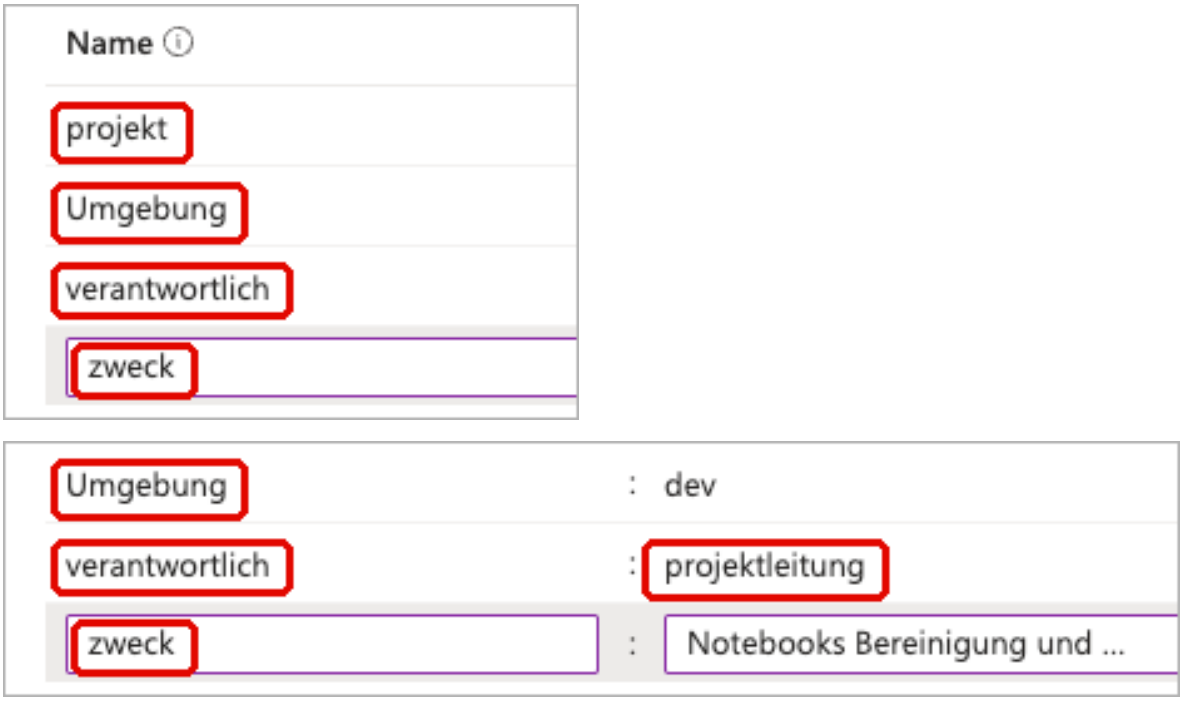


Abb. 96: Tags des Arbeitsbereichs: projekt = beehaven, umgebung = dev, verantwortlich = projektleitung, zweck = Notebooks Bereinigung und Gold (Spark). (Ausschnitte 1–2)

5. Reiter **Überprüfen und erstellen**: unter **Allgemeine Informationen Region** Germany West Central, **Arbeitsbereichsname** (Neu) syn-beehaven-dev-gwc, **Data Lake Storage Gen2-Konto** (Adresse von `mabeehavenlakehouse`, im Bild geschwärzt), **Data Lake Storage Gen2-Dateisystem** (Neu) synapse, **Verwaltete Ressourcengruppe** Keine und **Rollenzuweisungen** (Mitwirkender an Storage-Blobdaten für den Arbeitsbereich und für dich) prüfen → **Erstellen**. Die Bereitstellung dauert einige Minuten.



Abb. 97: Überprüfen und erstellen: Zusammenfassung prüfen (Name, Region, Data Lake, Dateisystem synapse) → Erstellen. (Ausschnitte 1–2)

Teil B – Spark-Pool anlegen (Arbeitsbereich → Übersicht)

1. Arbeitsbereich `syn-beehaven-dev-gwc` öffnen → **Übersicht**: **Status** zeigt **Succeeded**, unten stehen die Tags. Oben **+ Neuer Apache Spark-Pool** anklicken (**Aktualisieren** und **SQL-Administratorkennwort zurücksetzen** werden nicht gebraucht). Die Kachel **Synapse Studio öffnen** folgt in Schritt 29.

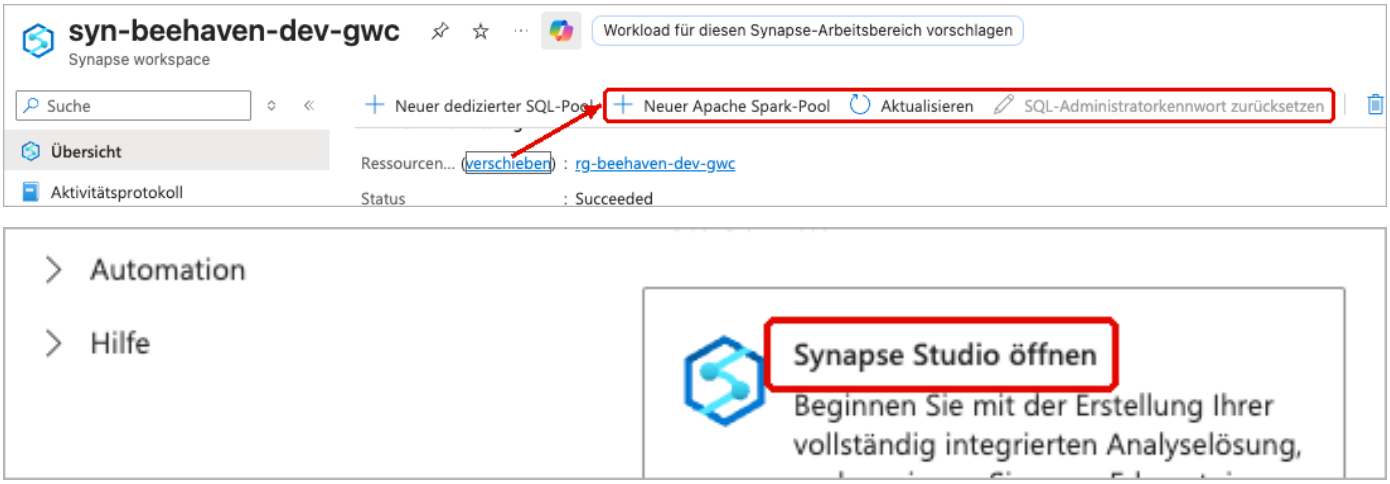


Abb. 98: Übersicht des Arbeitsbereichs: + Neuer Apache Spark-Pool; später Synapse Studio öffnen. (Ausschnitte 1–2)

2. Reiter **Grundlagen**: **Name des Apache Spark-Pools** `sparkbeehaven` (nur Buchstaben und Ziffern), **Knotengröße Small (4 virtuelle Kerne/32 GB)** – die kleinste Größe reicht für einige Millionen Zeilen; **Familie für Knotengröße** (MemoryOptimized) ist fest. **Autoskalierung Deaktiviert** und **Anzahl der Knoten** `3` (Minimum) – feste Größe, planbare Kosten. **Executors dynamisch zuordnen** bleibt **Deaktiviert**. Der gelbe Hinweis zu den Rollenzuweisungen ist durch den Haken in Teil A schon erledigt (siehe Zusammenfassung). Kontrolle: **Geschätzter Preis 1.80 USD** pro Stunde – nur solange der Pool läuft (Kurszusammenfassung Kap. 9: Synapse-Notebooks sind teuer, deshalb klein und kurz).

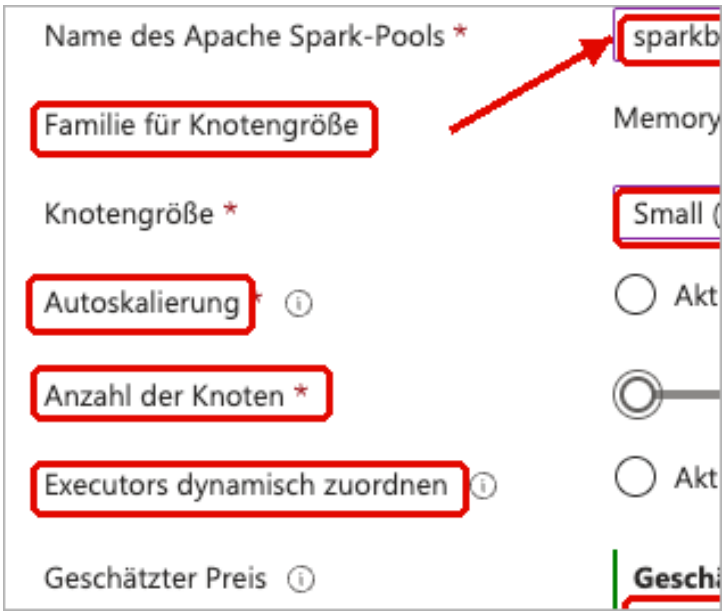


Abb. 99: Spark-Pool – Grundlagen: Name, Knotengröße Small, Autoskalierung aus, 3 Knoten; geschätzt 1,80 USD je Stunde.

3. Reiter **Zusätzliche Einstellungen**: **Automatisches Anhalten Aktiviert** und **Leerlaufzeit in Minuten** von 15 auf `5` senken – der Pool hält 5 Minuten nach der letzten Arbeit an und kostet dann nichts mehr. **Apache Spark-Pool** (Version) **3.5** mit Python 3.11 (neueste angebotene Version). **Apache Spark-Konfiguration** bleibt **Standardkonfiguration verwenden**.

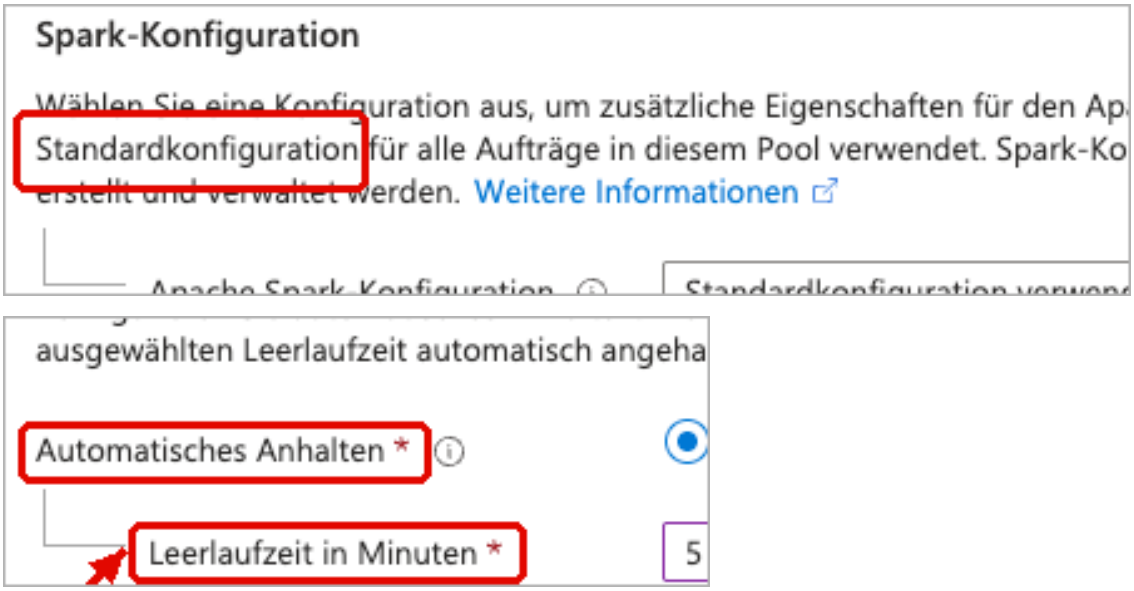


Abb. 100: Zusätzliche Einstellungen: automatisch anhalten nach 5 Minuten Leerlauf, Spark 3.5 (Python 3.11). (Ausschnitte 1–2)

4. Reiter **Tags**: **projekt** = `beehaven`, **umgebung** = `dev`, **verantwortlich** = `projektleitung`, **zweck** = `Spark-Pool für Notebooks`.



Abb. 101: Tags des Spark-Pools: projekt = beehaven, umgebung = dev, verantwortlich = projektleitung, zweck = Spark-Pool für Notebooks. (Ausschnitte 1–2)

5. Reiter **Überprüfen und erstellen** → „Die Validierung war erfolgreich“ → **Erstellen**. Nach einigen Minuten meldet die Bereitstellung **„Ihre Bereitstellung wurde abgeschlossen.“**

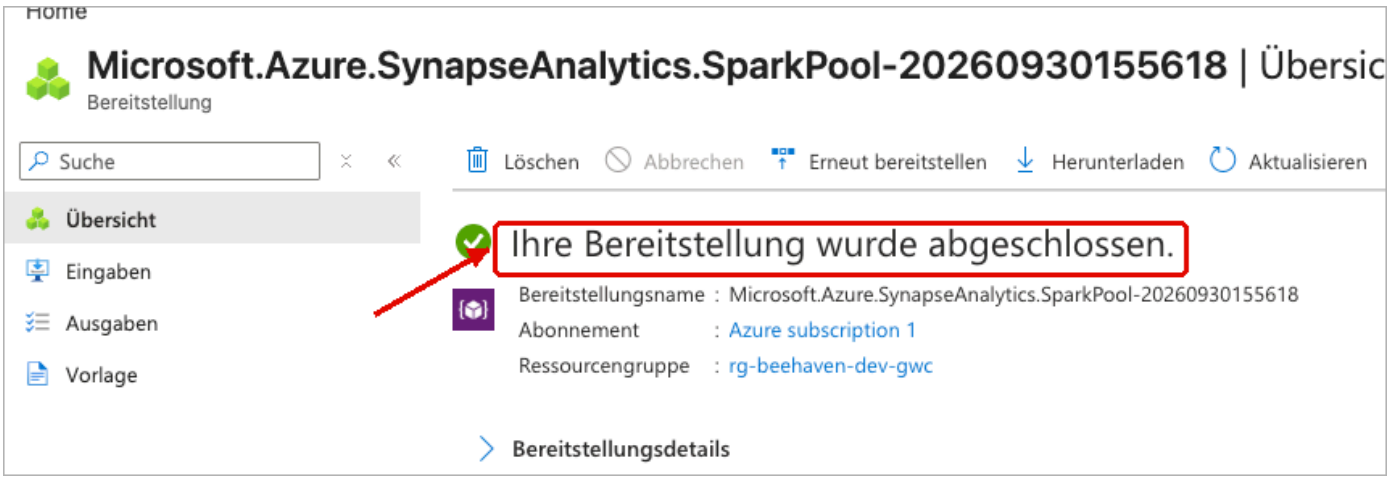


Abb. 102: Bereitstellung abgeschlossen – Spark-Pool angelegt.

> ✅ **Prüfen:** Übersicht des Arbeitsbereichs: Status **Succeeded**, unter **Analysepools** steht `sparkbeehaven` (Small). Preis laut Portal: 1,80 USD je Stunde, nur während der Pool läuft.

> ⚠️ **Achtung: Nur eine Sitzung gleichzeitig:** Das Kontingent des Arbeitsbereichs (12 vCores) reicht für genau eine Sitzung mit 3 Knoten. Jede Notebook-Registerkarte startet ihre eigene Sitzung – vor dem nächsten Notebook die Sitzung im anderen Tab **beenden**, sonst: `AVAILABLE_WORKSPACE_CAPACITY_EXCEEDED`.

Schritt 29 – Notebooks in Synapse importieren und testen

> **Warum?** Die Synapse-Kopien der Notebooks (Ordner `03_notebooks_ITS-Data/synapse/`, vom Generator erzeugt) werden importiert und einmal von Hand getestet – erst dann kommen sie in die Pipeline.

Teil A – importieren (Synapse Studio)

1. Arbeitsbereich → Kachel **Synapse Studio öffnen** → **Öffnen**. Cookie-Hinweis: **Ablehnen**.
2. Links **Entwickeln (Develop)** → **+** → im Menü ganz unten **Importieren** (nicht **Notizbuch** – das legt ein leeres an) → die 5 Dateien `00_steuerung_silver` … `04_gold_synapse` markieren (nicht `_archiv`) → **Öffnen**.

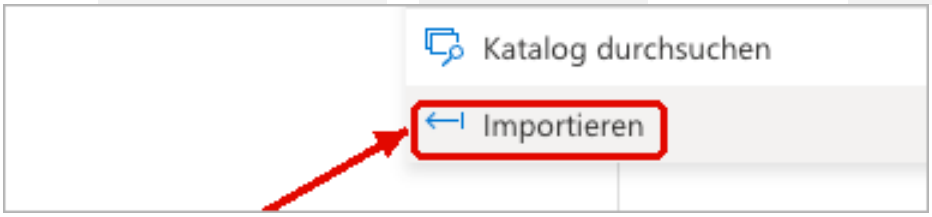


Abb. 103: Synapse Studio → Entwickeln → + → Importieren.

3. Oben stehen jetzt die Registerkarten der importierten Notebooks. In jeder Registerkarte **Anfügen an (Attach to)** = `sparkbeehaven` (auf diesem Pool läuft das Notebook), Sprache **PySpark (Python)**.

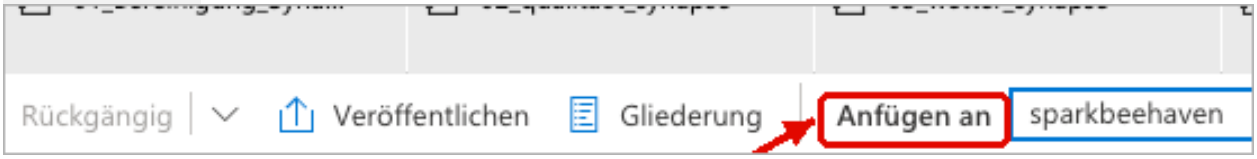


Abb. 104: Jedes Notebook an den Spark-Pool anfügen: Anfügen an = sparkbeehaven, Sprache PySpark (Python).

4. **Alle veröffentlichen (Publish all)**: Unter **Ausstehende Änderungen (5)** stehen die 5 Notebooks `00_steuerung_silver` … `04_gold_synapse` → **Veröffentlichen**. Erst veröffentlichte Notebooks kann die Data Factory später starten.



Abb. 105: Alle veröffentlichen: die 5 Notebooks werden in die Liveumgebung übernommen.

> ✅ **Prüfen:** Unter **Notebooks** stehen 5 Einträge, an **Alle veröffentlichen** steht keine Zahl mehr.

> 💡 **Gut zu wissen:** Der Import zeigt ein Notebook eventuell mit Großbuchstaben an (z. B. `01_Bereinigung_Synapse`). Maßgeblich ist das Feld **Name** unter **Eigenschaften** – Synapse unterscheidet Groß-/Kleinschreibung beim Namen nicht.

Teil B – testen: erst Silver, dann Gold

1. Registerkarte `00_steuerung_silver` → **Alle ausführen**. Erster Start einer Sitzung: 3–4 Minuten.
2. Unter der Zelle mit `runMultiple` erscheinen die Links **Notebookausführung anzeigen** zu N1, N2, N3. Ergebnis abwarten: `'exception': None` je Notebook heißt „kein Fehler“, die letzte Zeile meldet „Stufe Silver **fertig** in … s“ → **Sitzung beenden** (Symbol oben rechts neben dem Zahnrad), damit der Pool nicht weiter kostet.

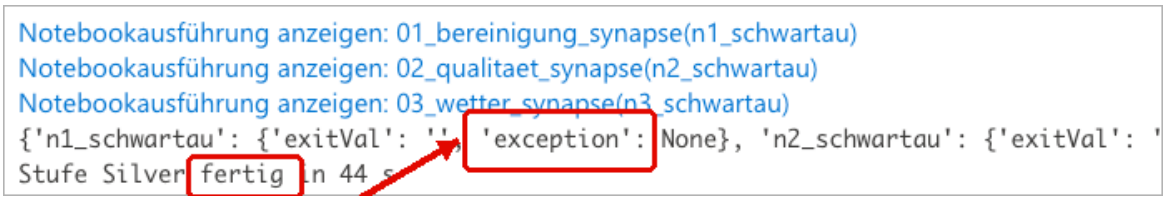


Abb. 106: Steuer-Notebook: Links zu N1, N2, N3, alle ohne Ausnahme (exception: None) – Stufe Silver fertig in 44 s.

3. Registerkarte `04_gold_synapse` → **Alle ausführen**. Am Ende listet das Notebook die Gold-Dateien mit Größe: **stunden_schwartau.csv**, `stunden_schwartau.parquet` (976 KB), `tageswerte_schwartau.csv`, `tageswerte_schwartau.parquet` (48 KB) → danach **Sitzung beenden**.

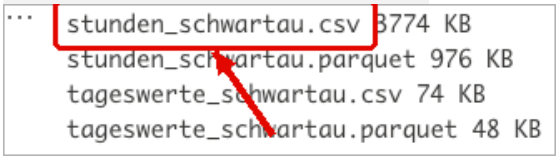


Abb. 107: N4 Gold: Stunden- und Tagestabelle als Parquet und CSV in gold/schwartau – Größen wie lokal.

4. Gegenprobe: **Daten → Verknüpft → lakehouse → gold → schwartau**.

> ✅ **Prüfen:** Steuer-Notebook: `{'n1_schwartau': {'exitVal': '', 'exception': None}, …}` und „Stufe Silver fertig in ca. 45 s“. N4: `stunden_schwartau.parquet` ca. 976 KB, `tageswerte_schwartau.parquet` ca. 48 KB – wie lokal.

Typische Fehler beim ersten Test

Meldung	Ursache	Lösung
N4: `IndexError` bei `versionen[-1]`	`silver/cleaned` ist in Azure noch leer	erst `00_steuerung_silver`, dann N4
`ArrowNotImplementedError: … without Azure Blob File System support`	pandas versucht bei `storage_options=None` pyarrow direkt mit `abfss://`	in Azure `storage_options={}` (Generator ab 30.09.2026)
`LIVY_JOB_TIMED_OUT`	Sitzung nach Leerlauf abgelaufen	noch einmal **Alle ausführen**
`AVAILABLE_WORKSPACE_CAPACITY_EXCEEDED`	andere Sitzung belegt alle 12 vCores	Sitzung im anderen Tab beenden
Steuer-Notebook: `RunMultipleFailedException`	ein Teil-Notebook ist gescheitert	Link **Notebookausführung anzeigen** → erste rote Zelle → letzte Zeile der Meldung

> ⚠️ **Achtung: Code in Synapse ändern:** Zeile markieren mit Klick vor das erste Zeichen und ⇧-Klick hinter das letzte – nicht dreimal klicken, sonst geht der Zeilenumbruch verloren. Besser: lokal im Generator ändern und neu importieren.

> 💡 **Gut zu wissen: Einfach erklärt:** Wie ein Umzug in eine neue Küche: Die Rezepte bleiben gleich, aber ein Gerät passte nicht in die neue Steckdose – ein Adapter (eine Zeile Code) löst es. Und es passt immer nur ein Koch gleichzeitig hinein.

Schritt 30 – Data Factory an Synapse anbinden (verknüpfter Dienst, Rollen, Notebook-Aktivität)

> **Warum?** Bisher liefen die Notebooks in Synapse nur von Hand (Schritt 29). Jetzt soll die **Data Factory** sie starten – als Aktivität in `pl_bronze_to_silver`. Dafür braucht sie eine Verbindung zum Synapse-Arbeitsbereich und die passenden Rechte.

> 💡 **Gut zu wissen: Hintergrund:** Die Data Factory meldet sich mit ihrer **verwalteten Identität** an – ohne Passwort oder Schlüssel. Welche Rechte sie in Synapse hat, regelt die **Synapse-Zugriffssteuerung** (eigene Rollen, getrennt von den Azure-Rollen im Portal).

Teil A – Notebook-Aktivität und verknüpften Dienst anlegen (ADF Studio → Erstellen → pl_bronze_to_silver)

1. Links **Aktivitäten** → **Synapse** → **Notizbuch** auf die Arbeitsfläche ziehen. Reiter **Allgemein** → Name `silver_automatisierung`, Beschreibung: „Startet das Synapse-Notebook 00_steuerung_silver: je Stadt N1 Bereinigung, N2 Qualität, N3 Wetter in einer Spark-Sitzung. Ergebnis: silver/cleaned (Parquet), Wetter-Rohdaten in bronze/archive."
2. Unten neben **Allgemein** den Reiter **Azure Synapse Analytics (Artefakte)** öffnen → **+ Neu** (der Reiter trägt eine rote **1**, solange die Verbindung fehlt). Die Liste ist anfangs leer – das ist richtig.

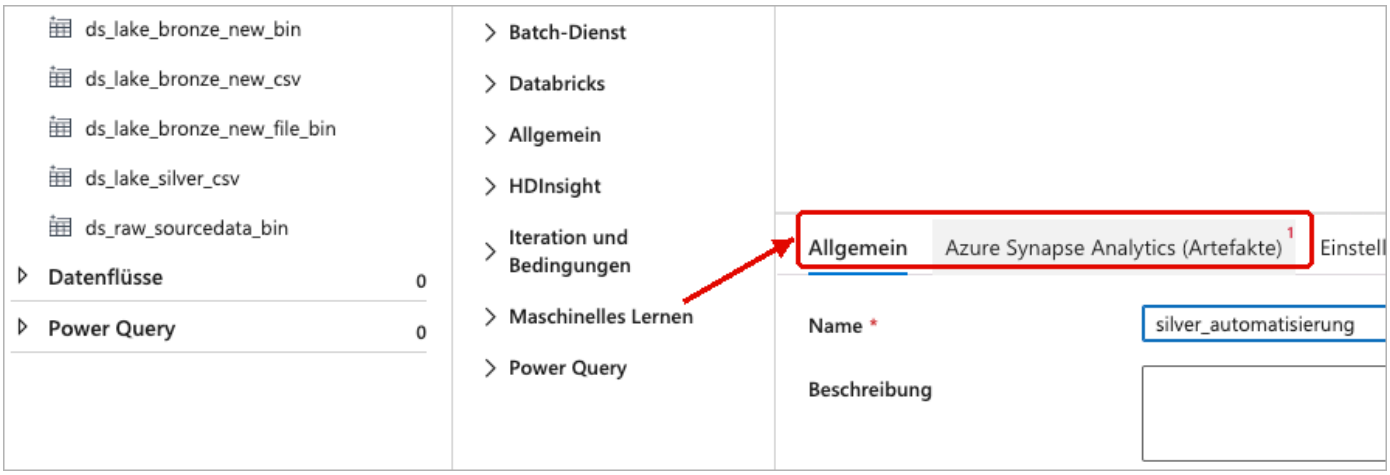
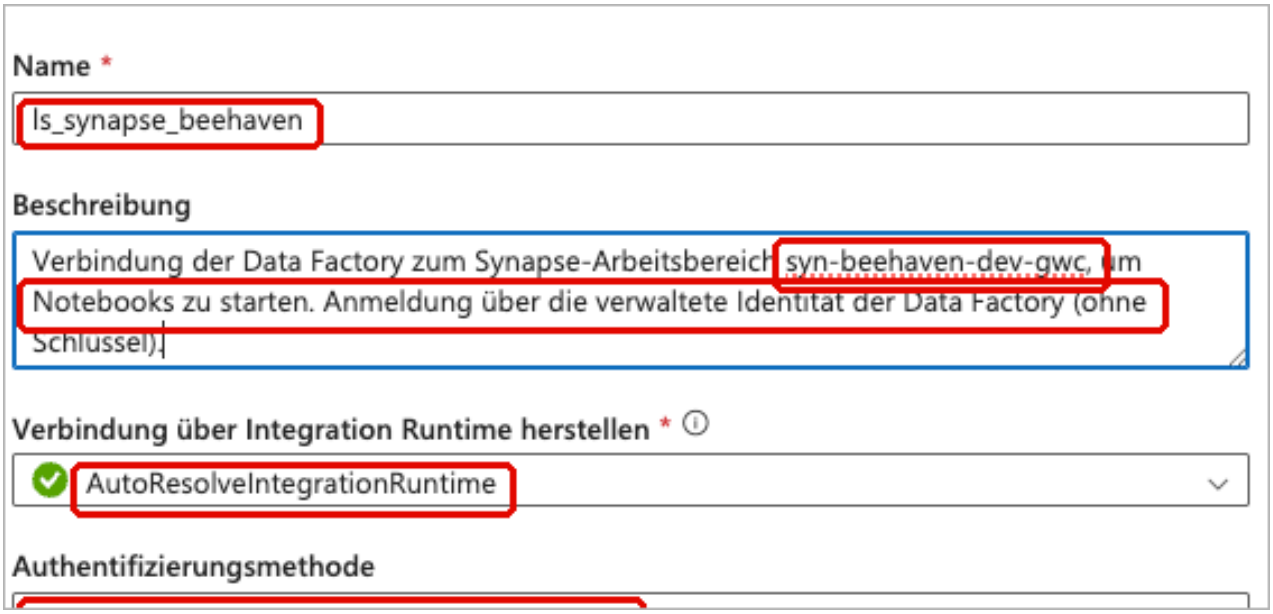


Abb. 108: Aktivität Notizbuch aus Synapse in pl_bronze_to_silver – Reiter „Azure Synapse Analytics (Artefakte)" noch ohne verknüpften Dienst.

3. Felder: Name `ls_synapse_beehaven` · Beschreibung „Verbindung der Data Factory zum Synapse-Arbeitsbereich `syn-beehaven-dev-gwc`, um Notebooks zu starten. Anmeldung über die verwaltete Identität der Data Factory (ohne Schlüssel)." · Integration Runtime `AutoResolveIntegrationRuntime` · Authentifizierung **Systemseitig zugewiesene verwaltete Identität** · Auswahl **Aus Azure-Abonnement** · Arbeitsbereich `syn-beehaven-dev-gwc`. Unten den **Namen der verwalteten Identität** ablesen (`adf-beehaven-dev-gwc`) – genau diesem Namen gibt Teil B die Rechte. **Anmerkungen**, **Parameter** und **Erweitert** bleiben leer → **Erstellen**.



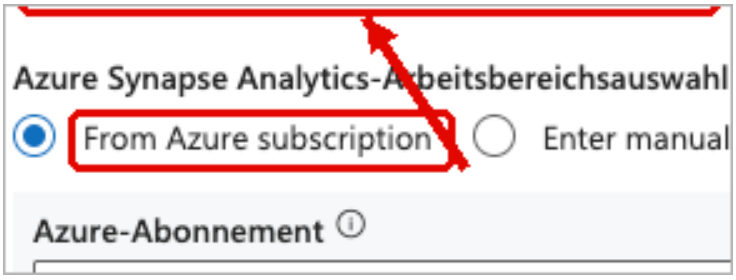


Abb. 109: Neuer verknüpfter Dienst ls_synapse_beehaven: verwaltete Identität der Data Factory, Arbeitsbereich syn-beehaven-dev-gwc. (Ausschnitte 1–2)

> ⚠ **Achtung: Noch nicht „Verbindung testen“** – ohne Rollen schlägt der Test fehl. Erst Teil B.

Teil B – Rollen für die Data Factory (Synapse Studio → Verwalten → Zugriffssteuerung)

1. **+ Hinzufügen** → Bereich **Arbeitsbereich** → Rolle `Synapse Compute Operator` (darf Spark-Sitzungen starten, aber keinen Code ändern) → **Benutzer auswählen** `adf-beehaven-dev-gwc` – steht danach unter **Ausgewählte Benutzer** → **Anwenden**.



Abb. 110: Synapse Studio → Verwalten → Zugriffssteuerung → Hinzufügen: Rolle für die Data Factory.

2. Wiederholen mit `Synapse Artifact User` und `Synapse Credential User`. Die Rollenliste zeigt nur 8 Einträge – für „Credential User“ ins Feld **Filtern** `Credential` tippen.
3. **Aktualisieren** → `adf-beehaven-dev-gwc` steht dreimal in der Liste: **Synapse Artifact User** (darf Notebooks lesen), **Synapse Compute Operator** (darf sie starten), **Synapse Credential User** (darf die Anmeldung des Arbeitsbereichs nutzen). Die Einträge **Synapse Administrator** für dich und den Arbeitsbereich `syn-beehaven-dev-gwc` hat Azure beim Erstellen selbst angelegt.

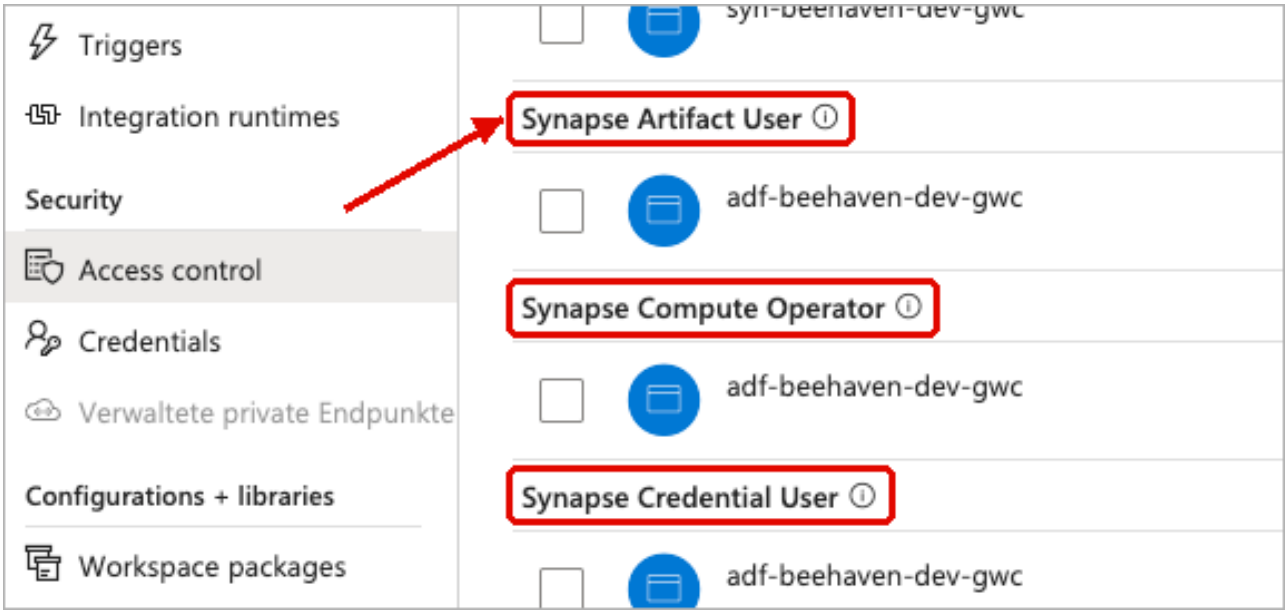




Abb. 111: Ergebnis: adf-beehaven-dev-gwc mit genau drei Rollen (geringste Rechte). (Ausschnitte 1–2)

Rolle	erlaubt der Data Factory …	ohne sie …
Synapse Compute Operator	eine Spark-Sitzung auf dem Pool zu starten	startet kein Notebook
Synapse Artifact User	Notebooks zu lesen und auszuführen (nicht zu ändern)	findet das Notebook nicht
Synapse Credential User	die Identität des Arbeitsbereichs zu nutzen (Zugriff des Notebooks auf den Data Lake)	darf das Notebook nicht lesen/ schreiben

> 💡 **Gut zu wissen: Warum nicht „Synapse Administrator“ (Kursvorgabe)?** Prinzip der geringsten Rechte: Ein Automatik-Dienst bekommt nur, was er braucht. Wird die Data Factory einmal falsch eingestellt, kann sie in Synapse nichts löschen oder umbauen.

Teil C – Notebook einstellen und Verbindung testen (zurück im ADF Studio)

1. Reiter **Azure Synapse Analytics (Artefakte)** → **Verbindung testen** → „Verbindung erfolgreich“. Neue Rollen wirken nach einigen Minuten.



Abb. 112: Verbindung testen: erfolgreich (prüft nur die Erreichbarkeit, nicht die Rechte).

2. Reiter **Einstellungen**: **Notizbuch** `00_steuerung_silver`, **Spark-Pool** `sparkbeehaven`. Sind die Listen leer (zwischengespeichert aus der Zeit vor den Rollen): **Dynamischen Inhalt hinzufügen** und den Namen eintragen. Basisparameter leer lassen (`STAEDTE = ""` = alle Städte). **Executorgröße**, **Executors dynamisch zuordnen**, **Min./Max. Executors** und **Treibergröße** bleiben leer – dann gelten die Einstellungen des Pools (Small, 3 Knoten).

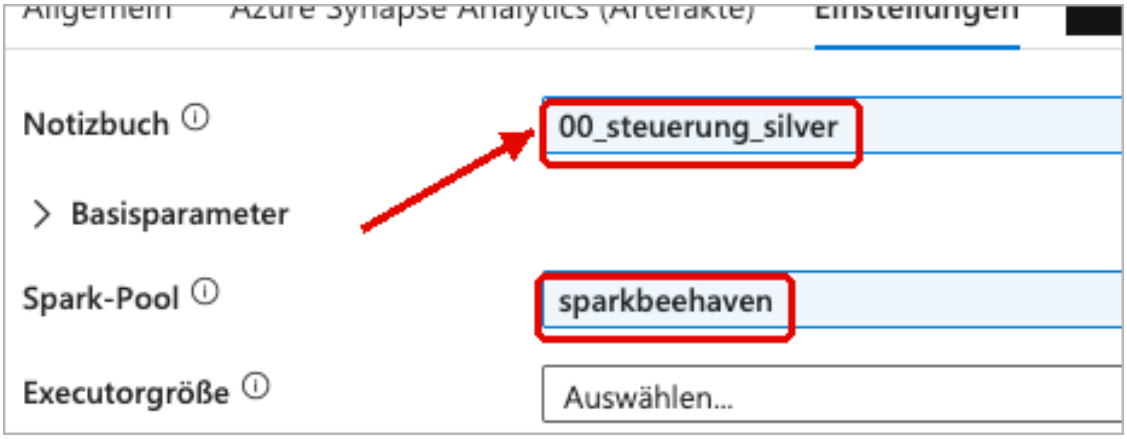


Abb. 113: Einstellungen: Notebook und Spark-Pool als dynamischer Inhalt (Liste war anfangs leer).

3. Reiter **Allgemein**: **Timeout** `0.00:30:00`, **Wiederholen** `1`, **Wiederholungsintervall (Sek.)** `360`; **Sichere Ausgabe** bleibt leer. **Aktivitätsstatus** bleibt hier **Aktiviert** – im Bild ist die Aktivität gerade deaktiviert und als **Erfolgreich** markiert, weil das Bild aus dem kostenlosen Test in Schritt 31 stammt.

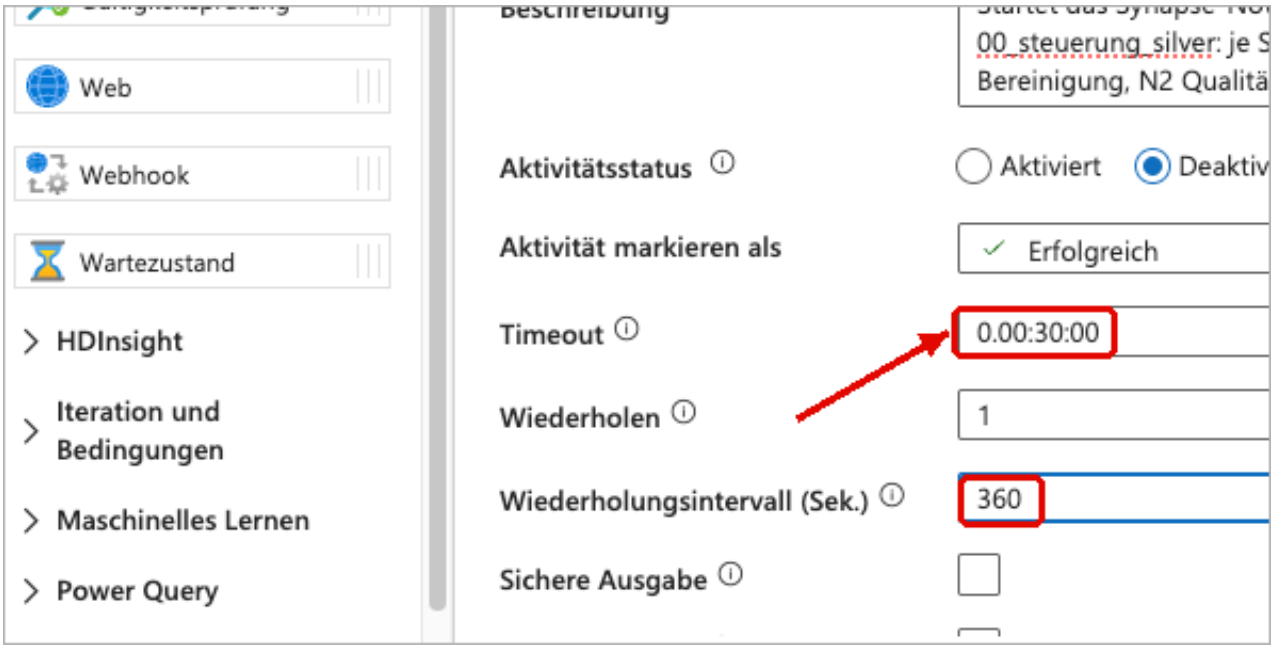


Abb. 114: Allgemein: Timeout 0.00:30:00, Wiederholen 1, Intervall 360 s.

4. Grünen Pfeil (bei Erfolg) von **ForEach** auf das Notizbuch ziehen → **Überprüfen**.

> 💡 **Gut zu wissen: Timeout und Wiederholen:** Standard sind 12 Stunden – ein hängendes Notebook würde so lange kosten. 30 Minuten reichen (Sitzungsstart 3–4 Min. + ca. 1 Min. je Stadt). Ein zweiter Versuch nach 6 Minuten fängt kurze Störungen ab (Kapazität belegt, Sitzung abgelaufen). **Mindestwert für Timeout: 10 Minuten.**

> ⚠️ **Achtung: In der Data Factory ohne Git nie „Aktualisieren" (↻ oben rechts), solange „Alle veröffentlichen" eine Zahl zeigt** – alle unveröffentlichten Änderungen gehen verloren. „Alle veröffentlichen" scheitert, solange die Prüfung Fehler meldet (z. B. Notebook fehlt). Synapse sieht nur **veröffentlichte** Notebooks.

Teil D – Testlauf (Debuggen) mit Gegenprobe

Ausgangslage prüfen	Erwartung
`bronze/new` leer, `silver/processing/schwartau/<sensor>/` gefüllt	ForEach läuft 0-mal (erfolgreich), das Notebook verarbeitet `processing`

1. **Debuggen** → unten **Ausgabe** → mit dem Kreispfeil neben der Ausführungs-ID aktualisieren, bis `silver_automatisierung` erfolgreich ist (Spalte **Dauer**, im Bild 4m 24s). **Pipelinestatus** zeigt dabei oft noch kurz **In Bearbeitung** – noch einmal aktualisieren.

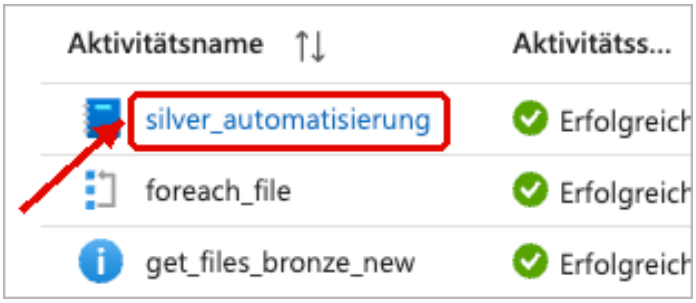




Abb. 115: Debug-Lauf: alle Aktivitäten erfolgreich, Notebook 4 Min. 24 s (inkl. Start der Spark-Sitzung). (Ausschnitte 1–2)

1. Gegenprobe im Speicherbrowser: `lakehouse` → `silver` → `cleaned` → `schwartau` → `flow` → **Aktualisieren**: neue Datei **flow_schwartau_cleaned__2026-10-01_095241_UTC.parquet** mit dem UTC-Stempel des Laufs, der Vorgänger liegt im Ordner **_archiv** (jede Version bleibt nachvollziehbar).

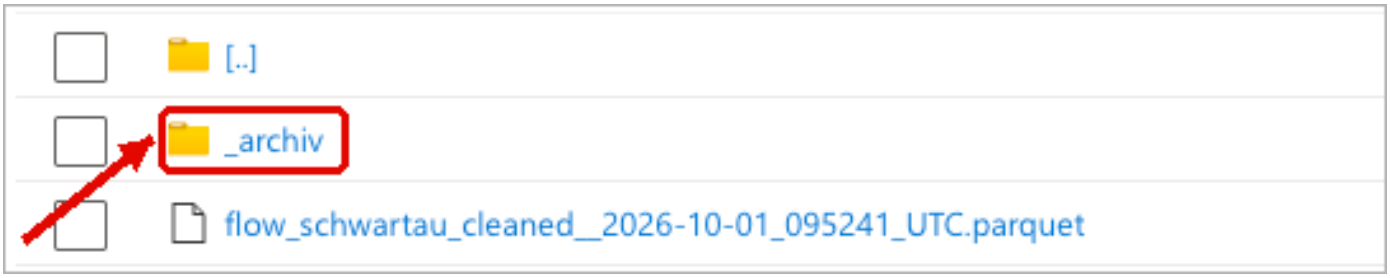


Abb. 116: Gegenprobe silver/cleaned/schwartau/flow: neue Datei, Vorgänger im _archiv, gleiche Größe.

> ✅ **Prüfen:** Alle Aktivitäten erfolgreich, Notebook ca. 4–5 Min. Die neue Datei in `silver/cleaned/schwartau/flow/` ist ca. 15,5 MiB groß (Spalte **Größe** rechts), Vorgänger im `_archiv`.

> 💡 **Gut zu wissen:** **Ausgabe des Notebooks lesen** (Symbol ↗ bei der Aktivität): `runStatus: Succeeded` zählt. `state: killed` ist **kein Fehler** – die Data Factory beendet die Sitzung nach dem Lauf, damit keine Kosten weiterlaufen. `exitValue: null` heißt: Das Notebook gibt keine Zusammenfassung zurück (Verbesserung vorgemerkt). Kosten des Laufs: ca. 0,12–0,15 €.
>
> 💡 **Gut zu wissen:** **Einfach erklärt:** Die Data Factory ist der Dirigent, Synapse das Orchester. Der verknüpfte Dienst ist die Telefonleitung, die Rollen sind die Erlaubnis, das Orchester spielen zu lassen – aber nicht, die Noten umzuschreiben.

Schritt 31 – Erfolgspfad: Arbeitskorb nach erfolgreichem Lauf leeren

> **Warum?** Kursaufgabe „Erfolg und Fehler behandeln“: Nach einem **erfolgreichen** Silver-Lauf werden die Arbeitskopien in `silver/processing` gelöscht, damit der nächste Lauf leer beginnt. Bei einem **Fehler** bleiben sie zur Prüfung liegen – dafür muss nichts gebaut werden, die Pipeline zeigt rot.

Teil A – Löschen-Aktivität anlegen

1. **Aktivitäten** → **Allgemein** → **Löschen** auf die Fläche ziehen. Name `processing_leeren`, Beschreibung: „Leert nach erfolgreichem Silver-Lauf den Arbeitskorb silver/processing (Arbeitskopien). Originale liegen mit Zeitstempel in bronze/archive.“ Timeout `0.00:10:00`, Wiederholen `1`, Intervall `60`.
2. **Grünen Pfeil** von `silver_automatisierung` auf `processing_leeren` – nur bei Erfolg.
3. Reiter **Quelle**: Dataset `ds_lake_silver_csv` · `folder_name` = `processing` (**ohne** `silver/` – das setzt das Dataset selbst davor) · `file_name` = `*.csv` · Dateipfadtyp **Platzhalterdateipfad** · **Wildcard file name** `*.csv` → **Tab** drücken (sonst wird der Wert nicht gespeichert) · **Nach letzter Änderung filtern** leer · **Recursively (Rekursiv)** ✓.

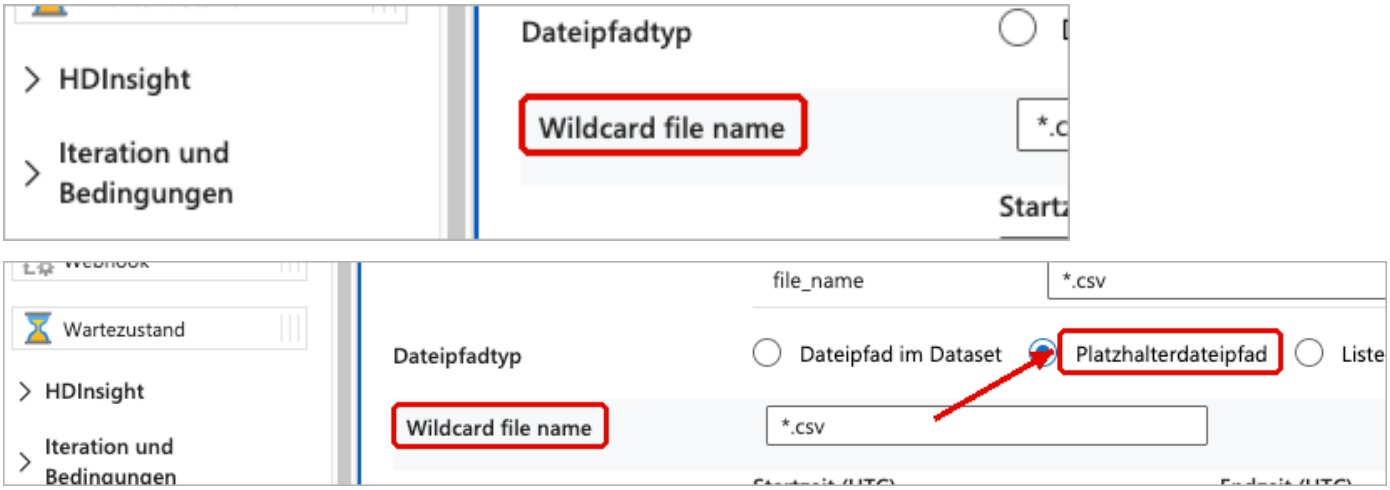


*Abb. 117: Löschen-Aktivität processing_leeren, Reiter Quelle: folder_name processing, Platzhalter *.csv, rekursiv. (Ausschnitte 1–2)*

4. Reiter **Protokollierungseinstellungen**: Protokollierung aus.

> 💡 **Gut zu wissen: Platzhalter statt Dateiname:** `*.csv` trifft jede CSV-Datei. Dynamische Werte (z. B. `@item().name`) braucht man nur, wenn sich etwas je Lauf ändert. **Rekursiv** erfasst alle Unterordner (Stadt, Sensor).

Teil B – kostenlos testen (Notebook überspringen)

1. `silver_automatisierung` → **Allgemein** → **Aktivitätsstatus: Deaktiviert** → **Aktivität markieren als: Erfolgreich**. Die Karte zeigt ein Deaktiviert-Symbol.
2. **Debuggen** → Ausgabe: Notebook „Inactive“, `processing_leeren` erfolgreich (wenige Sekunden).



Abb. 118: Kostenloser Test: Notebook deaktiviert (Inactive), Löschen erfolgreich.

3. Gegenprobe: `silver/processing/schwartau/<sensor>/` → **Aktualisieren** → „Alle 0 Elemente werden angezeigt“: keine CSV mehr, der Arbeitskorb ist leer.

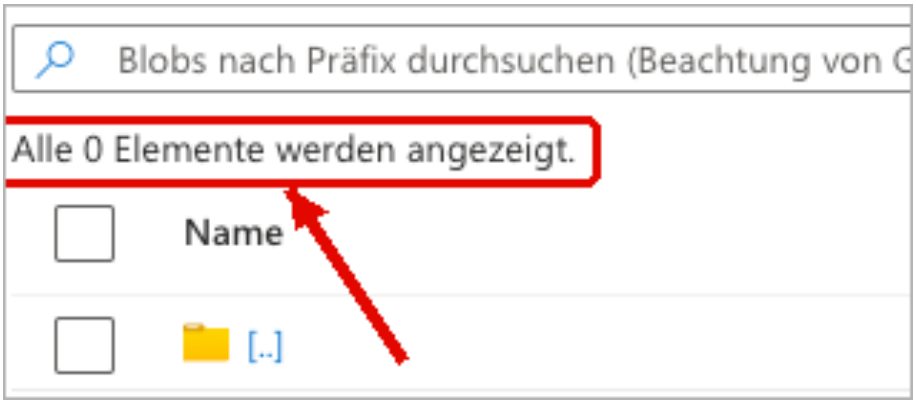


Abb. 119: Gegenprobe: silver/processing/schwartau/flow ist leer.

4. Notebook wieder **Aktiviert** → **Überprüfen**.
5. **Alle veröffentlichen**: Unter **Ausstehende Änderungen (2)** stehen die Pipeline **pl_bronze_to_silver** (**Bearbeitet**) und der verknüpfte Dienst **ls_synapse_beehaven** (**Neu**, aus Schritt 30) → **Veröffentlichen**. Erst danach nutzt ein Trigger die neue Fassung.



Abb. 120: Alle veröffentlichen: geänderte Pipeline und neuer verknüpfter Dienst.

⚠ **Achtung: „Erfolgreich" heißt nicht „richtig":** Findet die Löschen-Aktivität nichts (falscher Ordner, Platzhalter nicht gespeichert), meldet sie trotzdem Erfolg. Deshalb immer die Gegenprobe – und im Zweifel mit `</>` (Code anzeigen) prüfen, ob `"wildcardFileName": "*.csv"` wirklich gespeichert ist. „Executors dynamisch zuordnen: Deaktiviert" im Notebook ist eine andere Einstellung als der Aktivitätsstatus.

💡 **Gut zu wissen: Einfach erklärt:** Der Koch räumt die Arbeitsfläche erst ab, wenn das Gericht fertig ist. Geht etwas schief, bleibt alles liegen, damit man sieht, wo es hakt.

Schritt 32 – Gold-Pipeline pl_silver_to_gold (je Stadt aus der Standort-Tabelle)

Warum? Die Gold-Stufe soll **für jede Stadt** laufen, die in `konfiguration/standorte.csv` steht – neue Stadt = neue Zeile, ohne Pipeline-Umbau. Die Data Factory liest dazu die Tabelle und startet das Gold-Notebook je Stadt.

Teil A – Dataset für die Standort-Tabelle

1. **Datensätze → … → Neues Dataset → Azure Data Lake Storage Gen2 → DelimitedText** → Name `ds_lake_konfiguration_standorte_csv`, verknüpfter Dienst `ls_adls_mabeehavenlakehouse`, Pfad `lakehouse/konfiguration/standorte.csv`, **Erste Zeile als Überschrift ✓**, Schema **Aus Verbindung/Speicher → OK → Datenvorschau**.

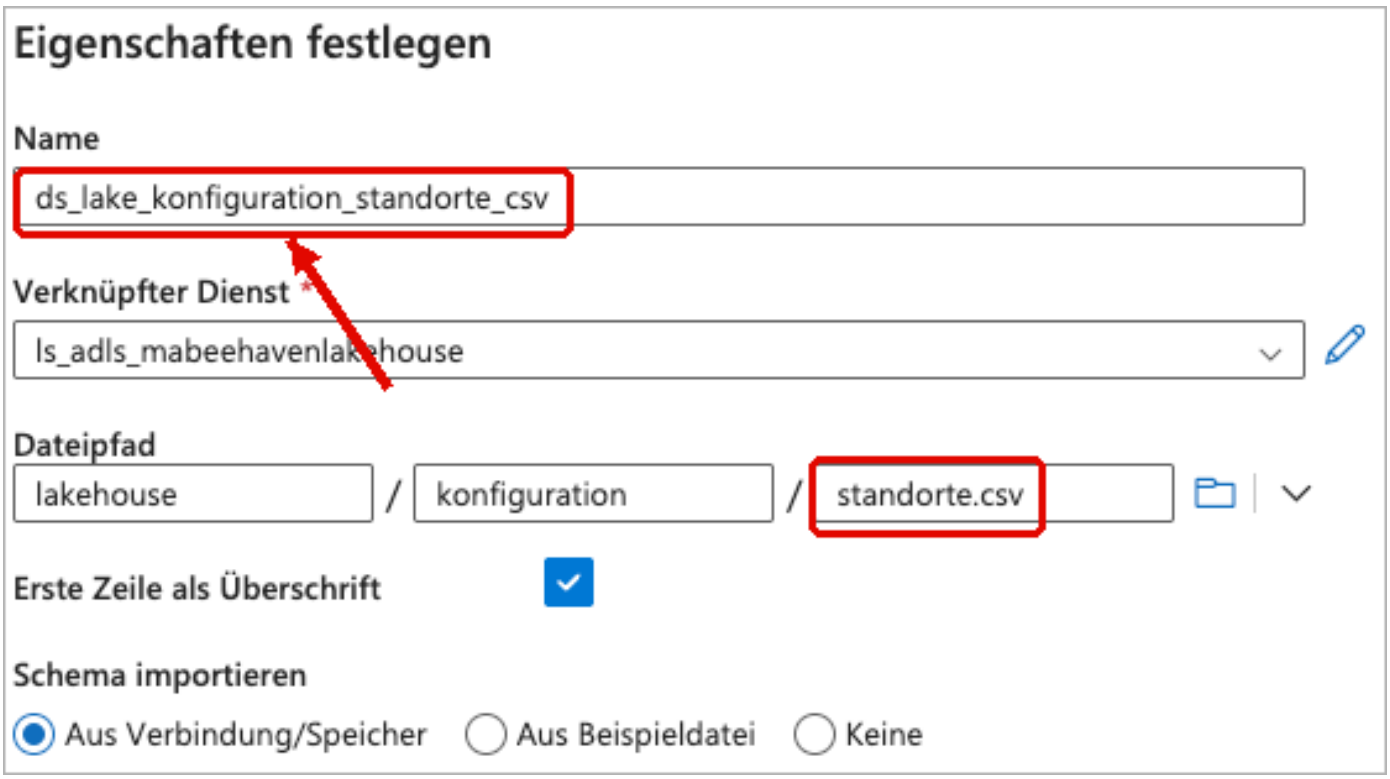


Abb. 121: Dataset für die Standort-Tabelle (ADLS Gen2, DelimitedText).

⚠ **Achtung: Browser-Übersetzung:** Übersetzt der Browser die Seite, zeigt die Vorschau z. B. „Stadt" und „Schwarzau" statt `stadt` und `schwartau`. Werte und Ausdrücke immer aus der Anleitung kopieren, nie vom Bildschirm abschreiben.

Teil B – Pipeline mit Suche, Schleife und Notebook

1. **Pipelines** → **…** → **Neue Pipeline** → Name `pl_silver_to_gold`, Beschreibung: „Stufe Silver → Gold: liest die Standort-Tabelle und startet je Stadt das Synapse-Notebook 04_gold_synapse.“ (Der neue Name erscheint links erst nach Veröffentlichen + Aktualisieren.)
2. **Allgemein** → **Suche** → Name `standorte_lesen`, Timeout `0.00:10:00` · **Einstellungen**: Dataset `ds_lake_konfiguration_standorte_csv`, **Nur erste Zeile** ohne Haken – sonst käme nur die erste Stadt zurück. **Dateipfadtyp** (Dateipfad im Dataset), **Recursively** und die übrigen Felder bleiben, wie sie sind (der Pfad zeigt auf genau eine Datei).

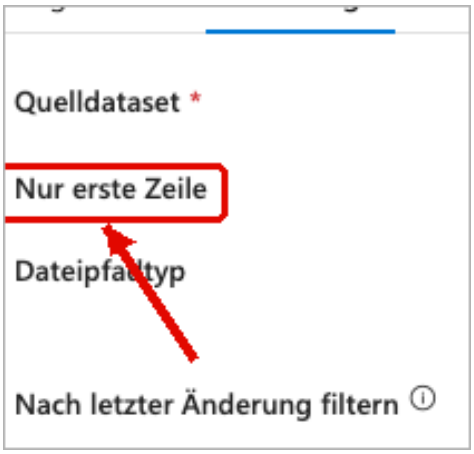


Abb. 122: Suche standorte_lesen: „Nur erste Zeile“ ohne Haken – alle Städte werden gelesen.

3. **Iteration und Bedingungen** → **ForEach** → grüner Pfeil von der Suche · Name `foreach_stadt` · **Sequenziell** ✓ · **Elemente** = `@activity('standorte_lesen').output.value`.

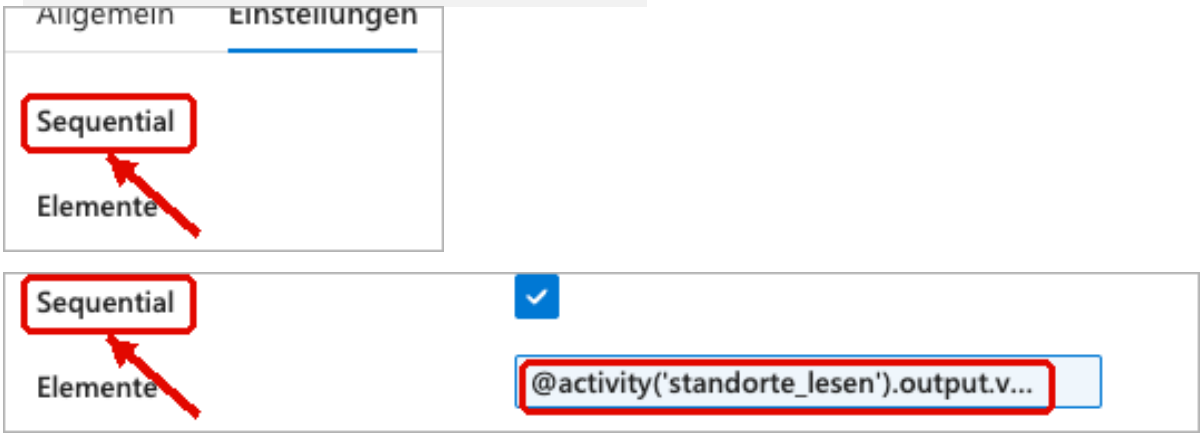


Abb. 123: ForEach foreach_stadt: sequenziell, Elemente = Ausgabe der Suche. (Ausschnitte 1–2)

4. In der ForEach-Karte **Stift** → innen **Synapse → Notizbuch** → Name `n4_gold` (Timeout `0.00:30:00`, 1, 360) · verknüpfter Dienst `ls_synapse_beehaven` · Notizbuch `04_gold_synapse` · Spark-Pool `sparkbeehaven` · **Basisparameter** + Neu: **Name** `STADT` · **Typ** Zeichenfolge · **Wert** `@item().stadt` – so bekommt das Notebook je Durchlauf die Stadt der aktuellen Zeile. **Executorgröße** und **Executors dynamisch zuordnen** bleiben leer (Werte des Pools).



Abb. 124: Notebook n4_gold: Basisparameter STADT = @item().stadt überschreibt die Parameterzelle.

> 💡 **Gut zu wissen:** **Basisparameter** überschreiben die Standardwerte in der Parameterzelle des Notebooks (`STADT = "schwartau"`). Die Schleife gibt bei jedem Durchgang die Stadt der aktuellen Zeile mit. **Sequenziell**, weil der Pool nur eine Spark-Sitzung gleichzeitig schafft. Bei vielen Städten lohnt später ein Gold-Steuer-Notebook (alle Städte in einer Sitzung) – siehe Anhang E.

1. Oben auf `pl_silver_to_gold` (Pfad über der Arbeitsfläche) → **Überprüfen** → **Debuggen** (ca. 0,12–0,15 €) → unten **Ausgabe**: **Pipelinestatus Erfolgreich**, `standorte_lesen`, `foreach_stadt` und `n4_gold` erfolgreich (Spalte **Dauer**, im Bild 3m 24s).

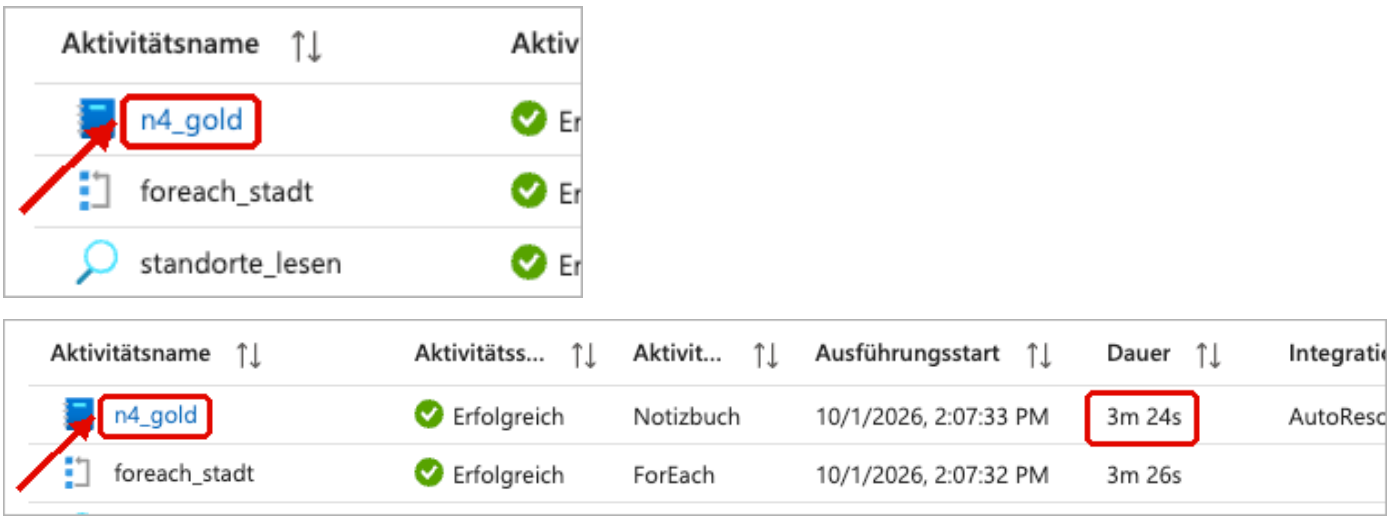


Abb. 125: Debug pl_silver_to_gold: Suche, Schleife und Gold-Notebook erfolgreich. (Ausschnitte 1–2)

1. Gegenprobe im Speicherbrowser: `lakehouse` → `gold` → `schwartau` → **Aktualisieren**: **stunden_schwartau.parquet**, **stunden_schwartau.csv**, **tageswerte_schwartau.parquet** und **tageswerte_schwartau.csv** mit festen Dateinamen (Dashboard und Analyse finden sie immer unter demselben Namen), Vorgänger im Ordner **_archiv**.



Abb. 126: Gegenprobe gold/schwartau: feste Dateinamen, Größen wie vorher, Vorgänger im _archiv.

> ✅ **Prüfen:** `standorte_lesen` wenige Sekunden, `n4_gold` ca. 3–4 Min. Größen (Spalte **Größe**): `stunden_schwartau.parquet` 953,25 KiB, `tageswerte_schwartau.parquet` 46,51 KiB; Vorgänger in `_archiv` mit `__bis_<UTC>`.

> 💡 **Gut zu wissen: KiB oder KB?** Azure zeigt KiB (1 KiB = 1.024 Byte), Python meist KB (1.000 Byte): 953,25 KiB = 976 KB. Gleiche Eingangsdaten ergeben gleiche Größen – ein einfacher Nachweis, dass die Verarbeitung reproduzierbar ist.

Schritt 33 – Gesamtlauf pl_daily_processing (Abnahme Bronze → Silver → Gold)

> **Warum?** Erst der **Gesamtlauf** mit frischen Rohdaten zeigt, ob alle Stufen in der richtigen Reihenfolge zusammenspielen – so, wie die Pipeline später täglich laufen würde.

Teil A – Gold-Stufe einhängen

1. `pl_daily_processing` öffnen → Pfeil zwischen `bronze_nach_silver` und `bronze_archivieren` anklicken → **Entf**.
2. **Pipeline ausführen** auf die Fläche → Name `silver_nach_gold`, Beschreibung „Stufe 3: startet pl_silver_to_gold (Gold-Tabellen je Stadt aus der Standort-Tabelle).“ · **Einstellungen**: aufgerufene Pipeline `pl_silver_to_gold`, **Auf Abschluss warten ✓**.



Abb. 127: Pipeline ausführen silver_nach_gold: aufgerufene Pipeline, Auf Abschluss warten.

3. Grüne Pfeile (bei Erfolg): `bronze_nach_silver` → `silver_nach_gold` → `bronze_archivieren` – Gold kommt vor das Archiv, damit bei einem Fehler die Rohdaten noch in `bronze/new` liegen. → **Überprüfen** → **Alle veröffentlichen**.

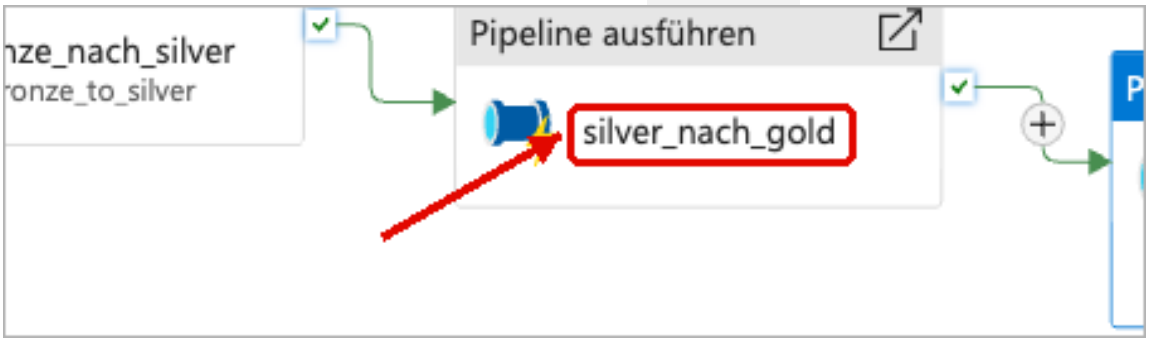


Abb. 128: pl_daily_processing mit vier Stufen; Archivieren immer zuletzt.

> 💡 **Gut zu wissen: Auf Abschluss warten:** Ohne Haken würde Gold nur angestoßen und das Archivieren liefe sofort weiter – ein Gold-Fehler bliebe unbemerkt. **Archivieren immer zuletzt:** Bricht eine Stufe ab, bleiben die Rohdaten in `bronze/new` und können neu verarbeitet werden.

Teil B – Gesamtlauf und Beobachtung

1. `pl_daily_processing` → **Debuggen** (zwei Spark-Sitzungen, ca. 9–12 Min., ca. 0,30 €).
2. Innere Läufe verfolgen: **Monitor** (Tacho links) → **Pipelineausführungen** → Reiter **Debuggen** – jede innere Pipeline steht als eigener Lauf, „Ausgelöst durch“ zeigt die Lauf-ID des Gesamtlaufs.

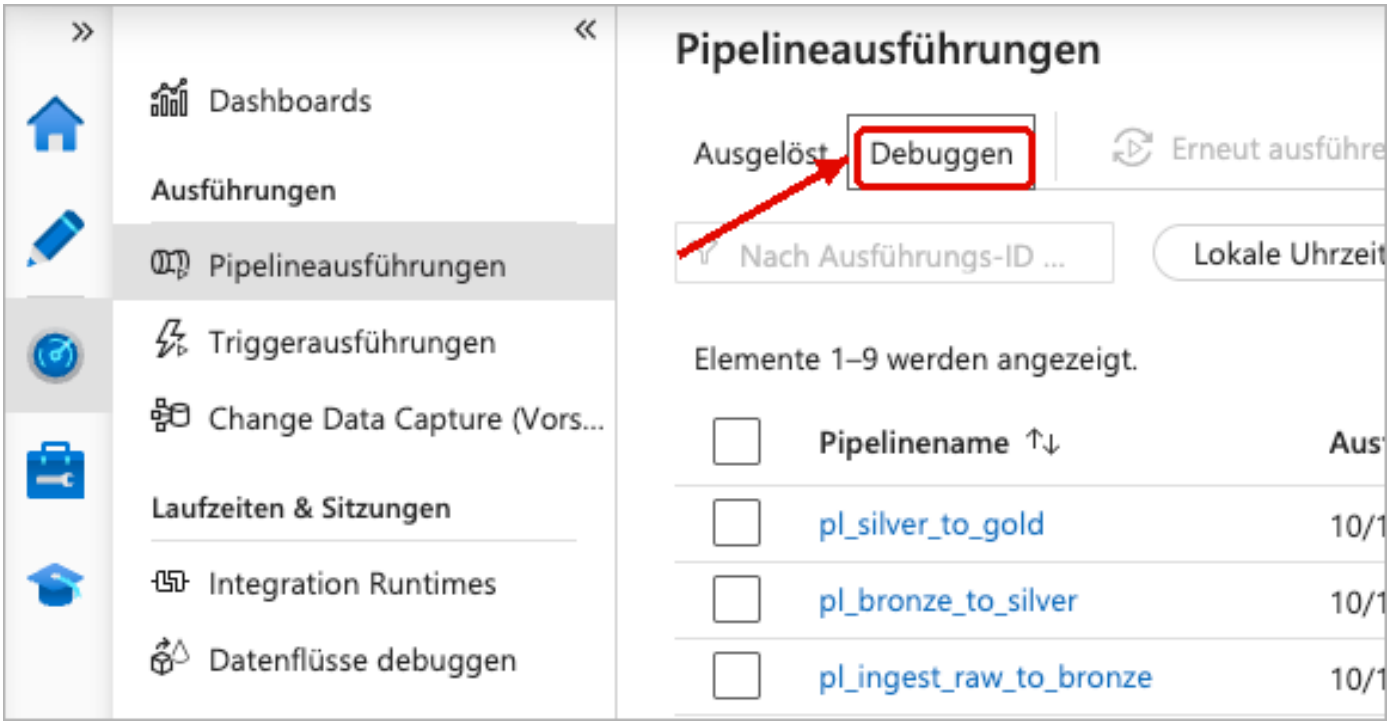


Abb. 129: Monitor → Pipelineausführungen → Debuggen: Gesamtlauf mit den inneren Pipelines.

3. Ausgabe von `pl_daily_processing`: **Pipelinestatus Erfolgreich**, alle vier Stufen (`rohdaten_nach_bronze`, `bronze_nach_silver`, `silver_nach_gold`, `bronze_archivieren`) erfolgreich; die Spalte **Dauer** zeigt, dass Silver am längsten braucht (Notebooks).

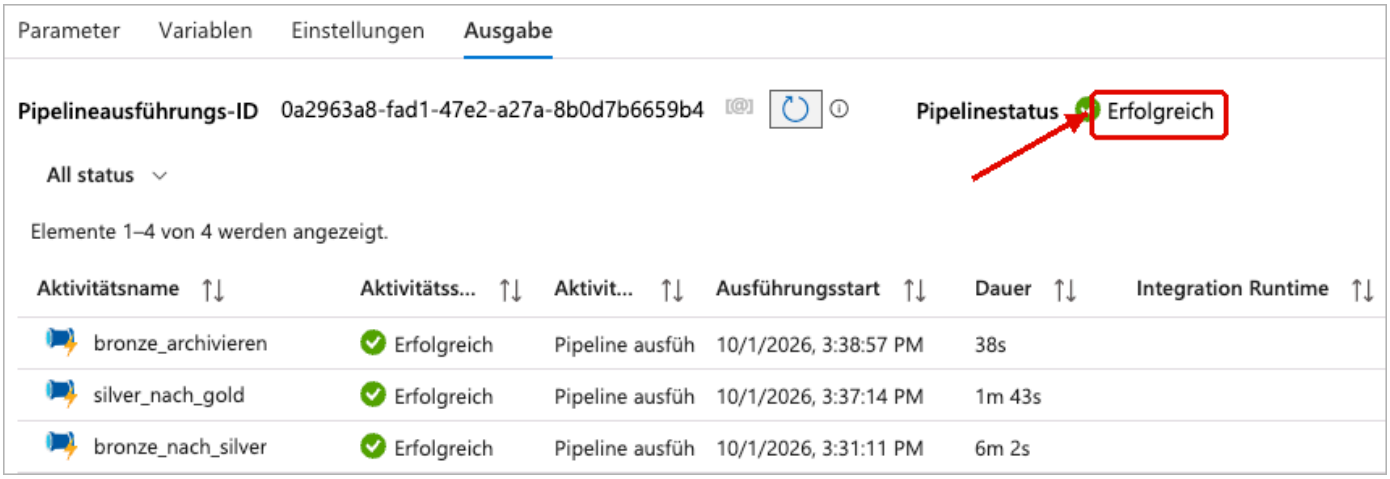


Abb. 130: Gesamtlauf pl_daily_processing: alle vier Stufen erfolgreich (ca. 9 Min.).

Teil C – Abnahme-Gegenprobe (Speicherbrowser, jeweils Aktualisieren)

Ordner	Erwartung (Gesamtlauf 01.10.2026)
`bronze/new/`	leer
`bronze/archive/flow_schwartau_raw/`	neue Datei `…__2026-10-01_133913_UTC.csv` (54,23 MiB), ältere Lieferungen bleiben – lückenlose Historie
`silver/processing/schwartau/<sensor>/`	leer
`silver/cleaned/schwartau/`	6 Ordner (4 Sensoren + `sonne`, `wetter`), Dateien von heute, Vorgänger in `_archiv`
`gold/schwartau/`	Dateien von heute, Größen wie zuvor

1. Beispiel `bronze/archive/flow_schwartau_raw/` → **Aktualisieren**: unten die neue Datei **flow_schwartau_raw__2026-10-01_133913_UTC.csv** aus dem Gesamtlauf, darüber die Lieferungen vom 24.09., 25.09., 28.09. und 30.09. – die Historie bleibt lückenlos. Die übrigen Ordner der Tabelle genauso prüfen.

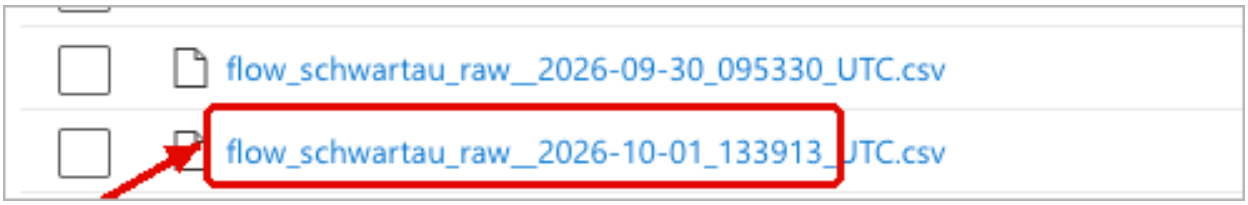


Abb. 131: Gegenprobe bronze/archive: neue Rohdatei mit UTC-Zeitstempel, ältere Lieferungen bleiben.

✅ **Prüfen:** Gesamtlauf ca. 9 Min.: Rohdaten 27 s · Silver 6 Min. 2 s · Gold 1 Min. 43 s (Pool noch „warm“) · Archiv 38 s. Alle Gegenproben bestanden → Bronze → Silver → Gold abgenommen.

💡 **Gut zu wissen:** **Einfach erklärt:** Generalprobe vor der Premiere: Alle spielen einmal das ganze Stück am Stück. Danach wird geprüft, ob jedes Requisit am richtigen Platz liegt.

Schritt 34 – Zweite Stadt (Würzburg) und Standortregeln aus der Konfiguration

Warum? „Neue Stadt = neue Zeile“ gilt für die Pipelines. Der erste Test mit Würzburg hat gezeigt, dass auch die **Notebook-Regeln** je Stadt verschieden sein müssen: Würzburg liefert das Gewicht in **kg** (Schwartau in **g**), misst minütlich statt alle 12 Stunden und enthält Fehlerwerte bis −227 °C.

💡 **Gut zu wissen:** **Hintergrund – Stammdaten statt Sonderfälle im Code:** Regeln, die je Standort verschieden sind, gehören in Konfigurationstabellen. Neue Stadt = neue Zeilen, kein Programmieren. So bleibt der Code für alle gleich und prüfbar.

Teil A – Konfiguration (lokal: 02_daten_ITS-Data/lakehouse/konfiguration/)

Datei	Spalten	Beispiel
`standorte.csv`	stadt, name, breite, laenge, zeitzone, quelle	`wurzburg, Würzburg, 49.777, 9.975, Europe/Berlin, …`
`sensoren.csv`	stadt, sensor, einheit_roh, min, max, bemerkung	`wurzburg, weight, kg, 20, 150, …` · `schwartau, weight, g, 20, 150, …` · Temperatur −40 … 60 °C
`ausschluesse.csv`	stadt, sensor, von_utc, bis_utc, grund	`schwartau, weight, 2018-04-28 12:00, 2018-07-02 00:00, Stock nicht auf Waage (E5)`

Teil B – lokal testen (ohne Azure-Kosten)

1. Neue Lieferung mit der vorhandenen vergleichen (Prüfsummen, Zeilen, Zeitraum) – Kursfassung Würzburg: flow ohne Stunde 00:00–00:59 ab 02.01.2019 (dokumentiert in `quelle/README.md`).
2. Würzburg-Fassungen der Notebooks werden aus den Hauptnotebooks erzeugt (`03_notebooks_ITS-Data/wurzburg/`, `STADT = "wurzburg"`, Bilder in `04_images/wurzburg/`) und nacheinander N1 → N5 ausgeführt.
3. Kontrolle: Schwartau nach jeder Regeländerung neu rechnen – Ergebnisse müssen **identisch** bleiben (Gold auf das Byte gleich).

> ✅ **Prüfen:** Würzburg N1–N5 ohne Fehler: Gewicht 1.015.006 gültige Werte (vorher 0), Tagesminimum −10 °C (vorher −227 °C), 249 auswertbare Saisontage. Schwartau unverändert.

Teil C – in Azure (Kurs „New Data Drop“, 06.10.2026)

1. `standorte.csv`, `sensoren.csv`, `ausschluesse.csv` nach `lakehouse/konfiguration/` hochladen, **Überschreiben** ✓ (Schritt 25). Danach stehen **ausschluesse.csv**, **sensoren.csv** und **standorte.csv** im Ordner – die Notebooks lesen daraus die Regeln je Stadt.



Abb. 132: lakehouse/konfiguration: drei Konfigurationsdateien (Größen wie lokal).

2. Synapse → Entwickeln → **+** → **Importieren** → im Dateidialog den Ordner **synapse** öffnen (nicht die gleichnamigen lokalen Notebooks darüber – im Bild sind sie noch markiert) → die vier `…_synapse.ipynb` markieren → **Öffnen**.

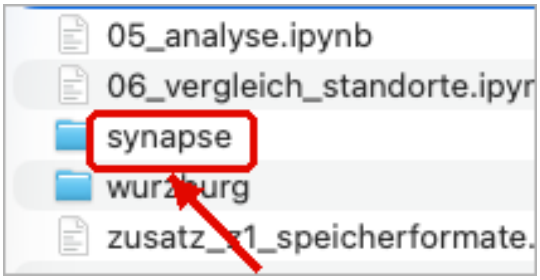


Abb. 133: Import: im Ordner synapse/ die vier Dateien mit _synapse wählen.

3. Meldung **Der Elementname ist bereits vorhanden (1 of 4)** → **Ersetzen** (viermal) – so wird die alte Fassung überschrieben statt eine Kopie angelegt.

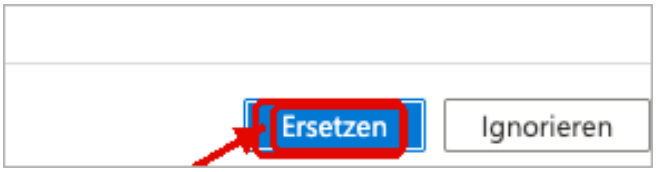


Abb. 134: Der Elementname ist bereits vorhanden → Ersetzen.

4. Kontrolle: in `02_qualitaet_synapse` stehen die Zeilen, die **"sensoren.csv"** und `ausschluesse.csv` lesen – nur dann ist die neue Fassung angekommen (Ersetzen greift nicht immer).

```
# --- Standortdaten aus der Konfiguration (Koordinaten, Zeitzone) ---
standorte = pd.read_csv(pfad("konfiguration", "standorte.csv"), index_col="
if STADT not in standorte.index:                                  # unbekannte Stad
    raise ValueError(f"Standort '{STADT}' fehlt in konfiguration/standorte.
ORT = standorte.loc[STADT]                                        # eine Zeile: nam
ZEITZONE = ORT["zeitzone"]                                        # Ortszeit der Ro

# --- Sensor-Regeln je Standort (Einheit der Rohdaten, gültiger Bereich) un
sensoren_cfg = pd.read_csv(pfad("konfiguration", "sensoren.csv"), storage_o
REGELN = sensoren_cfg[sensoren_cfg["stadt"] == STADT].set_index("sensor")
ausschluesse = pd.read_csv(pfad("konfiguration", "ausschluesse.csv"), stora
AUSSCHLUESSE = ausschluesse[ausschluesse["stadt"] == STADT].copy()
AUSSCHLUESSE[["von_utc", "bis_utc"]] = AUSSCHLUESSE[["von_utc", "bis_utc"]]
```

Abb. 135: Kontrolle nach dem Import: neue Zeilen mit sensoren.csv und ausschluesse.csv in N2.

5. Jedes Notebook **Anfügen an** `sparkbeehaven` → **Alle veröffentlichen**: Unter **Ausstehende Änderungen (4)** stehen `01_bereinigung_synapse` … `04_gold_synapse` (**Bearbeitet**) → **Veröffentlichen**.

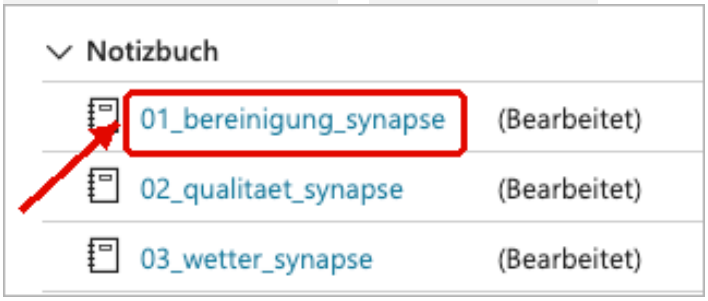


Abb. 136: Alle veröffentlichen: vier Notebooks bearbeitet, Namen unverändert.

6. Würzburg-Rohdaten in die Quelle `mabeehavenraw/source-data` (ohne Überschreiben) → „Alle 8 Elemente“: je Sensor `…_schwartau.csv` und `…_wurzburg.csv` (z. B. **flow_wurzburg.csv**).

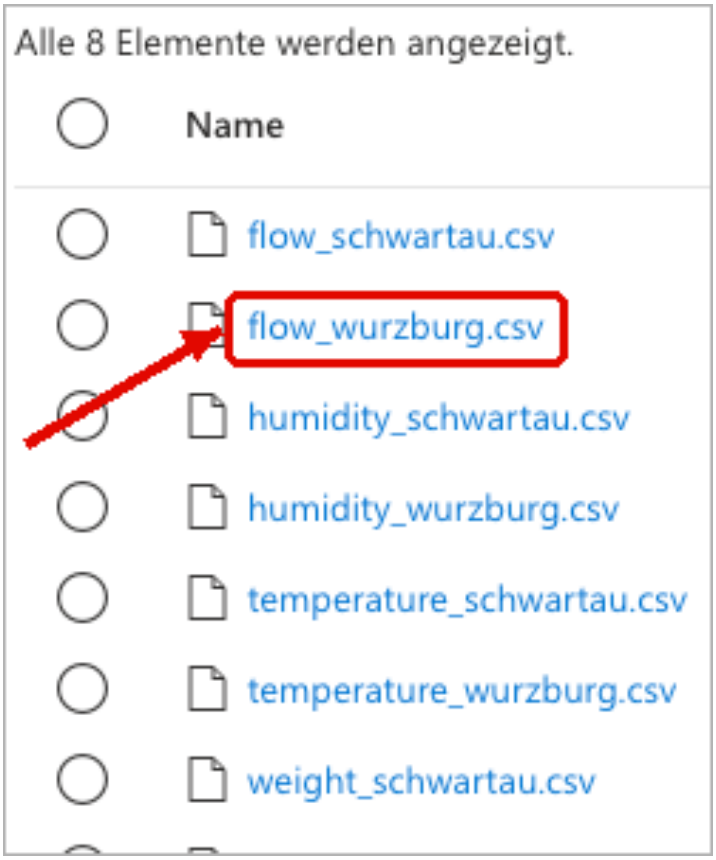


Abb. 137: Quelle mabeehavenraw/source-data: je vier Dateien Schwartau und Würzburg.

7. `pl_daily_processing` **Debuggen** → Gegenprobe für beide Städte (Schritt 33), z. B. liegen in `gold/wurzburg/` die vier Dateien **stunden_wurzburg.csv**, `stunden_wurzburg.parquet`, `tageswerte_wurzburg.csv` und **tageswerte_wurzburg.parquet** – die zweite Stadt läuft ohne Änderung an Pipelines oder Notebooks.

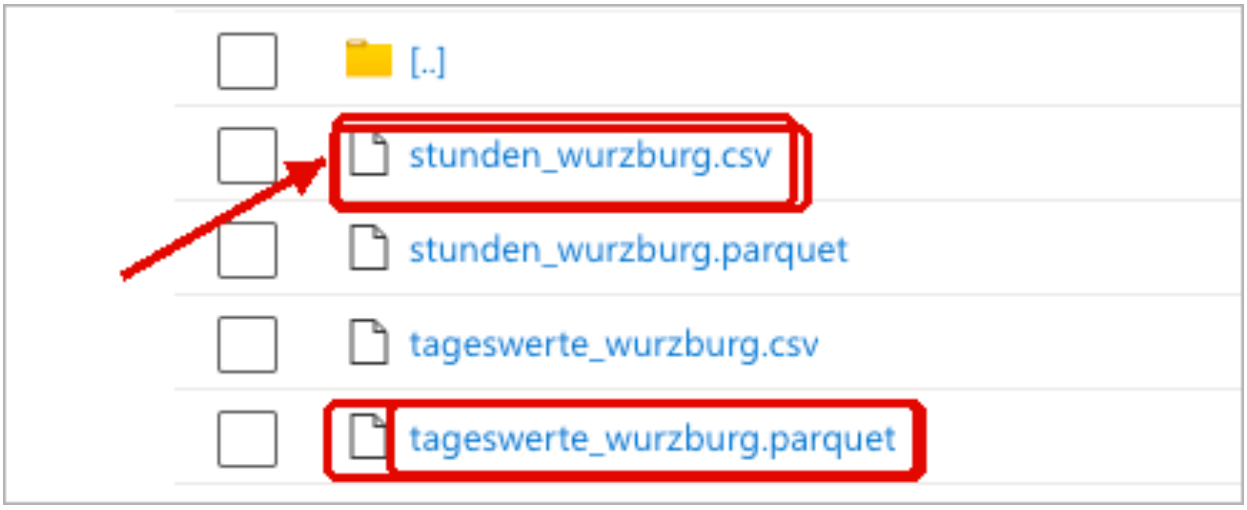


Abb. 138: Gegenprobe nach dem Gesamtlauf beider Städte: gold/wurzburg enthält Stunden- und Tageswerte als CSV und Parquet.

> ✅ **Prüfen:** Azure 02.10.2026: Gesamtlauf beider Städte 23 s / 8 Min. 9 s / 3 Min. 43 s / 56 s. `gold/wurzburg`: CSV identisch mit lokal (3,48 MiB / 66,33 KiB).

> 💡 **Gut zu wissen: Gegenprobe Azure ↔ lokal über die CSV-Größe:** Parquet-Dateien sind in Azure etwa 0,5 % größer (andere Version der Parquet-Bibliothek pyarrow), der Inhalt ist gleich. Bei Parquet ±1 % tolerieren.

> ⚠️ **Achtung: Import in Synapse:** im Ordner `synapse/` die Dateien mit `_synapse` wählen – nicht die gleichnamigen lokalen Notebooks eine Ebene höher. Beim Import mehrerer Dateien greift „Ersetzen“ nicht immer: danach Inhalt stichprobenartig prüfen (z. B. `sensoren.csv` in N2), notfalls einzeln importieren. Hochladen in die **Quelle** ohne „Überschreiben“, in **konfiguration** mit „Überschreiben“.
>
> ⚠️ **Achtung:** Die Synapse-Notebooks lesen ab Version 01.10.2026 `sensoren.csv` und `ausschluesse.csv`. Fehlen die Dateien in `konfiguration/`, bricht N1 sofort ab – deshalb Teil C in genau dieser Reihenfolge.

Kennzahl (vorläufig)	Schwartau	Würzburg
Gewicht (Median)	60 kg	53,7 kg
Honig je Einflug	6,3 mg	7,8 mg
Verbrauch ohne Flug	ca. 259 g/Tag	ca. 156 g/Tag
Gewinnschwelle	ca. 41.500 Einflüge/Tag	ca. 20.000 Einflüge/Tag

> 💡 **Gut zu wissen: Einfach erklärt:** Zwei Waagen zeigen dasselbe Gewicht – die eine in Gramm, die andere in Kilogramm. Ohne Umrechnung hält das Programm den zweiten Bienenstock für fast leer.

Schritt 35 – Zeitplan-Trigger, Nebenläufigkeit und Fehler-E-Mail (Betrieb absichern)

> **Warum?** Im Betrieb soll die Kette ohne Handarbeit laufen, nie doppelt gleichzeitig, und Fehler oder hängende Läufe sollen sofort auffallen. Für das Lernprojekt wird der Trigger **angelegt, aber nicht gestartet** – jeder Lauf kostet ca. 0,50 €.

Teil A – Trigger anlegen (ADF Studio → Erstellen → pl_daily_processing)

1. Oben **Trigger hinzufügen** → **Neu/Bearbeiten** (nicht „Jetzt auslösen“ – das startet sofort) → **+ Neu**.
2. Name `tr_taeglich_0000_utc` · Beschreibung „Startet pl_daily_processing täglich um 00:00 UTC. Zunächst deaktiviert (Kosten ca. 0,50 € je Lauf); Aktivierung nur nach Freigabe.“ · Typ **Zeitplan** · Startdatum (nächster Tag 00:00) · Zeitzone **Coordinated Universal Time (UTC)** · Wiederholung alle **1 Tag(e)** · **Erweiterte Wiederholungsoptionen** (Stunden, Minuten) leer = 00:00 aus dem Startdatum · **Enddatum angeben** leer · **Trigger bei Erstellung starten: kein Haken** (jeder Lauf kostet – erst nach Freigabe starten) → **OK** → **OK** → **Alle veröffentlichen**.

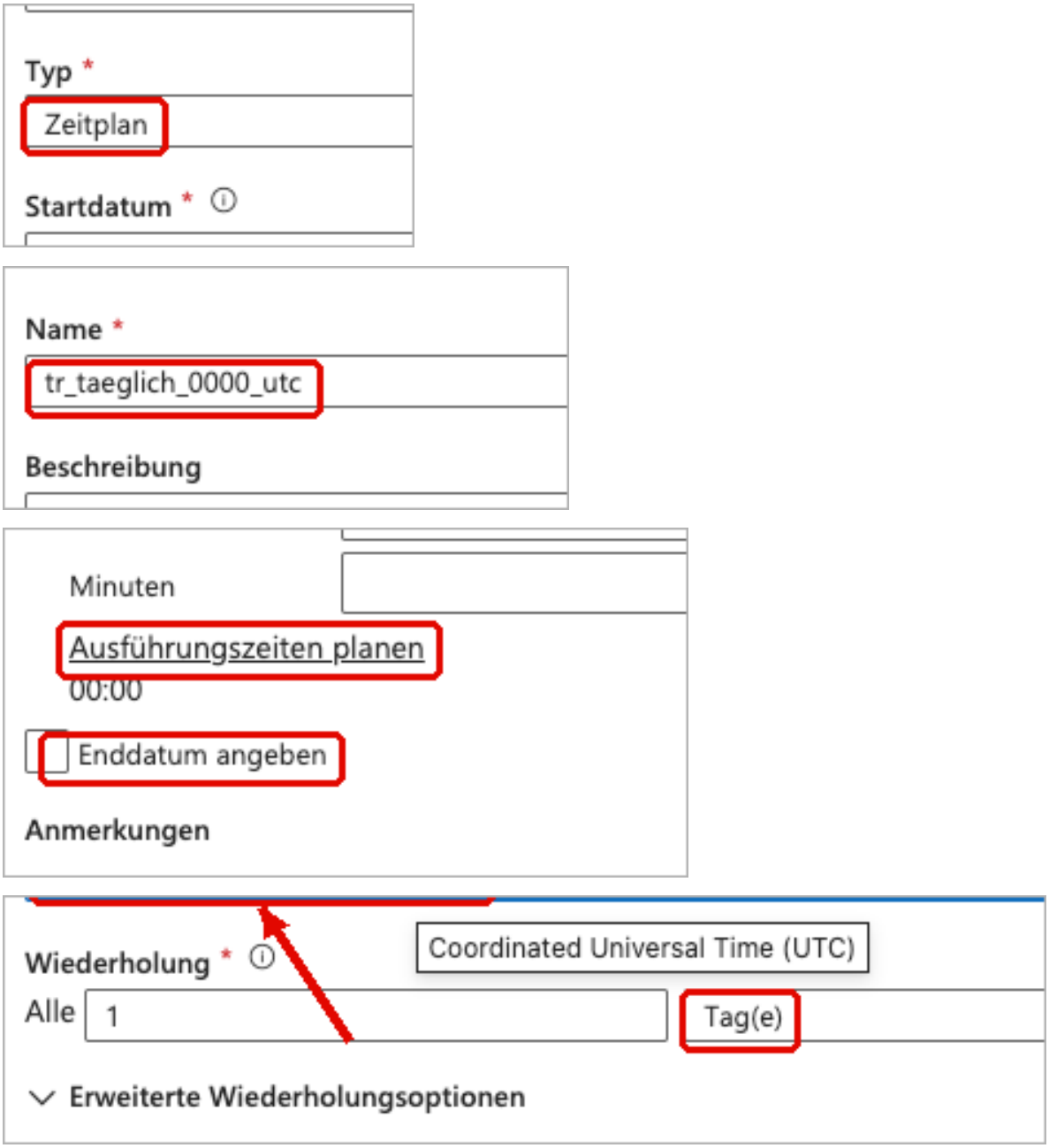


Abb. 139: Trigger: Zeitzone UTC (ohne Zahl), täglich, nicht bei Erstellung starten. (Ausschnitte 1–4)

3. Kontrolle: **Verwalten** → **Auslöser** → Status **Beendet**.

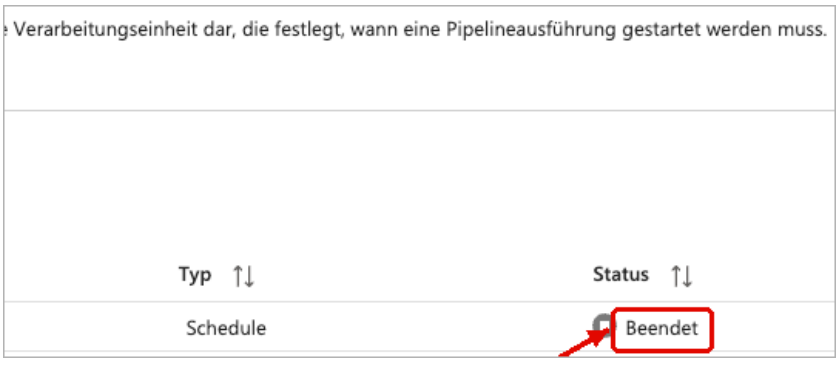


Abb. 140: Verwalten → Auslöser: tr_taeglich_0000_utc, Status Beendet.

> ⚠ **Achtung: Zeitzonen-Falle:** Die Suche „utc“ wählt „UTC-12“ oder „UTC-11“. Richtig ist der Eintrag **ohne** Zahl. Nicht „London (UTC+00:00)“ – dort gilt Sommerzeit. Im deutschen Menü heißt der Trigger **„Auslöser“** (Verwalten → Autor); Status **Beendet** = läuft nicht.

Teil B – Nebenläufigkeit und Laufzeit-Grenze

1. `pl_daily_processing` → leere Stelle der Arbeitsfläche → Reiter **Einstellungen**: **Nebenläufigkeit** `1` · **Metrik für verstrichene Zeit** ✓ `0.00:30:00` → **Alle veröffentlichen**.

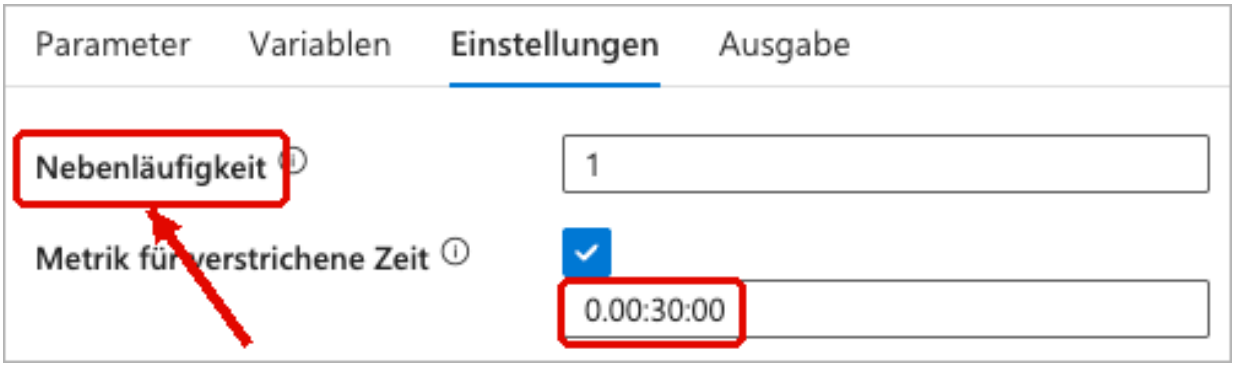
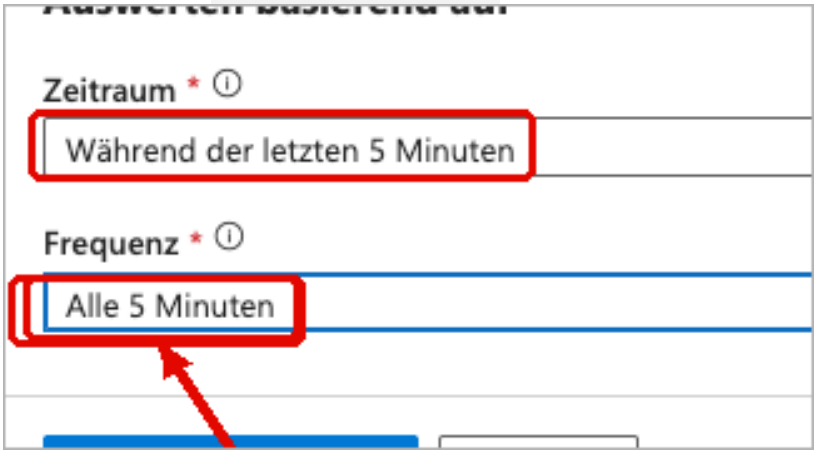


Abb. 141: pl_daily_processing → Einstellungen: Nebenläufigkeit 1, Laufzeit-Grenze 30 Min.

> 💡 **Gut zu wissen: Hintergrund:** Nebenläufigkeit 1 verhindert, dass ein neuer Lauf startet, während der vorige noch arbeitet (dieselben Dateien!). Die Laufzeit-Metrik meldet Läufe, die länger als 30 Minuten dauern (normal ca. 13 Min.). Die Timeouts der Aktivitäten gelten bei jedem Start – auch beim Trigger.

Teil C – Fehler-E-Mail (Überwachen → Warnungen und Metriken)

1. **+ Neue Warnungsregel** → Name `al_pipeline_fehler`, Schweregrad 1 → **Kriterien hinzufügen** → „Failed pipeline runs metrics“ → Name `pl_daily_processing`, FailureType leer → Ist größer als · Gesamt · `0` · 5 Minuten / alle 5 Minuten.



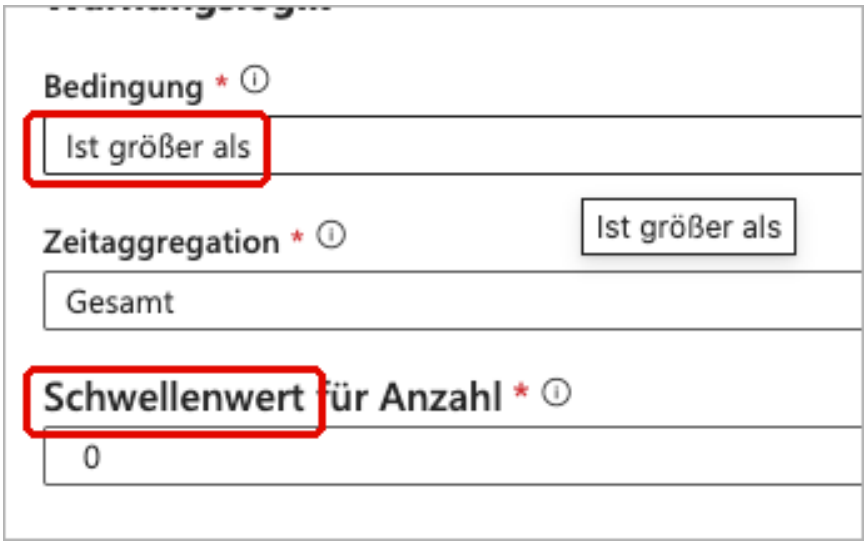




Abb. 142: Warnlogik: Failed pipeline runs > 0, 5 Minuten. (Ausschnitte 1–3)

2. **Benachrichtigung konfigurieren** → **Neu erstellen**: Aktionsgruppe `ag_beehaven_mail`, Kurzname `beehaven` → Benachrichtigung: Feld **Name der Aktion** `mail_projektleitung`, **E-Mail** anhaken und die eigene Adresse eintragen;

SMS, **Azure-App-Pushbenachrichtigungen** und **Sprache** bleiben leer (E-Mail reicht, SMS kann Gebühren kosten) → **Benachrichtigung** → **Warnungsregel erstellen**.

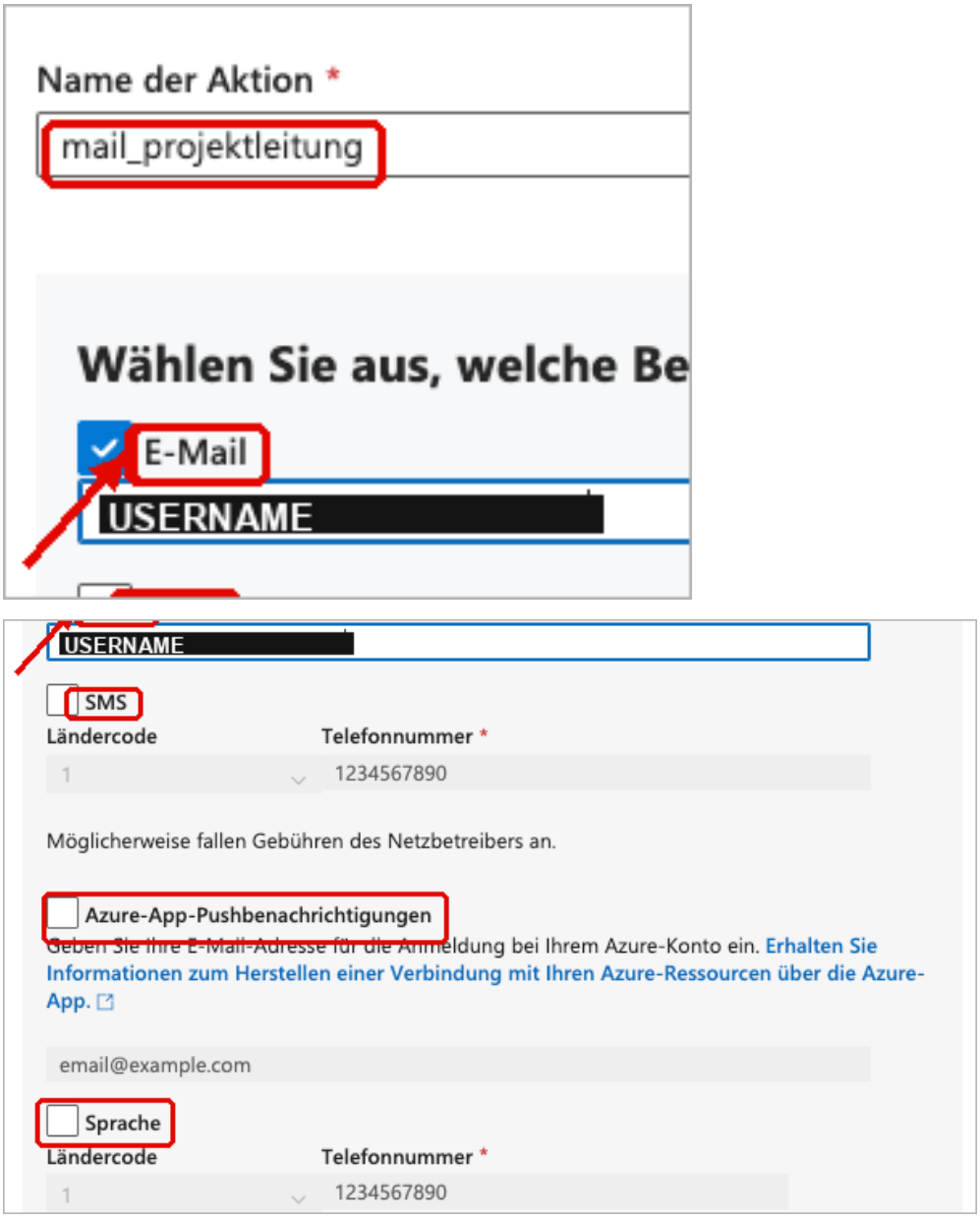


Abb. 143: Aktionsgruppe: nur E-Mail (SMS, App, Sprache aus). (Ausschnitte 1–2)

3. Bestätigungs-E-Mail von Microsoft: **Verify my email** und Code eingeben (30 Minuten gültig) – ohne Bestätigung keine Warnungen.
4. Zweite Regel `al_pipeline_laufzeit`, Schweregrad 2, Metrik „Elapsed Time Pipeline Runs Metrics“ > 0 → Benachrichtigung **Vorhandene verwenden** → `ag_beehaven_mail`.

Falls beim Erstellen ein Fehler kommt: Ressourcenanbieter registrieren

1. Meldung **Fehler** mit `"code":"MissingSubscriptionRegistration"` und `microsoft.insights`: Das Abonnement hat Azure Monitor (zuständig für Warnregeln) noch nicht freigeschaltet – in neuen Abonnements ist das normal. Formular **nicht** schließen.

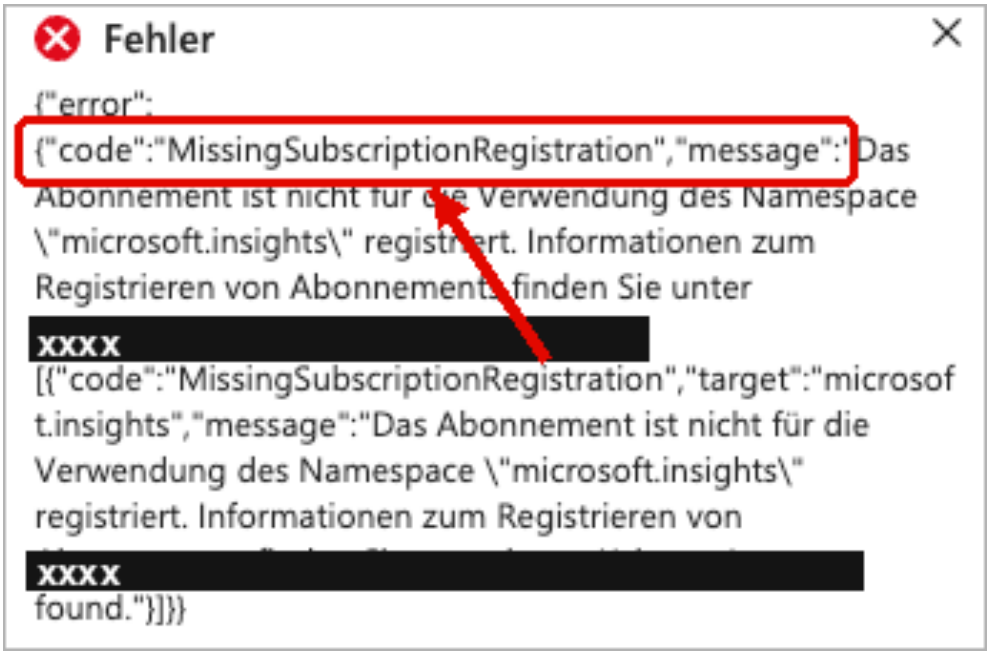


Abb. 144: Fehler beim ersten Versuch: Ressourcenanbieter microsoft.insights nicht registriert.

2. In einem **zweiten Browser-Tab** (das Formular im ersten bleibt erhalten): Abonnement → **Einstellungen** → **Ressourcenanbieter** → im Suchfeld `Microsoft.Insights` → Zeile `microsoft.insights` über den Kreis links markieren (nicht auf den Namen klicken) → oben **Registrieren** → nach einigen Minuten **Aktualisieren**: **Status** = **Registered**. Kostenlos; **Registrierungsrichtlinie** RegistrationRequired heißt nur, dass Azure die Freischaltung nicht selbst vornimmt.



Abb. 145: Abonnement → Ressourcenanbieter: microsoft.insights Registered.

3. Im ersten Tab die Regel erneut erstellen. Danach **Warnungen und Metriken** → **Aktualisieren**: **al_pipeline_fehler** und **al_pipeline_laufzeit** stehen mit **Aktiviert** = **Ein** in der Liste.



Abb. 146: Warnungen und Metriken: zwei Regeln aktiv.

> 💡 **Gut zu wissen:** **Esc** schließt Azure-Fenster ohne Speichern. Die Checkliste für neue Abonnements (Vorlage VL-ITS-002) nennt die Registrierung als ersten Punkt, damit der Fehler gar nicht erst auftritt.

> ✅ **Prüfen:** Auslöser `tr_taeglich_0000_utc` Status **Beendet** · Nebenläufigkeit 1 · Warnungen und Metriken: 2 Regeln **Ein**. Kosten: ca. 0,10 USD je Regel und Monat; Ist-Kosten des Projekts bis 02.10.2026: 1,02 €.

> 💡 **Gut zu wissen:** **Einfach erklärt:** Der Wecker ist gestellt, aber ausgeschaltet; die Tür lässt nur einen Koch gleichzeitig herein; und ein Rauchmelder schickt eine Nachricht, wenn etwas anbrennt oder zu lange dauert.

Ausblick – nächste Schritte

#	Baustein	Warum	Stand
1	Kosten eines Notebook-Laufs schätzen und freigeben	Spark kostet je Minute	erreicht (Schritt 28)
2	Synapse-Arbeitsbereich, Notebooks, Aktivitäten `silver_automatisierung` und `n4_gold` (`pl_silver_to_gold`)	Bereinigung und Gold in Azure	erreicht (Schritte 29–33)
3	Trigger täglich 00:00 UTC (gestoppt), Nebenläufigkeit, Fehler-E-Mail	läuft ohne Handarbeit, Fehler fallen auf	erreicht (Schritt 35)
4	Dashboard mit Warnhinweisen (Kennzahlen-Konzept in `08_dashboard_ITS-DA`)	Nutzen für die Imkerei	Folgeprojekt FP-2
5	Zweite Stadt Würzburg in Azure (Konfiguration hochladen, Notebooks neu importieren, Gesamtlauf beider Städte)	Mehrere Standorte, Standortregeln	erreicht (Schritt 34)
6	Git für Data Factory und Synapse, inkrementelle Verarbeitung, Lieferprüfung, Berechtigungskonzept	Sicherheit, Kosten, Nachvollziehbarkeit	Folgeprojekte FP-1, FP-5
7	Checkliste neues Abonnement (Ressourcenanbieter, Budget, Rechte, Warnregeln)	Fehler wie MissingSubscriptionRegistration vermeiden	Vorlage `06_vorlagen` VL-ITS-002

Häufige Fragen (FAQ)

Grundlagen

Was ist der Unterschied zwischen Azure-Portal und ADF Studio?

Im Portal legst du Ressourcen an und vergibst Rechte. Im ADF Studio baust du Verbindungen, Datasets und Pipelines.

Liegt das Lakehouse im Container?

Ein Lakehouse ist ein Architekturkonzept (Data Lake + abfragbare Tabellen). Technisch: Speicherkonto mit hierarchischem Namespace = Data Lake, Container = Bereich, Ordner = Medallion-Stufen.

Warum zwei Speicherkonten?

Quelle = Posteingang eines Unternehmens; Data Lake = Ort der Veredelung. Die Data Factory transportiert dazwischen.

Wofür ist Gold?

Fertige Kennzahlen für Entscheidungen, z. B. in Power BI – mit reinem Leserecht.

Was kostet das?

Für diesen Aufbau nur Cent-Beträge (Speicher im MB-Bereich, wenige Pipeline-Läufe). Teuer wird es erst mit Data Flows (Spark-Cluster) – dort den Debugmodus nach der Arbeit ausschalten.

Namen und Rechte

Was bedeuten rg, adf, ls, ds, pl?

Namens-Präfixe (Tabelle unten): Man erkennt sofort, welche Art von Ressource es ist.

Warum Kürzel/Projekt nicht im Containernamen?

Containernamen gelten nur innerhalb ihres Speicherkontos; Projekt und Kürzel stecken schon im Kontonamen.

Warum Rolle statt Person im Tag?

Tags werden weltweit gespeichert (keine personenbezogenen Daten); Personen wechseln, Rollen bleiben.

Warum verwaltete Identität statt Kontoschlüssel?

Kein geheimer Schlüssel mit Vollzugriff im Umlauf; nur die nötigen Rechte; nichts, was ablaufen oder verloren gehen kann.

Die Pipeline meldet „Zugriff verweigert“ – was tun?

In der IAM des Speicherkontos prüfen: Hat `adf-beehaven-dev-gwc` die Datenrolle (Storage Blob Data …) und nicht nur „Leser“? Nach dem Zuweisen einige Minuten warten.

Pipelines

Was ist der Unterschied zwischen Debuggen und Veröffentlichen?

Debuggen = Probelauf mit dem aktuellen Entwurf. Veröffentlichen = Entwurf dauerhaft speichern, damit Trigger ihn nutzen können.

Ich sehe links noch den alten Namen – was ist falsch?

Nichts: Namen erscheinen in der Liste erst nach **Publish all** und ↻ **Refresh**.

Warum Binär in Bronze, CSV in Silver?

Bronze kopiert Dateien 1:1 ohne sie zu lesen. Silver muss den Inhalt als Tabelle lesen, um ihn später zu bereinigen.

Was ist childItems?

Die Liste der Dateien und Ordner, die Get Metadata in einem Ordner findet – jeder Eintrag hat `name` und `type`.

Warum @item().name und nicht der Dateiname fest?

Die Schleife setzt nacheinander jeden Dateinamen ein – neue Dateien werden automatisch mitverarbeitet.

Die Pipeline ist „erfolgreich“, aber der Ordner ist leer?

Senke und Pfad prüfen (richtiges Dataset? richtiger Ordner?) und ob `bronze/new` vor dem Lauf überhaupt Dateien enthielt.

Warum UTC im Dateinamen?

Keine Sommerzeit-Sprünge, eindeutig sortierbar; die Zeitzone ist im Namen dokumentiert.

Wie kommentiere ich eine Pipeline?

Feld **Description (Beschreibung)** in jeder Aktivität und in der Pipeline ausfüllen – ADF-Ausdrücke selbst haben keine Kommentare.

Namens-Präfixe

Präfix	Bedeutung EN (DE)	Beispiel
`rg-`	Resource Group (Ressourcengruppe)	`rg-beehaven-dev-gwc`
`adf-`	Azure Data Factory	`adf-beehaven-dev-gwc`
`ls_blob_` / `ls_adls_`	Linked Service (verknüpfter Dienst) zu Blob / Data Lake Storage	`ls_adls_mabeehavenlakehouse`
`ds_`	Dataset (Datensatz) – Endung _bin/_csv = Format	`ds_lake_silver_csv`
`pl_`	Pipeline	`pl_bronze_to_silver`
`-dev` / `-prod`	Development (Test) / Production (Betrieb)	`rg-beehaven-dev-gwc`
`-gwc`	Germany West Central	`adf-beehaven-dev-gwc`

Notebooks

Warum mehrere Notebooks statt einem?

Jedes Notebook hat eine Aufgabe (Kursstandard); ändert sich eine Regel, läuft nur das betroffene Notebook neu; kürzere Notebooks sind leichter zu prüfen. In Azure hält das Steuer-Notebook die Kosten trotzdem niedrig.

Warum UTC?

UTC kennt keine Sommerzeit; jede Uhrzeit ist eindeutig. Die 24 „Duplikate“ bei temperature waren nur Folgen der Zeitumstellung.

Warum werden Lücken erst in Gold aufgefüllt?

Silver soll die Messung zeigen, wie sie war. Verschiedene Verwendungen brauchen verschiedene Regeln; in Gold lassen sie sich ändern, ohne neu zu bereinigen – und die Kennzeichen-Spalte zeigt, was geschätzt ist.

Wo liegen die Notebooks?

Im Ordner `03_notebooks_ITS-Data/`: `00_steuerung_silver`, `01_bereinigung`, `02_qualitaet`, `03_wetter`, `04_gold`, `05_analyse`. Daten in `02_daten_ITS-Data/`, Grafiken in `04_images/`.

Wie kommt eine neue Stadt dazu?

Zeile in `konfiguration/standorte.csv` (Stadt, Koordinaten, Zeitzone) und Dateien `<sensor>_<stadt>.csv` in die Quelle legen. Achtung: andere Standorte können andere Einheiten und Messtakte haben (Beispiel Würzburg: Gewicht in kg, minütlich) – vorher prüfen.

Soll VS Code auf Deutsch umgestellt werden?

Empfehlung: Englisch lassen – Anleitungen, Foren und Fehlermeldungen sind meist englisch.

Automatisierung und Betrieb (Phase 3)

Was kostet ein Lauf, und wann entstehen Kosten?

Kosten entstehen fast nur, solange der Spark-Pool läuft (1,80 USD je Stunde, minutengenau, danach pausiert er nach 5 Minuten). Ein Gesamtlauf mit zwei Städten dauert ca. 13 Minuten und kostet ca. 0,50 €; alle Tests des Projekts zusammen 1,02 €. Speicher und Data Factory kosten im Projektumfang nur Cent-Beträge.

Warum ist der Trigger angelegt, aber gestoppt?

Jeder Lauf kostet Geld. Der Trigger wird erst nach einer Kostenfreigabe gestartet (Human Loop) – sinnvoll erst mit inkrementeller Verarbeitung, damit nicht jede Nacht alle Jahre neu gerechnet werden (Folgeprojekt FP-1).

Der Monitor zeigt „Erfolgreich“ – ist dann alles richtig?

Nein. „Erfolgreich“ heißt nur „kein Abbruch“; auch ein Löschen ohne Treffer meldet Erfolg. Nach jedem Testlauf folgt die Gegenprobe im Data Lake (Arbeitsanweisung AA-ITS-001 in `07_prozesse_QM/`).

Was passiert bei einem Fehler in der Nacht?

Die Kette bricht ab, die Rohdaten bleiben in `bronze/new`, die Arbeitskopien in `silver/processing`. Die Warnregel schickt eine E-Mail; der nächste Lauf wiederholt alles – es geht nichts verloren.

Warum laufen die Notebooks über ein Steuer-Notebook?

Jede Notebook-Aktivität startet eine eigene Spark-Sitzung (3–4 Minuten Start, Kosten). Das Steuer-Notebook startet N1 → N2 → N3 aller Städte in einer Sitzung; die Notebooks bleiben trotzdem einzeln und übersichtlich.

Wie kommt eine neue Stadt in Azure dazu?

Zeilen in `standorte.csv`, `sensoren.csv` (und bei Bedarf `ausschluesse.csv`) ergänzen und nach `lakehouse/konfiguration/` hochladen, Rohdaten `<sensor>_<stadt>.csv` in `source-data` legen, Gesamtlauf mit Gegenprobe. Pipelines und Notebooks bleiben unverändert (Vorlage VL-ITS-001).

Warum fehlen Warnregeln oder lässt sich die Warnung nicht anlegen?

Im neuen Abonnement ist der Ressourcenanbieter `microsoft.insights` nicht registriert (Fehler MissingSubscriptionRegistration). Abonnement → Einstellungen → Ressourcenanbieter → registrieren; Checkliste VL-ITS-002.

Ich habe ein Notebook in Synapse geändert – warum ist es lokal anders?

Synapse-Fassungen werden aus den lokalen Notebooks erzeugt. Änderungen immer lokal machen, Generator laufen lassen, neu importieren und veröffentlichen; danach Inhalt stichprobenartig prüfen (Ersetzen beim Mehrfach-Import greift nicht immer).

Anhang A – Typische Fehler Azure

Situation	Fehler / Risiko	Richtig
Neue Ressource	Region vorbelegt „East US“	Germany West Central wählen
Marketplace	Fremdanbieter mit ähnlichem Namen, teuer	Microsoft-Kachel, „Nur Azure-Dienste“
Namen	nicht zweckbezogen, nicht umbenennbar	Konvention rg-/adf-/ls_/ds_/pl_; Container nach Zweck
Tags	Personennamen; nicht vererbt	Rollen; an jeder Ressource setzen
Löschen	falsche Gruppe (ähnliche Namen)	Name zweimal lesen; vorher Inhalt prüfen
IAM	„Leser“ statt Datenrolle; „Besitzer“; eigene Person statt verwaltete Identität	Storage-Blob…-Rollen, minimal; verwaltete Identität der ADF
Verzeichnis	Doppelklick → PathAlreadyExists	harmlos (Bild unten)
Studio starten	„keine Berechtigung“	erneut klicken / adf.azure.com / Inkognito
Linked Service	Blob statt ADLS Gen2; Kontoschlüssel	ADLS Gen2; verwaltete Identität
Veröffentlichen	vergessen; links alter Name	Publish all, danach ↻
Schleife	`@item().file_name`; Senke = Quelle	`@item().name`; richtige Senke
Binärkopie	Ordner (Platzhalter) → Datei	Datei → Datei mit Parameter-Dataset
Parameter	Werte vertauscht (alphabetische Sortierung)	am Zeilennamen orientieren
Dataset mit @item()	`@item().name` direkt im Dataset → Fehler „FailToResolveParameters…“	im Dataset nur `@dataset().file_name`; `@item()` nur in der Aktivität
Archivieren	nur kopieren → bronze/new wird nie leer; verschieben vor Silver → Silver leer	erst Silver, dann verschieben
Zeitstempel	`hh` (12 h), lokale Zeit	`HH`, UTC + _UTC
Fabric-Hinweis	kostenpflichtige Kapazität	klassische ADF behalten

1. Harmlose Meldung nach einem Doppelklick auf **Save (Speichern)**: **Fehler beim Hinzufügen des Verzeichnisses** mit `PathAlreadyExists` – der erste Klick hat den Ordner (hier `archive`) schon angelegt, der zweite findet ihn vor. Meldung mit **X** schließen, nichts weiter tun; der Ordner steht in der Liste.



Abb. 147: Harmlose Meldung „PathAlreadyExists“ nach doppeltem Klick auf Speichern.

Anhang B – Typische Fehler Notebooks

Situation	Fehler	Lösung
Code läuft nicht	Restricted Mode	Manage → Trust
Kernel fehlt	falsche Umgebung	Select Kernel → beehaven
NameError	vorherige Zellen nicht ausgeführt	Run All
„Standort fehlt in standorte.csv“	Stadt nicht eingetragen	Zeile ergänzen
Wetterabruf scheitert	Schnittstelle gestört	später erneut starten (4 Versuche eingebaut)
Synapse-Kopie veraltet	lokales Notebook geändert	Generator erneut ausführen

Anhang C – Projektbezogene Befehle

Allgemeine Befehle (Terminal, Conda, SQL, Azure) stehen in den Befehlsmatrizen bei QM (`organisation/Abteilungen/ITS_IT-Security/Befehlsmatrizen/`, leere Vorlage bei QM). Hier nur die Befehle dieses Projekts:

Befehl	Syntax / Beispiel	Beschreibung
Umgebung aktivieren	`conda activate beehaven`	Projekt-Umgebung im Terminal nutzen
Notebook ausführen	`jupyter nbconvert --to notebook --execute --inplace 04_gold.ipynb`	Notebook ohne VS Code komplett ausführen
Azure-Kopien erzeugen	`python Privat/03_notebooks_ITS-Data/generator/make_synapse_notebooks.py`	Synapse-Kopien aus den lokalen Notebooks
Test der Azure-Kopien	`… make_synapse_notebooks.py --test <lokaler Lake-Ordner> --ziel <Ordner>`	Kopien lokal gegen einen Testordner laufen lassen
Pipeline-Namen	`pl_daily_processing` → `rohdaten_nach_bronze` → `bronze_nach_silver` → `silver_nach_gold` → `bronze_archivieren`	Master-Pipeline und Stufen in Azure

Anhang D – Quellen

- Microsoft Learn – Ausdrücke und Funktionen in Azure Data Factory: learn.microsoft.com/azure/data-factory/control-flow-expression-language-functions
- Microsoft Learn – Benutzerdefinierte Datums- und Zeitformate (.NET), hh vs. HH: learn.microsoft.com/dotnet/standard/base-types/custom-date-and-time-format-strings
- Microsoft Learn – Azure-Rollen für Blob-Daten (Storage Blob Data Reader/Contributor)
- Microsoft Learn – Upgrade Azure Blob Storage to Azure Data Lake Storage (einmalig, nicht umkehrbar): learn.microsoft.com/azure/storage/blobs/upgrade-to-data-lake-storage-gen2
- Microsoft Cloud Adoption Framework – Benennungskonvention und Tagging
- Kursunterlagen WBS Kapitel 9 (DP-900) und Unterricht vom 23.–25.09.2026
- Microsoft Learn – Transform data by running a Synapse notebook; Microsoft Spark Utilities (`runMultiple`); Namensregeln für Pipelines und Aktivitäten (Stand 28.–30.09.2026)
- Microsoft Learn – Azure Cost Management: Budgets und Warnungen
- Datenbasis Wetter: Deutscher Wetterdienst (über Bright Sky); Sonnenstand: NOAA Solar Calculation; Gegenprüfung: Open-Meteo (CC BY 4.0)
- Datensatz: Kaggle „Beehive Metrics“ (HOBOS, www.hobos.de) – Lizenz unbekannt, Rohdaten nicht veröffentlichen

Anhang E – Alternative Wege (was wir gewählt haben, was ginge auch, warum so)

Thema	gewählt	Alternativen	warum so
Speicher	Data Lake Gen2 (hierarchischer Namespace)	Blob-Speicher; Datenbank	echte Ordner, schneller für Spark, Rechte je Ordner
Redundanz	LRS (3 Kopien in einem Rechenzentrum)	ZRS (Zonen), GRS (zweite Region)	Lern- und Testprojekt, günstigste Stufe
Zugriffsebene	Hot (heiß)	Cool, Archive	tägliche Verarbeitung; Archive nur für selten gelesene Daten
Vorgehen	ELT (laden, dann umwandeln)	ETL (umwandeln vor dem Laden)	Rohdaten bleiben unverändert in Bronze
Verarbeitung	Synapse-Notebooks (Spark, Python)	Databricks, Microsoft Fabric, Data Flow, Azure Function	Kursvorgabe; Notebooks lokal entwickel- und testbar
Notebooks in der Pipeline	ein Steuer-Notebook (`runMultiple`) für N1–N3	eine Aktivität je Notebook	eine Spark-Sitzung statt drei (Startzeit, Kosten)
Gold je Stadt	Suche + ForEach + Notebook je Stadt	Gold-Steuer-Notebook für alle Städte; Stadt fest eintragen	übersichtlich bei wenigen Städten; ab ca. 3 Städten Steuer-Notebook
Anmeldung	verwaltete Identität	Kontoschlüssel, SAS-Token	kein Geheimnis im Code, Rechte über Rollen
Rechte Data Factory in Synapse	3 Rollen (Least Privilege)	Synapse Administrator, Contributor	nur die nötigen Rechte
Dateiformat	Parquet (Silver/Gold), CSV zum Ansehen	nur CSV; Delta Lake	Datentypen bleiben erhalten, 13–18 % der CSV-Größe
Start der Pipeline	von Hand (Debuggen); Trigger angelegt, aber deaktiviert	Zeitplan täglich 00:00 UTC; Ereignis-Trigger bei neuer Datei	Kosten nur bei Bedarf; Zeitplan im Kurs am 05.10.
Fehlerpfad	Dateien bleiben liegen, Pipeline rot	Fehlerordner mit Zeitstempel; E-Mail (Azure Monitor)	einfach und nachvollziehbar; Benachrichtigung für den Betrieb
Standortregeln	Konfigurationstabellen (`sensoren.csv`, `ausschluesse.csv`)	Sonderfälle im Code	neue Stadt = neue Zeilen, kein Code-Umbau

Anhang F – Begriffe (Glossar)

Begriff	Bedeutung
Abonnement (Subscription)	Abrechnungseinheit in Azure; alle Kosten laufen darüber
Ressourcengruppe (Resource Group)	Ordner für alle Azure-Teile eines Projekts
Speicherkonto (Storage Account)	Ablageort für Dateien in der Cloud
Data Lake Gen2	Speicherkonto mit echten Ordnern (hierarchischer Namespace), für Big Data
Medallion (Bronze/Silver/Gold)	Stufen der Datenqualität: roh – bereinigt – auswertbar
ETL / ELT	Extract, Transform, Load: umwandeln vor bzw. nach dem Laden
Data Factory (ADF)	Dienst, der Datenflüsse steuert (Orchestrierung)
Pipeline	Kette von Aktivitäten in der Data Factory
Aktivität (Activity)	einzelner Arbeitsschritt (Kopieren, Löschen, Notebook, Suche, ForEach …)
Linked Service (verknüpfter Dienst)	gespeicherte Verbindung zu einem Speicher oder Dienst
Dataset (Datensatz)	Zeiger auf Ordner/Datei mit Formatangabe
Parameter / dynamischer Inhalt	Platzhalter, der erst beim Lauf gefüllt wird (z. B. `@item().name`)
Trigger (Auslöser)	startet eine Pipeline nach Zeitplan oder Ereignis
Debuggen	Testlauf einer Pipeline im Entwurf, ohne Veröffentlichen
Veröffentlichen (Publish)	Entwurf wird gültige Fassung (Liveumgebung)
Synapse Analytics	Analyse-Arbeitsbereich mit Spark und SQL
Spark-Pool	Rechner-Gruppe, die Notebooks ausführt; kostet nur im Lauf
Spark-Sitzung (Session)	laufende Verbindung zum Pool; Start 3–4 Min.
Notebook	Dokument mit Code und Text, Zelle für Zelle ausführbar
Parameterzelle / Basisparameter	Standardwerte im Notebook / von der Pipeline übergebene Werte
RBAC / IAM	rollenbasierte Zugriffssteuerung / Identitäts- und Zugriffsverwaltung
Verwaltete Identität (Managed Identity)	Azure-Identität eines Dienstes, ohne Passwort
Least Privilege	Prinzip der geringsten Rechte
Parquet	spaltenorientiertes Dateiformat, klein und typsicher
UTC	Weltzeit ohne Sommerzeit; Grundlage aller Zeitstempel
Gegenprobe	Prüfung der Ablage im Data Lake nach einem „erfolgreichen“ Lauf
KiB / KB	1.024 Byte / 1.000 Byte